СИСТЕМА ЭЛЕКТРОННОЙ ПОДАЧИ ЗАЯВОК «БЮРО ПРОПУСКОВ»

КОНФИДЕНЦИАЛЬНО

РУКОВОДСТВО ПО РАЗВЁРТЫВАНИЮ И СОПРОВОЖДЕНИЮ

Установка, настройка и эксплуатация

Версия документа: 1.80

Дата: 10.11.2026

© 2026 Правообладатель. Все права защищены.

УСЛОВИЯ ИСПОЛЬЗОВАНИЯ ДОКУМЕНТА

Правообладатель: Правообладатель

1. Правообладатель. Исключительное право на настоящий документ и на описанную в нём систему, включая её исходный код и интерфейсы, принадлежит правообладателю. Передача документа не влечёт перехода к Получателю прав на систему или её составные части.

2. Передача третьим лицам. Документ, его части и производные материалы - копии, выдержки, переводы, презентации, снимки экрана - не передаются третьим лицам без согласия правообладателя. Внутри организации Получателя документ доводится до работников, которым он необходим для работы с системой.

3. Отсутствие лицензии. Документ описывает систему и не является офертой или лицензией на её использование. Право использования системы возникает на основании отдельного договора с правообладателем.

© 2026 Правообладатель. Все права защищены.

СОДЕРЖАНИЕ

1. Общие сведения.....7
1.1. Назначение документа.....7
1.2. Оформление.....7
1.3. Приложения.....7
2. Термины.....8
3. Требования к оборудованию и программному обеспечению.....9
3.1. Конфигурация сервера.....9
3.2. Дисковое пространство.....9
3.3. Программное обеспечение сервера.....10
3.4. Требования к рабочим местам.....11
3.5. Сетевые требования.....11
4. Состав поставки.....12
4.1. Что передаётся.....12
4.2. Структура каталога системы.....16
4.3. Принцип наложения файлов описания.....17
4.4. Сторонние компоненты и условия их использования.....18
4.5. Чего в поставке нет.....20
5. Установка.....20
5.1. Подготовка сервера.....20
5.2. Установка Docker.....25
5.3. Размещение поставки.....27
5.4. Создание файла параметров.....28
5.5. Выбор имени базы данных.....34
5.6. Подготовка сертификата.....34
5.7. Первый запуск.....35
5.8. Создание учётной записи администратора.....35
5.9. Проверка после установки.....36
5.10. Ограничение доступа к серверу.....37
6. Параметры конфигурации.....43
6.1. Как задаются параметры.....43

6.2. Обязательные параметры....45
6.3. Основные параметры....46
6.4. Параметры базы данных....47
6.5. Пул соединений, таймауты и проверка пароля....48
6.6. Пересчёт параметров под другую конфигурацию....51
6.7. Почтовые уведомления....52
7. Защищённое соединение и сертификат....58
7.1. Зачем это нужно....58
7.2. Выбор способа получения сертификата....58
7.3. Выпуск сертификата....59
7.4. Установка корневого сертификата на рабочие места....60
7.5. Продление сертификата....61
7.6. Сервер за внешним входящим прокси....63
8. Проверочный стенд....64
8.1. Назначение....64
8.2. Отличия от рабочего сервера....64
8.3. Развёртывание стенда....65
8.4. Наполнение стенда вымышленными данными....67
8.5. Ограничение доступа к стенду....69
Проверка защиты стенда в изолированной среде....72
9. Эксплуатация рабочего сервера....73
9.1. Основные команды....73
9.2. Повседневные операции....75
9.3. Управление учётными записями....75
9.4. Восстановление доступа супер-администратора....83
9.5. Режим технических работ....84
9.6. Файловый архив бланков....85
9.7. Защита каталога архива....93
9.8. Переименование базы данных....94
9.9. Очистка накопленных данных....95
9.10. Архивация, обезличивание и удаление по ID....99
9.11. Уход с системы....112

9.12. Ключ шифрования персональных данных....114
10. Порядок внесения изменений в систему....116
10.1. Общая схема....116
10.2. Получение исходного кода и внесение правки....116
10.3. Проверка изменения до выкладывания....117
10.4. Проверка на стенде....118
10.5. Выкладывание на рабочий сервер....119
10.6. Изменения схемы базы данных....121
10.7. Возврат к предыдущей версии....121
11. Резервное копирование и восстановление....123
11.1. Устройство....123
11.2. Что входит в копию....123
11.3. Сроки хранения....125
11.4. Настройка....125
11.5. Проверка и наблюдение....132
11.6. Хранение вне сервера....139
11.7. Восстановление....139
11.8. Перенос на другой сервер....147
12. Мониторинг и журналы....147
12.1. Проверка состояния....147
12.2. Проверка живости с оповещением на почту....148
12.3. Журналы приложения....151
12.4. Журналы внутри системы....153
12.5. Что контролировать....154
13. Диагностика и типовые неисправности....156
13.1. Порядок диагностики....156
13.2. Система недоступна, страница не открывается....156
13.3. Обратный прокси не запускается....157
13.4. Предупреждение о недоверенном сертификате....157
13.5. Прикладной сервер не запускается....158
13.6. Изменение параметра не подействовало....158
13.7. Заканчивается место на диске....158

13.8. Система работает медленно.....159
13.9. Работник не видит раздел или получает отказ.....160
13.10. Не приходят обновления без перезагрузки страницы.....161
13.11. Выгрузка из файлового архива не начинается или обрывается.....162
13.12. Бланки не появляются в каталоге архива.....162
13.13. Письма не приходят.....163
13.14. Система не требует смены паролей по истечении срока.....165
13.15. Правка файла настроек не подействовала.....167
13.16. Ошибка получения исходников после команд root.....168
14. Перенос системы на другой сервер.....169
14.1. Что переносится.....169
14.2. Что меняется при переносе.....170
14.3. Подготовка нового сервера.....170
14.4. Выгрузка данных со старого сервера.....170
14.5. Восстановление на новом сервере.....172
14.6. Сертификат и запуск.....173
14.7. Проверка после переноса.....174
14.8. Переключение и вывод прежнего сервера из работы.....174
Приложение А. Порты и сетевые доступы.....175
Приложение Б. Перечень параметров конфигурации.....177
Приложение В. Файлы, необходимые для запуска.....189
Приложение Г. Проверочный лист ввода в эксплуатацию.....191
Этап 1. Подготовка сервера.....192
Этап 2. Установка и первый запуск.....192
Этап 3. Обвязка и наблюдение.....193

1. Общие сведения

1.1. Назначение документа

Документ содержит порядок развёртывания системы электронной подачи заявок «Бюро пропусков», её настройки, ввода в эксплуатацию и последующего сопровождения.

Документ рассчитан на специалиста, не работавшего ранее с данной системой. Порядок действий описан от состояния «есть сервер с установленной операционной системой» до состояния «система принимает заявки». Команды приведены в виде, пригодном для непосредственного выполнения, с указанием ожидаемого результата каждого шага.

Назначение системы, устройство и состав возможностей описаны в отдельном документе, «Техническое описание системы».

1.2. Оформление

Команды, имена файлов и параметры набраны моноширинным шрифтом. Угловые скобки означают подстановку: `<домен>` заменяется фактическим значением без скобок.

Сведения, требующие повышенного внимания, выделяются:

> **ПРИМЕЧАНИЕ.** Пояснение, полезное для понимания, но не влияющее на порядок действий.

> **ВАЖНО.** Пропуск приведёт к неверной работе или к дополнительной работе по исправлению.

> **ОПАСНО.** Последствия необратимы. Утрата данных или утрата доступа к ним.

1.3. Приложения

Документ заканчивается четырьмя приложениями. Одно из них - рабочее: **приложение Г** это пошаговая инструкция-проверочный лист, по которой систему вводят в эксплуатацию от чистого сервера до принятой

работы. Если задача - установить и настроить систему, начинать удобно с него: шаги идут в порядке выполнения, а у каждого указан подраздел с командами и ожидаемым результатом.

Таблица 1.1 — Состав приложений

Приложение	Что содержит	Когда нужно
А. Порты и сетевые доступы	Перечень портов, направлений и назначений	При настройке межсетевого экрана и согласовании доступов
Б. Перечень параметров конфигурации	Все параметры файла `.env`: назначение, значение по умолчанию, обязательность	При создании файла параметров и при изменении настроек
В. Файлы, необходимые для запуска	Что входит в поставку, что создаётся сценариями, чего в поставке нет	При проверке состава поставки и при переносе на другой сервер
Г. Проверочный лист ввода в эксплуатацию	Пошаговая инструкция и приёмка: три этапа, 33 шага со ссылками на подразделы	При установке системы и при сдаче работы

2. Термины

Таблица 2.1 — Термины, применяемые в документе

Термин	Значение
Рабочий сервер	Сервер, на котором система работает для сотрудников предприятия. В обиходе «прод», «продакшн»
Проверочный стенд	Отдельный сервер с той же системой для проверки изменений до выкладывания на рабочий. В обиходе «стенд», «стейдж»
Контейнер	Изолированная среда, в которой запускается одна часть системы. Части не мешают друг другу и не требуют установки в операционную систему
Образ	Заготовка, из которой запускается контейнер. Собирается из исходного кода один раз, затем запускается сколько угодно раз
Том	Хранилище данных контейнера, живущее отдельно от него. Пересоздание контейнера данные не затрагивает
Обратный прокси	Часть системы, принимающая соединения снаружи и распределяющая их между остальными частями
Сертификат	Электронный документ, которым сервер подтверждает браузеру свою подлинность и обеспечивает защищённое соединение
Удостоверяющий центр	Сторона, подписывающая сертификаты. Браузер доверяет сертификату, если доверяет подписавшему его центру
Файл параметров	Файл `.env` в каталоге системы, содержащий пароли, ключи и настройки конкретной установки

3. Требования к оборудованию и программному обеспечению

3.1. Конфигурация сервера

Параметры базы данных и прикладного сервера в поставке подобраны для указанного оборудования. Это исходная конфигурация, а не подтверждённая нагрузочными испытаниями вместимость: допустимое число одновременных пользователей определяют на целевом оборудовании с характерными заявками, выгрузками и временем ответа.

Таблица 3.1 — Рекомендуемая конфигурация сервера

Параметр	Значение
Процессор	6 физических ядер с частотой от 3,4 ГГц, например Intel Xeon E-2236
Оперативная память	32 ГБ с коррекцией ошибок
Дисковая подсистема	Два твердотельных накопителя от 480 ГБ в зеркальном массиве
Сеть	Один адрес, пропускная способность от 100 Мбит/с

ВАЖНО. Зеркальный массив обязателен. В базе хранятся персональные данные, и при отказе одиночного накопителя восстановление возможно только из резервной копии, то есть с потерей всех изменений, сделанных после её создания.

Минимальная конфигурация для пробной эксплуатации и проверочного стенда: 4 ядра, 8 ГБ памяти, 100 ГБ дискового пространства. При такой конфигурации параметры базы данных из комплекта поставки требуется уменьшить, иначе службе не хватит памяти. Порядок изменения описан в подразделе 6.6.

3.2. Дисковое пространство

Таблица 3.2 — Ориентировочное потребление дискового пространства

Потребитель	Оценка
Образы контейнеров и операционная система	6-8 ГБ сразу после установки; 15-20 ГБ с запасом на кэш сборки, который накапливается при обновлениях
База данных без журналов	1-3 ГБ за год работы
Журнал запросов	Основной потребитель. При высокой активности подробные записи за 30 суток занимают десятки гигабайт
Загруженные файлы	Фотографии, шаблоны бланков, приложения к заявкам
Резервные копии	Не менее трёх объёмов базы данных

Журнал запросов растёт быстрее остального, поэтому глубина хранения подробных записей вынесена в параметр `REQUEST_LOG_DETAIL_DAYS`. Уменьшение до 7-14 суток кратно снижает занимаемый объём и на работу системы не влияет.

Служебные записи, обесценивающиеся сами, система удаляет ежесуточно без участия человека. Историю, слепки таблиц и суточные итоги журнала при нехватке места удаляют командой из подраздела 9.9 «Очистка накопленных данных».

3.3. Программное обеспечение сервера

Таблица 3.3 — Требования к программному обеспечению

Компонент	Требование
Операционная система	Любой дистрибутив Linux с поддержкой контейнеров: Ubuntu Server 22.04 и новее, Debian 12 и новее, RHEL-совместимые (Rocky, Alma), Astra Linux, РЕД ОС
Docker Engine	Версия 25 и новее
Docker Compose	Версия 2.24 и новее, вызывается как `docker compose`
Сборщик BuildKit (buildx)	Плагин `docker-buildx`. Без него сборка образа веб-приложения прекращается сообщением `the --mount option requires BuildKit`
git	Получение исходного кода, обновление и возврат к предыдущей версии (разделы 4.1, 5.3, 10)
openssl	Генерация секретов и сертификата
age	Ключи шифрования резервных копий и файлового архива. Без него ключи создаются одноразовым контейнером, и тогда сценарию подготовки параметров требуется доступ к Docker, подраздел 5.4
ufw либо firewalld	Ограничение доступа к серверу, подраздел 5.10. В минимальных сборках дистрибутива отсутствует
certbot	Только при выпуске сертификата через общедоступный удостоверяющий центр

Отдельно устанавливать Go, Node.js, PostgreSQL или nginx не требуется: всё необходимое собирается и работает внутри контейнеров.

ВАЖНО. Наличие Docker и Docker Compose ещё не означает, что поставка соберётся. Сборка образа веб-приложения использует кэш пакетного менеджера и требует сборщика BuildKit, который поставляется отдельным плагином. Если плагина нет, `docker compose build` молча переключается на устаревший сборщик и прекращает работу на установке зависимостей интерфейса. Проверка - в подразделе 5.2.

3.4. Требования к рабочим местам

Браузер актуальной версии: Chrome, Yandex Browser, Firefox, Edge. Разрешение экрана от 1280 точек по ширине для рабочих мест бюро и постов. Интерфейс приспособлен и к мобильным устройствам.

Дополнительная установка чего-либо на рабочие места не требуется, за одним исключением: если сертификат сервера выпущен собственным удостоверяющим центром организации, потребуется однократно установить его корневой сертификат. Подробнее в подразделе 7.4.

3.5. Сетевые требования

Наружу открываются только два порта. Остальные порты частей системы существуют исключительно во внутренней сети контейнеров и не должны быть доступны с рабочих мест.

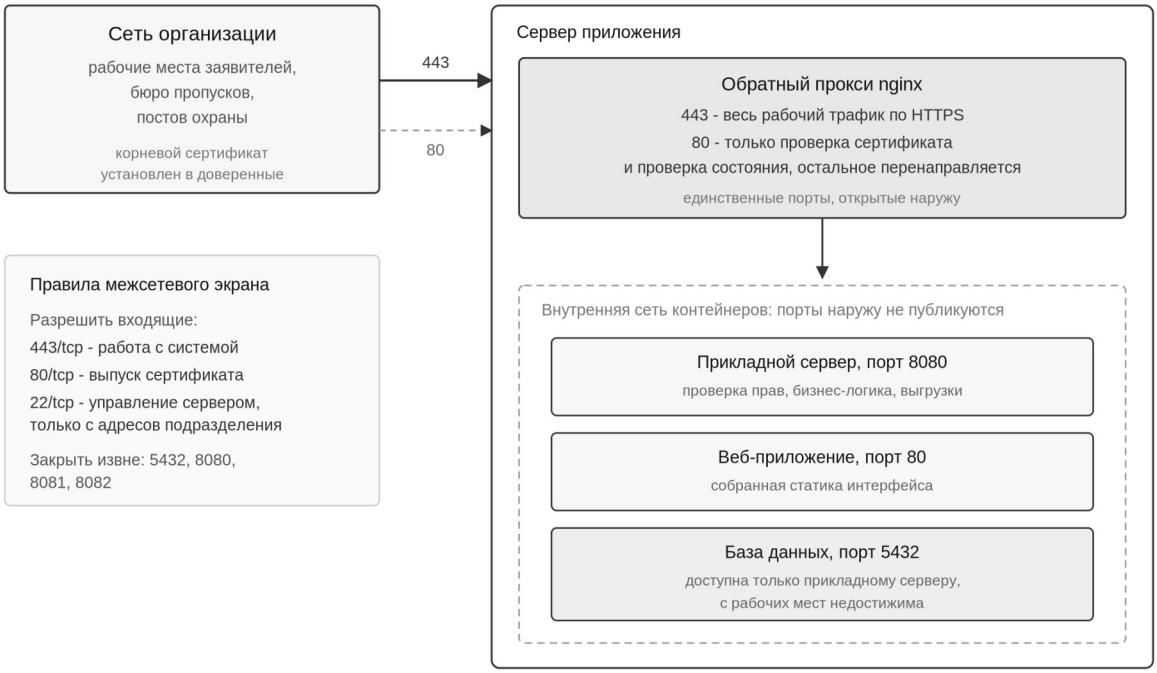


Рисунок 3.1 — Разграничение сетевых доступов

Исходящее соединение с сервера требуется, если настроена отправка писем: прикладной сервер подключается к почтовому серверу по адресу и порту из параметров `SMTP_HOST` и `SMTP_PORT`, обычно это порт 587 или 465. Соединение открывается в сторону почтового сервера, входящих

подключений оно не создаёт. Если правила межсетевого экрана его не пропускают, отправка заканчивается истечением срока ожидания, а письма остаются в очереди. Подробнее в подразделе 6.7.

Полный перечень портов с указанием области доступности приведён в приложении А.

4. Состав поставки

4.1. Что передаётся

Передаётся исходный код системы и файлы для сборки и запуска на совместимой площадке. Для автоматического обновления используются готовые образы из GHCR с меткой полного SHA коммита: они проходят проверку до публикации и не пересобираются на целевом сервере. Ручная сборка из исходников остаётся отдельным вариантом установки и сопровождения; её результат нельзя выдавать за образ, уже просканированный в CI.

Таблица 4.1 — Состав поставки

Элемент	**Назначение**
Исходный код серверной части, интерфейса и образы окружения	Позволяет повторить сборку; автоматическая поставка получает готовые образы проверенной ревизии
Файлы описания контейнеров	Определяют состав частей системы, их связи, ограничения ресурсов
Конфигурация обратного прокси	Правила распределения запросов, защищённое соединение, заголовки безопасности
Сборочный файл	Набор коротких команд управления вместо длинных команд Docker
Сценарий подготовки параметров	Создаёт файл параметров и генерирует пароли и ключи
Сценарий подготовки сертификата	Выпуск и продление сертификата, три способа на выбор
Сценарии резервного копирования и восстановления	Снятие копии, расписание, состояние, проверка восстановимости, восстановление
Сценарии проверки живости	Проверка сайта, входа и базы с оповещением на почту, постановка её на расписание
Сценарий сборки архива поставки	Снятие архива с исходным кодом для площадки без доступа к репозиторию

Продолжение таблицы 4.1

Элемент	Назначение
Перечень сторонних компонентов	Условия использования библиотек с открытым исходным кодом и уведомления об авторских правах, см. подраздел 4.4
Эксплуатационная документация	Настоящий документ, техническое описание и руководства пользователей

Перечень конкретных файлов, без которых система не запустится, приведён в приложении В.

Как передаётся код. Исходный код хранится в системе контроля версий Git, в репозитории `git@github.com:buropropuskov/systemburo.git` на площадке GitHub. Видимость репозитория определяет его владелец; она не ограничивает доступ к данным работающей установки.

И рабочий сервер, и проверочный стенд получают код прямо из репозитория. При установке каталог системы создаётся командой `git clone` (подраздел 5.3), при обновлении и при возврате к предыдущей версии тот же каталог переключается на нужную версию (подразделы 10.5 и 10.7). Обе операции требуют, чтобы каталог оставался рабочей копией репозитория, то есть содержал служебный подкаталог `.git`.

Как выдаётся доступ. Доступ выдаёт владелец репозитория со стороны разработчика по заявке ответственного за сервер. Способов два, и выбирают один.

Таблица 4.2 — Способы доступа к репозиторию

Способ	Что это	Когда применяется
Ключ развёртывания	Открытый ключ SSH, привязанный к одному репозиторию и по умолчанию действующий только на чтение	Доступ для сервера. Ничего, кроме этого репозитория, ключ не открывает
Машинная учётная запись	Отдельная учётная запись GitHub, включённая в организацию с правом чтения	Когда одному серверу нужны несколько репозиториев сразу

Для сервера предпочтителен ключ развёртывания. Он ограничен одним репозиторием и не связан с личной учётной записью сотрудника, поэтому

увольнение никого из работников доступ сервера не рвёт, а сам ключ, попав в чужие руки, не даёт ничего сверх чтения этого репозитория.

Пара ключей создаётся на самом сервере, от того пользователя, под которым работает система. Наружу передаётся только открытая часть.

Листинг 4.1 — Создание ключа доступа к репозиторию

```
ssh-keygen -t ed25519 -C "systemburo-prod" -f ~/.ssh/id_ed25519 -N ""
cat ~/.ssh/id_ed25519.pub
```

Ожидаемый результат: выведена одна строка, начинающаяся с `ssh-ed25519`. Её передают владельцу репозитория, который добавляет её в настройки репозитория как ключ развёртывания. Закрытая часть остаётся на сервере и никуда не пересылается: она такой же секрет, как содержимое файла параметров, и на неё распространяется правило хранения секретов из подраздела 5.4.

Листинг 4.2 — Проверка доступа

```
ssh -T git@github.com
```

Ожидаемый результат: приветствие с именем учётной записи или сообщение о том, что доступ к оболочке не предоставляется. Отказ по правам означает, что ключ ещё не добавлен либо добавлен не в тот репозиторий.

Как отзывается доступ. Ключ развёртывания отзывается удалением из настроек репозитория, машинная учётная запись - исключением из организации. Отзыв действует сразу: сервер перестаёт получать обновления, но уже полученный код и работающая система не затрагиваются. Доступ отзывают при выводе сервера из работы, при смене подрядчика и при любом подозрении, что закрытая часть ключа стала известна посторонним. После отзыва ключ на сервере удаляют, иначе команды раздела 10 будут завершаться отказом по правам без внятного объяснения.

Площадка без доступа в сеть общего пользования. Если сервер по требованиям безопасности не имеет выхода наружу, доступ к GitHub невозможен, и код передаётся архивом того же содержимого. Порядок установки от этого не меняется, кроме подраздела 5.3, где вместо получения

из репозитория архив распаковывается в рабочий каталог. Следствие одно, но существенное: каталог перестаёт быть рабочей копией репозитория, поэтому команды обновления и возврата к предыдущей версии из подразделов 10.5 и 10.7 на нём не выполняются. Порядок обновления такой установки описан в подразделе 10.5 отдельно.

Архив снимается на стороне разработчика одной командой.

Листинг 4.3 — Сборка архива поставки

```
make package
make package VERSION=1.2.0
```

Ожидаемый результат: файл `dist/systemburo-<версия>.tar.gz`, а под ним - версия с определителем и датой коммита, размер и число вошедших файлов. Без явной версии берётся последняя метка репозитория, а если меток нет - дата коммита и его короткий определитель.

Архив собирается не из рабочего каталога, а из зафиксированного в репозитории состояния. Отсюда два свойства. Первое: один и тот же коммит даёт один и тот же архив, поэтому поставку можно сверить с исходником. Второе, более важное: в архив физически не может попасть то, чего в репозитории нет, - файл параметров `.env` с паролем базы, ключами подписи маркеров и ключом шифрования персональных данных, загруженные файлы, собранные образы, рабочие каталоги. Собранный архив дополнительно просматривается на эти имена, и находка прекращает сборку с ошибкой, а не с предупреждением. Незакоммиченные правки в архив не попадают, о чём команда предупреждает отдельной строкой.

Не входят в архив и каталоги, отношения к установке не имеющие: описание конвейера сборки на стороне разработчика и нагрузочные сценарии.

ВАЖНО. Смешивать способы нельзя. Если система развёрнута из архива, а затем в тот же каталог выполнить `git clone`, каталог окажется непустым и команда завершится ошибкой. Переход с архива на репозиторий выполняется как перенос на другой сервер, см. раздел 14.

4.2. Структура каталога системы

После получения исходного кода в рабочем каталоге образуется следующая структура. Знать её требуется для изменения настроек и для диагностики.

Листинг 4.4 — Структура каталога системы

```
/opt/systemburo/
├── docker-compose.base.yml     состав служб, общая часть
├── docker-compose.prod.yml     наложение: рабочий сервер
├── docker-compose.staging.yml  наложение: стенд
├── docker-compose.yml          локальная разработка
├── Dockerfile                  образ прикладного сервера
├── Makefile                    команды управления
├── THIRD-PARTY-NOTICES.md      условия использования сторонних библиотек
├── .env                        параметры и секреты
├── .git/                       рабочая копия репозитория
├── cmd/ internal/ docs/        код серверной части
├── frontend/
│   ├── Dockerfile              образ веб-приложения
│   ├── nginx.conf              отдача статики в образе
│   └── src/ package.json       код интерфейса
├── nginx/
│   ├── nginx.conf              прокси рабочего сервера
│   ├── staging.conf            прокси стенда
│   ├── Dockerfile.staging      образ прокси для стенда
│   ├── certs/                  сертификат и ключ
│   └── acme-webroot/           выпуск сертификата
└── scripts/
    ├── init-env.sh             создание файла параметров
    ├── init-tls.sh             выпуск и продление сертификата
    ├── maintenance-off.sh      аварийное снятие режима технических работ
    ├── backup.sh               снятие резервной копии
    ├── backup-install.sh       постановка копирования на расписание
    ├── backup-install_test.sh  самопроверка выравнивания прав каталога копий
    ├── backup-status.sh        состояние копий и итог последнего запуска
    ├── backup-verify.sh        проверка восстановимости копии
    ├── restore.sh              восстановление из копии
    ├── health-check.sh         проверка живости с оповещением на почту
    ├── health-install.sh       постановка проверки живости на расписание
    ├── package.sh              сборка архива поставки
    └── gen-third-party-notices.py  пересборка перечня сторонних компонентов
```

Пять сценариев копирования составляют один механизм, описанный в разделе 11: постановка на расписание берёт каталог копий из файла параметров, состояние читает то, что записал сценарий копирования, а восстановление работает с созданными им файлами. Убирать из каталога какой-либо из них по отдельности нельзя. Стоящий рядом `backup-install_test.sh` в механизм не входит - это самопроверка установки, см. подраздел 11.4. Так же связаны два сценария проверки живости, подраздел 12.2. Последний в перечне, `gen-third-party-notices.py`, к эксплуатации

отношения не имеет: он пересобирает перечень сторонних компонентов при обновлении библиотек и выполняется на стороне разработчика, см. подраздел 4.4.

Каталог `.git` присутствует только при установке из репозитория. На площадке без доступа к нему, где код распакован из архива, каталога нет, и это меняет порядок обновления, см. подраздел 4.1.

4.3. Принцип наложения файлов описания

В поставке используются собственные конечные образы PostgreSQL и pgAdmin из каталогов `postgres/` и `pgadmin/`. Все четыре варианта Compose используют сборку PostgreSQL; pgAdmin предусмотрен только для локального запуска и стенда. При автоматической поставке `POSTGRES_IMAGE` и `PGADMIN_IMAGE` выбирают образы точного SHA; при отсутствии переменных сохраняются локальные имена и описание сборки. Это необязательные переменные выбора образа, а не новые обязательные секреты файла параметров.

Ни рабочий сервер, ни стенд не имеют самостоятельного файла описания. Состав служб задан один раз в базовом файле, а различия среды добавляются наложением поверх него.

Листинг 4.5 — Порядок указания файлов описания

```
sudo docker compose -f docker-compose.base.yml -f docker-compose.prod.yml \
  ps
```

Именно поэтому во всех командах настоящего документа указываются два файла подряд. Порядок существенен: базовый идёт первым, наложение вторым, значения из второго заменяют значения из первого.

ВАЖНО. Команда, в которой указан только файл наложения, работать не будет. Служба базы данных определена в базовом файле, и без него Docker сообщит, что такой службы не существует. Это частая причина ошибок при копировании команд из посторонних источников.

Сокращённые команды из сборочного файла (`make deploy-up`, `make deploy-logs` и прочие) подставляют оба файла самостоятельно, поэтому при их использовании об этом думать не требуется.

Передача запросов от nginx прикладному серверу использует дополнительную сеть Docker `backend_proxy`. В ней nginx имеет адрес с окончанием `.2`, а прикладной сервер — `.3`. Состав файлов Compose автоматически согласует эти адреса с доверием к входящему прокси. Для прямого подключения посетителей к ВМ заполнять `EDGE_TRUSTED_PROXY_CIDRS` не требуется.

Если перед nginx действительно работает ещё один входящий прокси, в `EDGE_TRUSTED_PROXY_CIDRS` задают только его подтверждённые адреса с длиной префикса, через запятую. Адрес не выводят из заголовков произвольного запроса и не заменяют доверием всей частной сети. После настройки проверяют адрес посетителя в журнале на обычном обращении через рабочий маршрут. Конкретные значения зависят от площадки установки.

Прежде чем размещать несколько установок на одном Docker-сервере, проверяют отсутствие пересечения их сетей. При необходимости каждой установке задают свободный `PROXY_NETWORK_PREFIX`. Маска сети составляет `/29`; пересечение с занятой сетью приводит к ошибке запуска и требует выбора другого префикса, а не отключения проверки адреса посетителя.

4.4. Сторонние компоненты и условия их использования

Система собрана с использованием библиотек с открытым исходным кодом: они дают работу с базой данных, формирование файлов PDF и Excel, отображение интерфейса в браузере и ещё несколько десятков задач, решать которые заново не имело смысла.

Лицензии таких библиотек (MIT, Apache-2.0, BSD, ISC и близкие к ним) разрешают применение, изменение и распространение и не требуют ни оплаты, ни раскрытия кода системы. Условие у них одно и общее:

уведомление об авторских правах и текст лицензии должны сохраняться при передаче программы дальше. Это обязательство переходит вместе с поставкой, поэтому оно описано здесь, а не оставлено на стороне разработчика.

Условие выполняет файл `THIRD-PARTY-NOTICES.md` в корне каталога системы. В нём перечислены все сторонние компоненты с версиями, названиями лицензий и правообладателями и приведены дословные тексты лицензий.

Разрешительными оказываются не все. Вторым разделом файла идут компоненты, условия которых сохранением уведомления не исчерпываются, - их читают отдельно, а не считают разрешительными по умолчанию. Раздел этот короткий и стоит первым намеренно: потеряться среди двух сотен строк с одинаковым «MIT» он не должен.

Из этого следуют два практических правила. Файл не удаляют из каталога системы и не заменяют сокращённым пересказом. Если поставка передаётся дальше - на другой сервер, в другое подразделение, стороннему подрядчику, - файл передаётся вместе с ней.

Перечень собран из репозитория, а не составлен вручную: состав интерфейса взят из файла замка зависимостей, состав серверной части - из модулей, которые попадают в собранные двоичные файлы. Пересобирается он сценарием `scripts/gen-third-party-notices.py` при обновлении библиотек. Выполняется сценарий на стороне разработчика, на сервере он не нужен; расхождение перечня с составом зависимостей останавливает проверки при сборке, поэтому устареть незамеченным он не может.

Отдельно стоит шрифт `DejaVuSans.ttf`, которым система формирует файлы PDF. Его условия требуют, чтобы лицензия сопровождала сам файл шрифта, а не только общий перечень, поэтому она лежит рядом с ним - `internal/export/fonts/LICENSE`. При уборке каталога этот файл не удаляют: без него условие перестаёт выполняться, хотя шрифт остаётся на месте.

4.5. Чего в поставке нет

В поставку намеренно не входят пароли, ключи шифрования и сертификаты. Они создаются на сервере при установке и остаются только у организации, где развёрнута система. Это означает, что разработчик не имеет доступа к данным работающей системы.

Не входят и данные: справочники, учётные записи, заявки. Система разворачивается пустой, после чего заполняется силами бюро пропусков через интерфейс.

5. Установка

Раздел описывает установку с нуля на сервер с установленной операционной системой. Результат: работающая система, доступная по защищённому соединению, с заведённой учётной записью администратора.

Команды выполняются от имени отдельного пользователя, заведённого для работы с системой: обращения к контейнерам и цели сборочного файла он выполняет через `sudo` с вводом пароля. Порядок описан в подразделе 5.1. Настройка самого сервера - установка пакетов, межсетевой экран, служба удалённого доступа, расписание резервного копирования - требует полных прав суперпользователя и выполняется при вводе сервера в работу.

5.1. Подготовка сервера

Установите служебные утилиты.

Листинг 5.1 — Установка служебных пакетов

```
# Ubuntu, Debian, Astra Linux
sudo apt update
sudo apt install -y openssl curl ca-certificates git age ufw

# RHEL-совместимые, РЕД ОС
sudo dnf install -y openssl curl ca-certificates git age firewalld
```

Ожидаемый результат: пакеты установлены, `git --version` и `openssl version` выводят номера версий. Средство `age` в части дистрибутивов

отсутствует в репозиториях; это не препятствие, но тогда сценарий подготовки параметров запускается с `sudo`, см. подраздел 5.4.

> **ВАЖНО.** Обновление списка пакетов и установка выполняются двумя отдельными командами, а не одной цепочкой через `&&`. В образах, подготовленных поставщиками услуг, нередко подключены сторонние репозитории - например, для интерпретаторов языков программирования, - и если ключ такого репозитория в системе отсутствует, обновление списка заканчивается ошибкой `The repository ... is not signed` и ненулевым кодом возврата. В цепочке через `&&` установка при этом не выполняется вовсе, а сообщение об ошибке относится к постороннему репозиторию и на пропущенную установку не указывает. Убедитесь, что пакеты действительно встали, прежде чем идти дальше.

Задайте часовой пояс: от него зависят отметки времени в журналах.

Листинг 5.2 — Установка часового пояса

```
sudo timedatectl set-timezone Europe/Moscow
```

Учётная запись для работы с системой. Создайте отдельного пользователя. Под учётной записью суперпользователя систему не устанавливают и не сопровождают: ошибочная команда там ничем не ограничена, а в журнале сервера не остаётся имени того, кто её выполнил.

Листинг 5.3 — Создание пользователя

```
sudo useradd -m -s /bin/bash buro
sudo passwd buro
```

Ожидаемый результат: пользователь заведён, пароль запрошен дважды и принят. Пароль потребуется при каждом обращении к системе, поэтому он должен быть известен тому, кто её сопровождает, и храниться по правилу из подраздела 5.4.

Почему не группа `docker`. В эту группу пользователя не добавляют. Членство в ней равносильно правам суперпользователя: служба Docker работает от суперпользователя, а член группы обращается к ней напрямую, без пароля и без записи в журнале. Достаточно запустить контейнер, подключив к нему корневой каталог сервера, чтобы получить полный доступ

к машине: к базе данных, к файлу параметров с ключом шифрования персональных данных и к резервным копиям.

Вместо членства в группе выдаётся разрешение выполнять две команды через `sudo`. Правила записываются отдельным файлом в каталоге `/etc/sudoers.d/`, а не правкой основного файла: отдельный файл переживает обновление системы, и ошибка в нём не ломает разбор правил целиком.

Листинг 5.4 — Создание файла правил

```
sudo visudo -f /etc/sudoers.d/systemburo
```

Листинг 5.5 — Содержимое файла /etc/sudoers.d/systemburo

```
buro ALL=(root) /usr/bin/docker, /usr/bin/make
```

Редактор `visudo` проверяет запись при сохранении и не даёт сохранить файл с ошибкой; он же выставляет права 440. Файл правил, доступный на запись кому-либо помимо суперпользователя, `sudo` не принимает и отказывается работать вовсе.

Листинг 5.6 — Проверка правил

```
sudo visudo -c
```

Ожидаемый результат: строка `/etc/sudoers: parsed OK` и такая же строка для `/etc/sudoers.d/systemburo`. Ошибка называется вместе с номером строки, в которой она найдена.

Правило сверяет полный путь исполняемого файла, поэтому пути в нём должны совпадать с фактическими. Проверить это можно после установки Docker, подраздел 5.2.

Листинг 5.7 — Проверка путей и действующих разрешений

```
command -v docker
command -v make
sudo -l -U buro
```

Ожидаемый результат: первые две команды выводят `/usr/bin/docker` и `/usr/bin/make`; если пути другие, подставьте фактические в файл правил.

Третья печатает перечень разрешённого пользователю `buro` - те же две команды. Пустой перечень означает, что файл правил не подхвачен: проверьте его имя, точки в нём быть не должно, такие файлы `sudo` пропускает.

Каталог установки. Каталог системы принадлежит этому пользователю: под ним получают исходный код, обновляют его и правят файлы описания. На новой установке каталог создаётся сразу с нужным владельцем в подразделе 5.3, и передавать ничего не требуется.

Передача нужна, когда система уже работает, а модель доступа приводится к описанной здесь. Каталог файлового архива при этом не трогают: бланки в нём принадлежат служебному пользователю прикладного сервера с идентификатором 1001, и смена владельца остановит запись, см. подраздел 9.6.

Листинг 5.8 — Передача каталога установки, минуя каталог архива

```
sudo find /opt/systemburo -path /opt/systemburo/archive -prune \
  -o -exec chown buro:buro {} +
ls -ld /opt/systemburo /opt/systemburo/archive
```

Ожидаемый результат: у каталога системы владелец `buro`, у каталога архива - `1001`. В установке по умолчанию архив рабочего сервера лежит вне каталога системы, и тогда исключение ни на что не влияет.

Исключение для системы контроля версий. Git отказывается работать в каталоге, принадлежащем другому пользователю. После смены владельца суперпользователь на первой же команде `git` получает отказ со словами `detected dubious ownership`, и обновление системы останавливается на этом месте. Исключение добавляется обеим учётным записям - и той, под которой ведут повседневную работу, и суперпользователю.

Листинг 5.9 — Исключение каталога установки

```
git config --global --add safe.directory /opt/systemburo
sudo git config --global --add safe.directory /opt/systemburo
```

Ожидаемый результат: обе команды завершаются молча, `git -C /opt/systemburo log -1` выводит последнее изменение и под тем, и под другим пользователем.

Как выглядит повседневная работа. На сервер заходят под `buro`. Команды к системе - запуск, остановка, журналы, резервные копии - выполняются с `sudo`: на первую за сеанс запрашивается пароль, следующие четверть часа он не спрашивается.

Листинг 5.10 — Обычное обращение к системе

```
sudo make deploy-logs
```

Ожидаемый результат: запрошен пароль пользователя `buro`, затем выведен журнал. Каждое такое обращение записывается: в Ubuntu, Debian и Astra Linux - в `/var/log/auth.log`, в RHEL-совместимых и в РЕД ОС - в `/var/log/secure`. В записи имя пользователя, время, рабочий каталог и полная команда.

Работы по самому серверу двумя разрешёнными командами не покрываются и выполняются с полными правами суперпользователя. Они разовые: установка пакетов, межсетевой экран, служба удалённого доступа, расписание резервного копирования, правка файлов описания служб.

Чего эта модель не даёт. Сказанное ниже важно понимать до того, как её примут за преграду для злоумышленника.

ВАЖНО. Разрешение выполнять `docker` через `sudo` в конечном счёте равнозначно правам суперпользователя. Контейнер запускает служба, работающая от суперпользователя, и запускающий вправе подключить внутрь любой каталог сервера, включая корневой. Тот, кто вошёл под `buro` и знает его пароль, полный доступ к машине получит, и правила `sudo` его не остановят. Это свойство Docker, а не недоработка системы: любой доступ к его сокету равнозначен правам суперпользователя, и членство в группе `docker` отличается от описанной модели лишь тем, что не спрашивает пароль и не оставляет записи в журнале.

Ценность настройки в другом, и она невелика, но реальна. Повседневная работа не ведётся под суперпользователем, поэтому

ошибочная команда - не в том каталоге, не с тем ключом - останавливается на запросе пароля, а не выполняется молча. Обращения к системе видны в журнале: известно, кто и когда их делал. Перечень из двух команд отделяет работу с системой от работы с сервером: переустановить пакеты или поправить настройки службы удалённого доступа под этой учётной записью не выйдет, для этого приходится осознанно брать полные права.

Защита от того, кто добрался до самой учётной записи `buro`, обеспечивается не здесь, а тем, что описано в подразделе 5.10: закрытыми портами, входом по ключу вместо пароля и блокировкой подбора.

5.2. Установка Docker

Версия из штатных репозиториев дистрибутива нередко устарела и может не поддерживать применяемый формат файлов описания. Устанавливайте из репозитория разработчика.

Листинг 5.11 — Установка Docker

```
# Ubuntu, Debian
curl -fsSL https://get.docker.com | sudo sh

# RHEL-совместимые
sudo dnf config-manager --add-repo \
  https://download.docker.com/linux/centos/docker-ce.repo
sudo dnf install -y docker-ce docker-ce-cli containerd.io \
  docker-compose-plugin docker-buildx-plugin
sudo systemctl enable --now docker
```

Для Astra Linux и РЕД ОС используйте пакеты из репозитория дистрибутива, если политика организации запрещает подключение внешних репозиториев.

Если Docker уже установлен. В образах, подготовленных поставщиками услуг, Docker нередко присутствует изначально. Переустанавливать его не требуется, но и полагаться на то, что раз он есть, то подойдёт, нельзя: проверьте версии и наличие сборщика, как показано ниже. Версия из образа бывает и новее требуемой, и старее, а плагин сборщика в такие образы кладут не всегда.

Проверьте установку.

Листинг 5.12 — Проверка установки Docker

```
docker --version
docker compose version
docker buildx version
```

Ожидаемый результат: три номера версии, у Docker Compose не ниже 2.24. Вторая команда обязана вызываться именно как `docker compose`, через пробел. Если она сообщает об ошибке, а работает `docker-compose` через дефис, установлена устаревшая версия, и файлы описания системы обработаны не будут. Третья команда отвечает версией сборщика; сообщение `docker: 'buildx' is not a docker command` означает, что плагин не установлен.

Листинг 5.13 — Установка сборщика, если его нет

```
# Ubuntu, Debian
sudo apt install -y docker-buildx-plugin

# RHEL-совместимые
sudo dnf install -y docker-buildx-plugin
```

Ожидаемый результат: `docker buildx version` выводит номер версии. Проверить наличие плагина можно и по каталогу: `ls /usr/libexec/docker/cli-plugins/` - в нём должны лежать и `docker-compose`, и `docker-buildx`.

ВАЖНО. Версия Docker Compose ниже 2.24 не понимает применяемый в описании рабочего сервера синтаксис снятия ограничений памяти и прервёт запуск сообщением о неверном формате файла. Версия из штатного репозитория дистрибутива нередко оказывается старее.

ВАЖНО. Без плагина сборщика установка доходит до сборки образов и прекращается на установке зависимостей интерфейса сообщением `the --mount option requires BuildKit`. Образ серверной части при этом успевает собраться, поэтому по началу вывода сборка выглядит идущей нормально. Проверять наличие сборщика нужно здесь, до подраздела 5.7, а не разбираться в причинах на середине сборки.

5.3. Размещение поставки

Код получают из репозитория. Доступ к нему должен быть выдан заранее, порядок - в подразделе 4.1.

Листинг 5.14 — Получение исходного кода

```
sudo mkdir -p /opt/systemburo
sudo chown buro:buro /opt/systemburo
git clone git@github.com:buropropuskov/systemburo.git /opt/systemburo
cd /opt/systemburo
```

Ожидаемый результат: каталог заполнен, команда `git log -1` выводит последнее изменение. Отказ по правам означает, что ключ доступа не выдан или выдан не для этого сервера, см. подраздел 4.1.

ВАЖНО. Первые две строки выполняет суперпользователь при вводе сервера в работу, а не оператор сопровождения. Разрешение, выданное учётной записи `buro` в подразделе 5.1, покрывает только `docker` и `make`, поэтому под ней `sudo mkdir` и `sudo chown` завершаются отказом `user buro is not allowed to execute`. Само получение кода и все дальнейшие команды выполняются уже под `buro`: каталог к этому моменту принадлежит ей.

ПРИМЕЧАНИЕ. Команда `git clone` требует, чтобы каталог назначения был пустым. Каталог, созданный первыми двумя строками, пуст, поэтому порядок выполняется как есть.

Выбор версии. Получение кода без дополнительных указаний даёт ветку разработки - ту, которая в репозитории назначена основной. Для рабочего сервера так оставлять нельзя: в ветке разработки лежит незавершённая работа. Сразу после получения кода переключитесь на выпущенную версию, тем же порядком, что при обновлении (подраздел 10.5).

Листинг 5.15 — Переключение на выпущенную версию

```
git tag --sort=-creatordate | head -5
git checkout <версия>
git log -1 --oneline
```

Ожидаемый результат: первая команда печатает несколько последних версий, последняя - изменение, соответствующее выбранной. Какую версию

ставить, сообщает разработчик при передаче поставки. На площадке без доступа к репозиторию версия определяется самим архивом, и переключаться не требуется.

ПРИМЕЧАНИЕ. Если первая команда не печатает ничего, меток версий в репозитории нет - разработчик их не расставлял. Переключаться тогда некуда: остаётся основная ветка, полученная при клонировании, и она и есть передаваемая поставка. Запишите в проверочный лист определитель изменения (`git log -1 --oneline`) - именно он в этом случае и обозначает, что установлено. Спросите у разработчика, какое изменение считать переданным: с непомеченной ветки можно случайно получить работу, которую ещё не сдавали.

На площадке без доступа в сеть общего пользования код передан архивом, снятым командой `make package` на стороне разработчика, подраздел 4.1. Вместо получения из репозитория он распаковывается в тот же каталог. Файла параметров в архиве нет и быть не может, поэтому следующий подраздел выполняется и на такой установке.

Листинг 5.16 — Распаковка поставки на площадке без доступа к репозиторию

```
sudo mkdir -p /opt/systemburo
sudo chown buro:buro /opt/systemburo
tar -xzf systemburo-<версия>.tar.gz -C /opt/systemburo --strip-components=1
cd /opt/systemburo
```

Все дальнейшие команды выполняются из этого каталога.

ВАЖНО. Имя каталога определяет имена томов с данными. Если каталог впоследствии переименовать, Docker посчитает установку новой и создаст пустые тома, а система запустится с пустой базой. Данные при этом не теряются, а от самой ситуации избавляет параметр `COMPOSE_PROJECT_NAME`: сценарий подготовки параметров задаёт ему значение `systemburo`, и имена томов перестают зависеть от имени каталога. Если файл параметров заполнялся вручную, проверьте, что эта строка в нём есть, и добавьте её до первого запуска.

5.4. Создание файла параметров

Файл параметров создаётся сценарием, который сам генерирует стойкие случайные пароли и ключи. Задавать их вручную не следует.

Листинг 5.17 — Создание файла параметров

```
make init-production DOMAIN=<адрес системы>            # если установлен age
sudo make init-production DOMAIN=<адрес системы>       # если age нет
```

Вместо `<адрес системы>` подставьте доменное имя, по которому система будет доступна, либо её сетевой адрес.

Ожидаемый результат: создан файл `.env`, на экран выведены путь к нему, фактические права доступа и перечень созданных секретов - по именам, без самих значений.

ВАЖНО. Какую из двух строк выполнять, определяет наличие средства `age`. Без него пара ключей шифрования файлового архива создаётся одноразовым контейнером, а обращение к Docker выдано только через `sudo`, подраздел 5.1. Запущенный без `sudo` на такой машине сценарий сообщает `permission denied while trying to connect to the docker API`, останавливается и файла параметров не создаёт - для рабочего сервера это ожидаемое поведение, а не сбой. Среди предлагаемых сценарием способов исправления есть «дать docker доступ в сеть»; в этом случае дело не в сети, а в правах на обращение к Docker, и достаточно повторить команду с `sudo`. Если сценарий выполнялся через `sudo`, не забудьте вернуть файл владельцу каталога - команда приведена ниже.

Права на файл и вывод сценария. Файл параметров создаётся с правами 600: читает его только владелец. Права выставляет сам сценарий, отдельной команды для этого не требуется. Выставляет он их до записи содержимого, поэтому файл не бывает общедоступным даже на те доли секунды, пока его заполняют.

Листинг 5.18 — Проверка прав на файл параметров

```
stat -c %a .env
```

Ожидаемый результат: `600`. Другое значение означает, что файл создавался не сценарием или права меняли после него.

Кому принадлежит файл. Сценарий выполняется от имени пользователя, которому принадлежит каталог системы, и файл параметров остаётся в его собственности. Через `sudo` его запускают в одном случае: когда на сервере нет средства `age` и пару ключей шифрования файлового архива приходится создавать одноразовым контейнером, для чего нужен

доступ к Docker. Тогда файл достаётся суперпользователю, и его возвращают владельцу каталога - иначе дальнейшие действия с файлом, запись ключа резервного копирования в подразделе 11.4 и проверка действующих значений в подразделе 13.6, потребуют прав, которых у оператора нет.

Листинг 5.19 — Возврат файла параметров владельцу каталога

```
sudo chown buro:buro /opt/systemburo/.env
```

Значения секретов на экран не выводятся намеренно. Сценарий запускается в том числе по сети из конвейера выпуска, и всё, что он печатает, оседает в журнале запуска конвейера и в истории терминала - там, где права на файл уже ничего не значат. Поэтому вывод называет только то, что создано: `DB_PASSWORD`, `JWT_SECRET`, `JWT_REFRESH_SECRET`, `DATA_ENCRYPTION_KEY`, а для рабочего сервера ещё и пару ключей файлового архива.

ВАЖНО. Перенесите секреты в хранилище секретов организации, взяв их из самого файла `.env`. Из этого следует обратное требование: файл `.env` содержит пароли и ключи открытым текстом, его нельзя копировать в общедоступные каталоги, прикладывать к обращениям и включать в резервные копии общего доступа.

Чем отличается набор параметров рабочего сервера. Одна и та же команда для рабочего сервера и для стенда заполняет файл по-разному. Различия не косметические: часть из них меняет поведение системы при запуске.

Таблица 5.1 — Отличия набора параметров рабочего сервера от стенда

Параметр	Рабочий сервер	Стенд
`REQUIRE_ENCRYPTION`	`true`: без ключей шифрования система не запускается	`true`: те же требования к ключам
`LOG_LEVEL`	`info`	`debug`
`ARCHIVE_HOST_PATH`	`/srv/systemburo/archive`, отдельный каталог вне установки	`./archive` в каталоге установки

Продолжение таблицы 5.1

Параметр	Рабочий сервер	Стенд
`ARCHIVE_AGE_RECIPIENT`, `ARCHIVE_AGE_IDENTITY`	Пара ключей создаётся сценарием	Пара генерируется сценарием; отсутствие ключей при REQUIRE_ENCRYPTION =true препятствует штатному запуску
`PGADMIN_EMAIL`, `PGADMIN_PASSWORD`	Не записываются	Записываются
`BASIC_AUTH_USER`, `BASIC_AUTH_PASS`	Не записываются	Записываются

Панель управления базой данных и запрос имени и пароля на входе описаны только в наложении стенда, поэтому на рабочем сервере соответствующие строки были бы лишними паролями в файле: заданные, но ничего не открывающие. Их там и нет.

Ключи шифрования файлового архива. Для рабочего сервера сценарий создаёт пару ключей средством `age` - тем же, которым шифруются резервные копии. Устанавливать его в систему не требуется: если средства нет, пара создаётся одноразовым контейнером. Открытая часть записывается в `ARCHIVE_AGE_RECIPIENT`, закрытая в `ARCHIVE_AGE_IDENTITY`.

Пока ключ принимающей стороны не известен, в `ARCHIVE_AGE_RECIPIENT` стоит открытая часть ключа самой системы: бланки лежат зашифрованными, и система их читает. Когда ключ принимающей стороны появится, строку заменяют, а записанные ранее файлы остаются читаемыми. Подробности - в подразделе 9.6.

Если ключ принимающей стороны известен заранее, его подставляют при запуске сценария.

Листинг 5.20 — Создание файла параметров с готовым ключом принимающей стороны

```
ARCHIVE_AGE_RECIPIENT=age1... ./scripts/init-env.sh production <адрес системы>
```

Свою пару сценарий в этом случае всё равно создаёт: закрытая часть нужна системе, чтобы читать собственный архив, - но открытая берётся из переданного значения, а не из созданной пары. Если переданы обе части, сценарий не создаёт ничего и записывает их как есть.

ВАЖНО. Если создать пару ключей не удалось — в системе нет `age` и недоступен реестр образов, — для рабочего сервера сценарий останавливается и файла параметров не создаёт. Он предлагает установить `age`, открыть доступ к реестру образов либо передать готовые ключи. Для стенда сценарий продолжает подготовку файла с предупреждением прежнего формата о незашифрованных бланках, но сохраняет `REQUIRE_ENCRYPTION=true`: запуск приложения без необходимых ключей всё равно будет отклонён. До запуска восстановите или задайте корректную пару ключей; предупреждение не разрешает хранить бланки открытым текстом.

Каталог файлового архива. По окончании работы сценарий печатает команды, которыми создаётся каталог архива. Выполнить их нужно до первого запуска: прикладной сервер работает под непривилегированным пользователем и чужой каталог себе не присвоит. Порядок и последствия описаны в подразделе 9.6.

Особое внимание к параметру `DATA_ENCRYPTION_KEY`. Им зашифрованы сведения документов, удостоверяющих личность.

ОПАСНО. При утрате ключа шифрования расшифровать персональные данные невозможно. Резервная копия базы не поможет: она содержит те же зашифрованные значения. Ключ должен храниться отдельно от сервера и отдельно от резервных копий; порядок хранения и смена ключа описаны в подразделе 9.12.

Сценарий намеренно отказывается перезаписывать существующий файл параметров.

ОПАСНО. Не запускайте создание параметров повторно на работающей системе. Будет сгенерирован новый ключ шифрования, а прежний потерян. Прикладной сервер такую подмену обнаружит при запуске и откажется подниматься (подраздел 9.12), то есть данные останутся целы, но система не заработает, пока прежний ключ не вернут из хранилища.

Правило хранения секретов. Оно одинаково для всех тайных значений системы: ключа шифрования персональных данных, паролей базы, ключей подписи маркеров доступа и закрытого ключа резервных копий.

Таблица 5.2 — Как хранить секреты системы

Требование	Причина
Не менее двух копий в разных местах	Единственная копия - такая же точка отказа, как единственный сервер: отказ носителя или потеря доступа к хранилищу означают безвозвратную утрату
Хранить отдельно от того, что они защищают	Ключ рядом с резервной копией или с базой обесценивает шифрование: получивший доступ получает и данные, и ключ
Обновлять копию при каждом изменении	Копия устаревшего файла параметров не откроет данные, зашифрованные новым ключом
Передавать защищённым каналом	Пересылка секрета обычной почтой или в переписке равносильна его публикации
Ограничить круг лиц	Секрет знают те, кому он нужен для работы, и этот список пересматривается при смене сотрудников

ОПАСНО. Утрата ключа шифрования персональных данных необратима. База восстановится, но сведения документов, удостоверяющих личность, останутся нечитаемыми. Восстановить их невозможно ни средствами системы, ни силами разработчика.

Что делать, если секрет утрачен. Восстановить его нельзя, но систему в большинстве случаев можно вернуть в рабочее состояние. Порядок зависит от того, что именно потеряно.

Таблица 5.3 — Действия при утрате секрета

Что утрачено	Что потеряно безвозвратно	Порядок действий
Закрытый ключ резервных копий, система работает	Все ранее снятые зашифрованные копии	Создать новую пару ключей по подразделу 11.4, заменить `BACKUP_AGE_RECIPIENT`, немедленно снять свежую копию и проверить её восстановимость, старые копии удалить: они больше не расшифровываются
Закрытый ключ копий и сама система одновременно	Данные и копии	Разворачивать систему заново по разделу 5 и вводить данные повторно
Файл параметров на сервере, копия в хранилище есть	Ничего	Вернуть файл из хранилища в каталог системы и перезапустить: `sudo make deploy-up`

Продолжение таблицы 5.3

Что утрачено	Что потеряно безвозвратно	Порядок действий
Файл параметров и его копия, база цела	Зашифрованные сведения документов недоступны без прежнего ключа	Данные базы сохраняются, но проверка ключа блокирует запуск приложения. Найти прежний `DATA_ENCRYPTION_KEY` в хранилище секретов. Генерация нового ключа не восстанавливает старые данные; если ключ окончательно утрачен, дальнейшее восстановление согласуется с владельцем данных
Пароль базы данных	Ничего	Задать новый в файле параметров и в самой базе, перезапустить систему

ВАЖНО. Обнаружив утрату, сначала снимите свежую копию и убедитесь, что она разворачивается. Работа с системой без проверенной копии - это работа без страховки, и любая следующая ошибка станет необратимой.

5.5. Выбор имени базы данных

Имя базы задаётся параметром `DB_NAME` в файле параметров. Если требуется имя, отличное от заданного по умолчанию, измените его **до первого запуска**: тогда база сразу создастся с нужным именем.

Листинг 5.21 — Пример изменения имени базы до первого запуска

```
DB_NAME=buropropuskov
DATABASE_URL=postgres://postgres:<пароль>@db/buropropuskov
```

Переименование уже работающей базы описано в подразделе 9.8.

5.6. Подготовка сертификата

Сертификат готовится до первого запуска: обратный прокси не стартует, если файла сертификата нет. Способы выпуска и выбор между ними описаны в разделе 7. Для установки внутри сети организации достаточно выполнить:

Листинг 5.22 — Выпуск сертификата для внутренней сети

```
./scripts/init-tls.sh self <адрес системы> <IP-адрес сервера>
```

Ожидаемый результат: на экран выведены сведения о выпущенном сертификате, включая срок действия, и указан путь к корневому сертификату для установки на рабочие места.

5.7. Первый запуск

Сборка образов выполняется на сервере и занимает от нескольких минут до получаса в зависимости от производительности.

Листинг 5.23 — Сборка и запуск

```
sudo make deploy-build
sudo make deploy-up
```

Первый запуск дольше последующих: создаётся схема базы данных, устанавливаются расширения, строятся индексы.

Листинг 5.24 — Наблюдение за запуском

```
sudo make deploy-logs
```

Ожидаемый результат: в журнале нет сообщений уровня `error`, последние строки сообщают о запуске сервера.

5.8. Создание учётной записи администратора

Листинг 5.25 — Создание администратора

```
sudo make deploy-seed PASS=<пароль администратора>
```

Команда создаёт учётную запись `buropropuskov` с правами супер-администратора и заводит для неё организацию и компанию «Бюро пропусков».

Обязательные справочники - типы работников и разделы встроенного руководства - заполняются раньше, при первом запуске прикладного сервера. Поэтому команда выполняется только после успешного запуска: без справочника типов она завершится ошибкой.

ВАЖНО. Аргумент `PASS` обязателен. Без него будет установлен пароль по умолчанию, известный из состава поставки.

Пароль хранится в виде необратимой свёртки и в открытом виде нигде не сохраняется.

Команда спрашивает подтверждение и ждёт слова `ПЕРЕЗАПИСАТЬ`: при первой установке вопрос лишний, зато он останавливает случайный

повторный запуск на работающей системе, где пароль администратора давно сменили. Порядок подтверждения описан в подразделе 9.1.

Демонстрационное наполнение (`make deploy-seed-demo`) на рабочем сервере запускать не следует: оно создаёт вымышленные заявки, организации и транспорт, которые придётся удалять вручную. Оно тоже спрашивает подтверждение, словом `НАЛИТЬ`.

5.9. Проверка после установки

Выполните проверки последовательно. Каждая проверяет свой уровень, и успех предыдущей не гарантирует успех следующей.

Таблица 5.4 — Проверки после установки

Проверка	Команда или действие	Ожидаемый результат
Части системы запущены	`sudo docker compose -f docker-compose.base.yml -f docker-compose.prod.yml ps`	Четыре службы в состоянии `Up`, база в состоянии `healthy`
Сервер отвечает	`curl -s http://localhost/health`	`{"status":"ok"}`
Защищённое соединение работает	`curl -sI https://<адрес>/`	Код 200, присутствует заголовок `Strict-Transport-Security`
Незащищённое перенаправляется	`curl -sI http://<адрес>/`	Код 301, заголовок `Location` начинается с `https://`
Вход выполняется	Открыть систему в браузере, войти под администратором	Открывается экран новостей и обзора, в меню есть группа «Администрирование»
Права применяются	Раскрыть в меню группу «Администрирование» и открыть раздел «Пользователи»	Открывается список учётных записей, в нём есть заведённый администратор

ПРИМЕЧАНИЕ. Проверка `/health` подтверждает только то, что процесс жив. К базе данных она не обращается, поэтому успешный ответ при недоступной базе возможен. Полноценной проверкой является вход в систему.

ПРИМЕЧАНИЕ. «Администрирование» в меню - это группа разделов, а не страница. Отдельного адреса `/admin` в системе нет, и обращение к нему отвечает сообщением «Страница не найдена»: это не признак неисправности. Разделы управления открываются из раскрытой группы, каждый по своему адресу, например `/admin/users`.

Две проверки защищённого соединения выполняются с рабочего места, по тому адресу, по которому системой будут пользоваться, а не с самого сервера. Если сервер стоит за прокси поставщика услуг, обе они ведут себя иначе, и порядок настройки описан в подразделе 7.6.

5.10. Ограничение доступа к серверу

Предыдущие подразделы описывали установку самой системы. Последний шаг относится к серверу, на котором она работает, и выполняется до того, как в системе появятся настоящие данные.

Зачем это нужно. Сервер с внешним адресом перебирают постоянно: программы обходят адреса подряд и пробуют пары имени и пароля из готовых списков. Это фоновый шум, а не целевая атака, и сам по себе он безобиден. Опасен он в сочетании с разрешённым входом по паролю: одной угаданной пары достаточно, чтобы получить полный доступ к машине, а вместе с ним прямой доступ к базе со всеми персональными данными и к файлу параметров, где лежит ключ их шифрования. Никакая защита внутри системы в этом случае уже не работает.

Настройка делается в три приёма: закрыть лишние порты, замедлить перебор и убрать сам его предмет, отключив вход по паролю. Запрос имени и пароля на входе в проверочный стенд, описанный в подразделе 8.5, к этому отношения не имеет: он закрывает веб-интерфейс стенда, а здесь речь о доступе к самой машине.

> **ПРИМЕЧАНИЕ.** Команды приведены для Ubuntu и Debian. В RHEL-совместимых и в РЕД ОС межсетевым экраном управляет `firewalld`, а служба удалённого доступа называется `sshd`. Смысл настройки тот же, точная форма команд берётся из документации дистрибутива.

Межсетевой экран. Снаружи оставляют три порта: порт SSH для управления сервером, 80 и 443 для работы с системой. Всё остальное закрывается политикой по умолчанию. Область доступности каждого порта приведена в приложении А.

ВАЖНО. Правило для порта SSH добавляется **до** включения экрана. Экран, включённый первым, обрывает текущее подключение и закрывает вход: вернуть доступ получится только через консоль управления у поставщика услуг или физически у машины.

Листинг 5.26 — Настройка межсетевого экрана

```
sudo ufw allow 22/tcp
sudo ufw allow 80/tcp
sudo ufw allow 443/tcp
sudo ufw --force enable
```

Ключ `--force` снимает вопрос о подтверждении: без него команда спрашивает согласия и предупреждает, что включение может разорвать соединение.

Листинг 5.27 — Проверка правил экрана

```
sudo ufw status verbose
```

Ожидаемый результат: первая строка `Status: active`, строка политик по умолчанию содержит `deny (incoming)` и `allow (outgoing)`, а в перечне ровно три разрешённых порта: 22, 80 и 443. Каждый из них показан двумя строками, отдельно для IPv4 и IPv6: это одно правило, а не дубль.

Если сервером управляют с известных адресов, правило для SSH сужают до них, и перебор перестаёт доходить до службы вовсе.

Листинг 5.28 — Разрешение SSH только из заданной сети

```
sudo ufw allow from <сеть подразделения ИТ> to any port 22 proto tcp
sudo ufw delete allow 22/tcp
```

Ожидаемый результат: в выводе `ufw status verbose` строка для порта 22 указывает источник вместо `Anywhere`. Вторая команда удаляет прежнее общее разрешение и выполняется только после того, как проверено подключение из разрешённой сети.

Сужение применимо там, где у обслуживающего персонала постоянные адреса. Если администраторы подключаются с меняющихся адресов, порт управления остаётся открытым отовсюду, и защита держится

на двух следующих частях настройки - входе по ключу и средстве блокировки подбора.

Особенность: порты, опубликованные Docker. Правила ufw не закрывают порты, которые публикуют контейнеры. Docker правит правила фильтрации напрямую, и его цепочки просматриваются раньше правил экрана, поэтому порт, указанный в разделе `ports` описания контейнера, остаётся доступен снаружи, даже если экран его запрещает. Рабочему серверу это не мешает: наружу публикует порты только обратный прокси, 80 и 443, а база данных, прикладной сервер и веб-приложение не публикуют ничего и живут во внутренней сети контейнеров.

> **ВАЖНО.** Из этой особенности следует, что закрыть порт базы данных правилом `ufw deny` невозможно. Если публикация порта появится - на сервере запустят описание для разработки, где порт 5432 публикуется намеренно, либо публикацию добавят вручную для разбора неисправности, - база окажется доступна из сети, а `ufw status` будет по-прежнему показывать порт запрещённым. Закрывается он только удалением публикации из описания контейнера.

Защита от подбора пароля. Средство `fail2ban` читает журнал входов и временно закрывает доступ адресам, с которых идут неудачные попытки.

Листинг 5.29 — Установка защиты от подбора

```
sudo apt install -y fail2ban
```

Настройки записываются в отдельный файл `/etc/fail2ban/jail.local`. Штатный файл настроек пакета при обновлении перезаписывается, а этот остаётся нетронутым.

Листинг 5.30 — Файл /etc/fail2ban/jail.local

```
[sshd]
enabled = true
maxretry = 5
findtime = 10m
bantime = 1h
```

Значения читаются так: пять неудачных попыток с одного адреса за десять минут закрывают этому адресу доступ к серверу на час.

Администратору, ошибшемуся паролем, такого запаса хватает, а перебору по списку нет: вместо тысяч паролей в час он успевает попробовать пять. Блокировку входа в саму систему, описанную в техническом описании, эта настройка не заменяет и не отменяет: та закрывает учётную запись работника, эта - доступ к машине с конкретного адреса.

Листинг 5.31 — Применение настроек

```
sudo systemctl enable fail2ban
sudo systemctl restart fail2ban
```

Листинг 5.32 — Проверка состояния защиты

```
systemctl is-active fail2ban
sudo fail2ban-client status sshd
```

Ожидаемый результат: первая команда отвечает одним словом `active`. Вторая выводит состояние правила `sshd` двумя частями: в разделе `Filter` - сколько неудачных попыток замечено (`Currently failed`, `Total failed`), в разделе `Actions` - сколько адресов закрыто (`Currently banned`, `Total banned`) и перечень этих адресов строкой `Banned IP list`. Ненулевые значения означают, что средство работает и уже отсекает перебор.

Ответ `inactive` у первой команды либо сообщение о неизвестном правиле у второй означают, что служба не запущена или файл настроек содержит ошибку. Причина видна в `journalctl -u fail2ban`.

Вход по ключу вместо пароля. Пока вход по паролю разрешён, перебор имеет смысл, и средство защиты лишь замедляет его. Ключ убирает саму возможность: подобрать его нельзя. Порядок из четырёх шагов, и он выполняется именно в этой последовательности.

ОПАСНО. Отключение парольного входа до того, как проверен вход по ключу, оставит сервер недоступным. Восстановить доступ можно будет только через консоль управления у поставщика услуг или физически у машины.

Шаг 1. На рабочем месте администратора создаётся пара ключей, если её ещё нет.

Листинг 5.33 — Создание ключа для входа на сервер

```
ssh-keygen -t ed25519 -C "admin@buro"
```

Закрытая часть остаётся на рабочем месте и никуда не пересылается: на неё распространяется правило хранения секретов из подраздела 5.4. Наружу передаётся только открытая часть.

Шаг 2. Открытая часть кладётся на сервер.

Листинг 5.34 — Перенос открытой части ключа на сервер

```
ssh-copy-id buro@<адрес сервера>
```

Команда запрашивает пароль - в последний раз - и дописывает ключ в файл `~/.ssh/authorized_keys` на сервере.

Шаг 3. Вход по ключу проверяется **отдельным** подключением, не закрывая текущее. Для этого открывается второе окно терминала.

Листинг 5.35 — Проверка входа по ключу

```
ssh buro@<адрес сервера>
```

Ожидаемый результат: подключение открывается без запроса пароля. Если пароль всё же запрашивается, ключ не принят, и дальше идти нельзя: сначала проверяются права на сервере - каталог `~/.ssh` должен быть с правами 700, файл `authorized_keys` с правами 600.

Шаг 4. Только после успешной проверки отключается вход по паролю. В файле `/etc/ssh/sshd_config` строка `PasswordAuthentication` приводится к виду:

Листинг 5.36 — Строка файла /etc/ssh/sshd_config

```
PasswordAuthentication no
```

Листинг 5.37 — Проверка настроек и перезапуск службы

```
sudo sshd -t
sudo systemctl restart ssh
```

Ожидаемый результат: первая команда не печатает ничего, это значит, что файл настроек читается без ошибок; вторая завершается молча. Уже

открытые подключения перезапуск не разрывает, поэтому текущее окно терминала остаётся рабочим.

Листинг 5.38 — Проверка действующего значения

```
sudo sshd -T | grep passwordauthentication
```

Ожидаемый результат: `passwordauthentication no`. Команда показывает значение, с которым служба работает на самом деле, а не то, что записано в основном файле.

ПРИМЕЧАНИЕ. Если проверка показывает `yes`, значение переопределено файлом из каталога `/etc/ssh/sshd_config.d/`. В образах, подготовленных поставщиками услуг, там нередко лежит файл, разрешающий вход по паролю. Действует первое встреченное значение, а этот каталог подключается в начале основного файла, поэтому правку вносят именно туда.

Как убедиться, что подбор идёт. Число неудачных попыток входа считается по журналу.

Листинг 5.39 — Подсчёт неудачных попыток входа

```
sudo journalctl -u ssh --since "30 days ago" | grep -c "Failed password"
```

Ожидаемый результат: одно число. Ненулевое, вплоть до нескольких тысяч, - это норма для сервера, доступного из сети общего пользования, а не признак того, что сервер взломан. Оно показывает ровно то, ради чего выполнялась настройка: попытки идут постоянно, и защита от подбора вместе с отключённым парольным входом оставляет их безрезультатными.

ПРИМЕЧАНИЕ. Глубина журнала ограничена его настройками хранения, поэтому за более давний срок число окажется меньше фактического. Средство защиты от подбора считает свои попытки с момента последнего запуска службы, и его счётчик с этим числом не совпадает.

Соответствующие пункты вынесены в проверочный лист ввода в эксплуатацию, приложение Г.

6. Параметры конфигурации

6.1. Как задаются параметры

Параметры передаются прикладному серверу переменными окружения. Файл `.env` в каталоге системы подставляет значения в описание контейнеров.

Описание рабочего сервера передаёт контейнеру не весь файл целиком, а перечисленный набор переменных, но в этот набор входят все параметры приложения Б, кроме нескольких служебных, названных ниже. То есть строка, вписанная в `.env`, действует после перезапуска системы.

Состав набора сверяется с кодом автоматически при каждой проверке исходного текста: параметр, который система читает, но который не передан контейнеру, останавливает проверку. Это гарантирует, что перечень приложения Б не разойдётся с тем, что доходит до сервера, и вписанная строка не окажется бездействующей молча.

Таблица 6.1 — Параметры, действующие через файл параметров

Назначение	Параметры
Подключение к базе	`DB_USER`, `DB_PASSWORD`, `DB_NAME`
Маркеры доступа	`JWT_SECRET`, `JWT_REFRESH_SECRET`, `JWT_ACCESS_TTL`, `JWT_REFRESH_TTL`
Журнал и программный интерфейс	`LOG_LEVEL`, `SWAGGER_ENABLED`, `CORS_ALLOWED_ORIGINS`
Шифрование	`DATA_ENCRYPTION_KEY`, `REQUIRE_ENCRYPTION`, `ARCHIVE_AGE_RECIPIENT`, `ARCHIVE_AGE_IDENTITY`
Загрузка файлов	`UPLOAD_MAX_FILE_SIZE`, `UPLOAD_ALLOWED_IMAGE_TYPES`, `UPLOAD_ALLOWED_DOC_TYPES`
Ограничения обращений	`RATE_LIMIT_PER_MINUTE`, `RATE_LIMIT_WINDOW_SEC`, `RATE_LIMIT_IP_PER_MINUTE`, `RATE_LIMIT_IP_WINDOW_SEC`, `PAGINATION_MAX_LIMIT`
Уведомления в браузер	`VAPID_PUBLIC_KEY`, `VAPID_PRIVATE_KEY`, `VAPID_SUBJECT`, `PUSH_SUBSCRIPTION_RETENTION_DAYS`
Адрес посетителя за прокси	`TRUSTED_PROXY_CIDRS`, `EDGE_TRUSTED_PROXY_CIDRS`, `PROXY_NETWORK_PREFIX`
Каталог файлового архива	`ARCHIVE_HOST_PATH`

Продолжение таблицы 6.1

Назначение	Параметры
Пул соединений с базой	DB_MAX_OPEN_CONNS, DB_MAX_IDLE_CONNS, DB_CONN_MAX_LIFETIME, DB_CONN_MAX_IDLE_TIME
Сроки ожидания прикладного сервера	HTTP_READ_HEADER_TIMEOUT, HTTP_READ_TIMEOUT, HTTP_WRITE_TIMEOUT, HTTP_IDLE_TIMEOUT
Проверка пароля и вход	ARGON2_HASH_CONCURRENCY, LOGIN_RATE_LIMIT_MAX, LOGIN_RATE_LIMIT_WINDOW_SEC, COOKIE_SECURE
Почтовая рассылка	SMTP_HOST, SMTP_PORT, SMTP_USERNAME, SMTP_PASSWORD, SMTP_FROM, SMTP_FROM_NAME, SMTP_TLS_MODE, SMTP_TIMEOUT_SEC, SMTP_RATE_PER_HOUR, MAIL_RETRY_ATTEMPTS, MAIL_WORKER_TICK
Файлы, приложенные к заявке	APPLICATION_FILE_MAX_COUNT, APPLICATION_FILE_MAX_TOTAL_SIZE, APPLICATION_FILE_DRAFT_MAX_COUNT, APPLICATION_FILE_DRAFT_MAX_TOTAL_SIZE, APPLICATION_FILE_DRAFT_TTL, APPLICATION_FILE_IMAGE_MAX_SIDE, APPLICATION_FILE_JPEG_QUALITY
Журналы приложения	LOG_MAX_SIZE_MB, LOG_MAX_AGE_DAYS, LOG_MAX_BACKUPS, LOG_COMPRESS
Сроки хранения	REQUEST_LOG_DETAIL_DAYS, REQUEST_LOG_PARTITION_PRECREATE_DAYS, PD_AUDIT_RETENTION_MONTHS, REFRESH_TOKEN_RETENTION_DAYS, READ_NOTIFICATION_RETENTION_DAYS, NOTIFICATION_RETENTION_DAYS
Обслуживание файлового архива	ARCHIVE_WORKER_TICK, ARCHIVE_SWEEP_INTERVAL
Прочее	RESET_TIMEZONE, ANALYTICS_CACHE_REFRESH_SEC

Пару ключей для уведомлений в браузер не придумывают руками - её выдаёт система:

Листинг 6.1 — Создание ключей уведомлений в браузер

```
sudo make deploy-vapid
```

Ожидаемый результат: две готовые строки, `VAPID_PUBLIC_KEY` и `VAPID_PRIVATE_KEY`, которые вставляются в файл параметров как есть. Открытый ключ уходит в браузер работника, закрытый остаётся на сервере: им подписываются уведомления от имени системы, и он хранится как секрет, наравне с ключами из подраздела 5.4. Контакт бюро `VAPID_SUBJECT` команда не выдаёт, его задают вручную - адрес вида `mailto:` либо адрес сайта.

Два параметра из этой таблицы ведут себя не так, как остальные, и это важно при правке файла руками.

Строка подключения `DATABASE_URL` в файле параметров есть, но в контейнер попадает не она: описание собирает строку заново из `DB_USER`, `DB_PASSWORD` и `DB_NAME`. Правка одной только `DATABASE_URL` ни на что не влияет, менять нужно эти три значения. Каталог `ARCHIVE_HOST_PATH` тоже не переменная окружения прикладного сервера: он задаёт, какой каталог хоста подключается к контейнеру, а внутри контейнера архив всегда виден по пути `/app/archive` (`ARCHIVE_PATH`).

Отдельно задаются служебные значения, которые определяют устройство самого контейнера, а не поведение системы: адрес и порт приёма соединений (`BIND_HOST`, `BIND_PORT`) закреплены внутри контейнера, потому что по ним к нему обращается обратный прокси, а пути `UPLOAD_PATH`, `ARCHIVE_PATH` и `ENTITY_EXPORT_PATH` указывают на каталоги внутри контейнера, и менять их через файл параметров незачем: снаружи каталог задаётся отдельной строкой вроде `ARCHIVE_HOST_PATH`.

Остальные параметры сценарий записывает в файл готовыми строками со значениями, совпадающими со встроенными. Правка такой строки применяется после перезапуска системы.

Листинг 6.2 — Изменение параметра

```
RESET_TIMEZONE=Asia/Yekaterinburg
REQUEST_LOG_DETAIL_DAYS=14
```

Листинг 6.3 — Применение изменений

```
sudo make deploy-up
```

Ожидаемый результат: контейнер прикладного сервера пересоздан, новые значения действуют. Проверить можно командой из подраздела 13.6.

6.2. Обязательные параметры

Три параметра обязательны. При их отсутствии или недопустимом значении прикладной сервер завершает работу с ошибкой, а не запускается в ослабленном режиме.

Таблица 6.2 — Обязательные параметры

Параметр	Проверка при запуске
DATABASE_URL	Должен начинаться с postgres:// или postgresql://
JWT_SECRET	Не короче 32 символов
JWT_REFRESH_SECRET	Не короче 32 символов

Для маркера доступа и маркера продления используйте разные секреты: это разделяет ключи двух операций. Сценарий подготовки параметров генерирует их независимо, поэтому при штатной установке они заведомо различны. Проверка действующей сессии выполняется отдельно; наличие подписанного маркера само по себе не подтверждает доступ.

ВАЖНО. Совпадение этих двух ключей система при запуске не проверяет. Задавая их вручную, убедитесь, что они разные.

6.3. Основные параметры

Полный перечень приведён в приложении Б. Здесь перечислены параметры, которые чаще всего требуется изменять.

Таблица 6.3 — Часто изменяемые параметры

Параметр	По умолчанию	Назначение
LOG_LEVEL	info	Подробность журнала: debug, info, warn, error
RESET_TIMEZONE	Europe/Moscow	Часовой пояс суточного сброса территориальных статусов
REQUEST_LOG_DETAIL_DAYS	30	Глубина хранения подробных записей журнала запросов
RATE_LIMIT_PER_MINUTE	200	Допустимое число запросов в минуту от одного работника
UPLOAD_MAX_FILE_SIZE	10485760	Наибольший размер загружаемого файла, 10 МБ
SWAGGER_ENABLED	false	Публикация описания программного интерфейса
REQUIRE_ENCRYPTION	true	Запрет запуска без ключей шифрования: персональных данных и файлового архива

Встроенное умолчание REQUIRE_ENCRYPTION=true совпадает с параметрами, которые сценарий установки создаёт и для рабочего сервера, и для стенда. Ручное отключение шифрования не является штатным способом

исправления запуска: сначала восстановите корректные ключи и проверьте их сохранность.

ВАЖНО. На рабочем сервере `REQUIRE_ENCRYPTION` должен оставаться в значении `true`. Это исключает ситуацию, когда система запустилась без ключа шифрования и записала персональные данные в открытом виде. Проверка охватывает и ключи файлового архива: незаполненные `ARCHIVE_AGE_RECIPIENT` и `ARCHIVE_AGE_IDENTITY` означают, что бланки и приложенные к заявкам файлы лягут на диск открытыми, а заметить это можно только по именам файлов в каталоге. Снятие требования, чтобы «система хотя бы запустилась», лечит симптом и оставляет данные незащищёнными: разбираться нужно с ключами.

6.4. Параметры базы данных

Заданы в файле описания рабочего сервера и подобраны под конфигурацию из подраздела 3.1.

Таблица 6.4 — Параметры базы данных и среды выполнения

Параметр	Значение	Смысл
`shared_buffers`	`8GB`	Память под собственный кэш страниц базы
`effective_cache_size`	`20GB`	Оценка доступной памяти под кэш, влияет на выбор плана запроса
`work_mem`	`32MB`	Память на одну операцию сортировки или соединения
`maintenance_work_mem`	`1GB`	Память под построение индексов и обслуживание
`max_connections`	`150`	Наибольшее число одновременных подключений
`max_wal_size`	`4GB`	Объём журнала предзаписи между контрольными точками
`random_page_cost`	`1.1`	Стоимость произвольного чтения, значение для твердотельных накопителей
`effective_io_concurrency`	`200`	Число параллельных операций ввода-вывода
`GOMAXPROCS`	`6`	Число ядер, используемых прикладным сервером
`GOMEMLIMIT`	`6GiB`	Порог, при котором усиливается освобождение памяти

6.5. Пул соединений, таймауты и проверка пароля

Парольные операции допускаются до открытия транзакции. `ARGON2_HASH_CONCURRENCY` ограничивает одновременно выполняемые вычисления; мест ожидания вдвое больше, ожидание не превышает двух секунд. Переполнение возвращает HTTP 503 с Retry-After: 2 и не является неверным паролем. Завершение HTTP-запроса не освобождает слот уже начатого вычисления до его фактического окончания. Увеличивать предел следует по измерениям процессора и памяти, а не ради удаления сообщения о занятости.

Для тяжёлых прикладных команд, импорта, загрузок и подготовки выгрузок есть общий бюджет процесса: min(2, max(1, GOMAXPROCS)) выполняемых операций, вдвое больше мест ожидания и не более двух секунд ожидания. Это константный профиль кода, отдельного параметра `.env` для него нет. Команды и импорт ограничены контекстом до 120 секунд; работа БД и подготовка выгрузки ограничиваются отдельно от передачи файла. Проверять нужно обе причины отказа: занятость возвращает 503 и Retry-After: 2, прерванная операция — отдельный ответ. Предел тела малых запросов входа, продления и выхода — 64 КиБ; он не меняет допустимый размер обычной загрузки файлов.

Три группы параметров задают поведение системы под нагрузкой. До их появления все три работали на встроенных значениях библиотек, и это давало отказы, которые снаружи выглядят как «система тормозит».

Пул соединений с базой. Прикладной сервер не открывает соединение на каждый запрос, а держит их наготове. Пока пул не был настроен, действовали значения по умолчанию: число открытых соединений не ограничено, простаивающих держится два. Первое означает, что всплеск запросов упирается уже в предел самой базы и возвращается отказами; второе - что соединение открывается почти на каждый запрос, а база отводит под каждое отдельный процесс.

Таблица 6.5 — Пул соединений с базой

Параметр	По умолчанию	Назначение
`DB_MAX_OPEN_CONNS`	`50`	Наибольшее число одновременных соединений с базой
`DB_MAX_IDLE_CONNS`	`25`	Сколько соединений держится наготове между запросами
`DB_CONN_MAX_LIFETIME`	`1h`	Наибольший срок жизни соединения
`DB_CONN_MAX_IDLE_TIME`	`10m`	Через сколько закрывается простаивающее соединение

ВАЖНО. Предел пула считается заодно с пределом базы. `max_connections` базы равен 150 (подраздел 6.4), пул занимает из них 50, то есть треть. Остаток - не запас на всякий случай: мимо пула к базе ходят консольные команды (очистка, файловый архив, выгрузка сущности), панель управления базой на стенде, резервное копирование и служебный резерв самой базы. Поднимая `DB_MAX_OPEN_CONNS`, поднимайте и `max_connections`, иначе выигрыш обернётся отказами у того, кто в пул не входит.

Отрицательные значения четырёх параметров прекращают запуск. Если задан положительный предел открытых соединений, число простаивающих не может быть больше него. Ноль допустим, но в текущей реализации означает «не применять этот параметр к пулу»: сохраняется исходная настройка библиотеки. Это не универсальная команда снять ограничение; например, исходное число простаивающих соединений не становится нулевым.

Таймауты соединения. Не были заданы ни одного: зависшее соединение занимало ресурсы сервера до перезапуска, а медленная отправка заголовков стоила нападающему одного сокета.

Таблица 6.6 — Таймауты прикладного сервера

Параметр	По умолчанию	Назначение
`HTTP_READ_HEADER_TIMEOUT`	`10s`	На приём заголовков запроса; защита от соединений, отправляющих запрос по байту
`HTTP_READ_TIMEOUT`	`120s`	На приём запроса целиком, вместе с телом
`HTTP_WRITE_TIMEOUT`	`120s`	На работу обработчика и отправку ответа
`HTTP_IDLE_TIMEOUT`	`120s`	Сколько открытое соединение ждёт следующего запроса

Значения согласованы с обратным прокси, а не выбраны произвольно: обычные обращения прокси обрывает раньше, на 60 секундах, поэтому 120 секунд на стороне приложения физически не могут оборвать то, чего не оборвал бы прокси вдвое раньше. Три обращения живут дольше по замыслу - поток обновлений без перезагрузки страницы и обе выгрузки файлового архива, - и для них срок снимается в самом обработчике. Держать ради этих трёх общий таймаут в час значило бы не иметь таймаута вовсе. Ноль в любом из четырёх означает «без ограничения» и оставлен как отдушина: обработчик, который на данных конкретной организации законно идёт дольше, выключается параметром, а не правкой кода.

Проверка пароля. Пароли проверяются намеренно дорогим вычислением: каждая проверка занимает 19 МБ памяти и ядро процессора целиком. Это защита от подбора, но без предела на число одновременных проверок утренний вход смены умножает 19 МБ на число вошедших разом, и защита превращается в способ положить сервер силами собственных работников.

Таблица 6.7 — Ограничение одновременных проверок пароля

Параметр	По умолчанию	Назначение
`ARGON2_HASH_CONCURRENCY`	`0`	Сколько проверок пароля выполняется одновременно; `0` - по числу ядер процессора

Лишние входы ждут очереди, а не получают отказ: вход, отклонённый из-за нагрузки, человек повторит вручную, и очередь от этого станет длиннее. Задавать больше числа ядер бессмысленно - вычисление упирается в процессор, и одновременные проверки только делят между собой то же время, добавляя расход памяти.

Девять параметров этого подраздела задаются через файл параметров наравне с остальными: сценарий подготовки записывает их готовыми строками, и правка применяется после перезапуска системы.

6.6. Пересчёт параметров под другую конфигурацию

При установке на сервер с иными характеристиками параметры базы данных требуется пересчитать, иначе службе не хватит памяти либо она не использует доступную.

Таблица 6.8 — Правила пересчёта

Параметр	Правило
`shared_buffers`	Около четверти оперативной памяти
`effective_cache_size`	Около двух третей оперативной памяти
`maintenance_work_mem`	Около 1/32 оперативной памяти
`work_mem`	Столько, чтобы произведение на `max_connections` не превышало четверти памяти. На сервере с 8 ГБ и 50 соединениями это 8-16 МБ
`max_connections`	Сообразно памяти и числу ядер: на 8 ГБ - около 50, на 4 ГБ - около 30. Значение из поставки (150) рассчитано на 32 ГБ
`max_wal_size`	Не более десятой части свободного места на диске. При 100 ГБ это 4 ГБ из поставки, при 20 ГБ - не выше 1-2 ГБ
`GOMAXPROCS`	Равно числу ядер процессора
`GOMEMLIMIT`	Около одной пятой оперативной памяти

Пример для сервера с 8 ГБ памяти и 4 ядрами: `shared_buffers=2GB`, `effective_cache_size=5GB`, `maintenance_work_mem=256MB`, `work_mem=16MB`, `max_connections=50`, `GOMAXPROCS=4`, `GOMEMLIMIT=1600MiB`.

`shared_buffers` выделяет общий буфер базы. `effective_cache_size` — оценка доступного кэша для планировщика, сама память под неё не выделяется. `maintenance_work_mem` ограничивает отдельные операции обслуживания, а `work_mem` — отдельные операции сортировки и хеширования; в сложном запросе их может быть несколько. Поэтому размер памяти оценивают с учётом одновременно выполняемых запросов и обслуживания, а не только числа соединений. Приведённые значения — начальная оценка, которую проверяют измерениями на целевой нагрузке.

ВАЖНО. Вместе с `max_connections` уменьшайте предел пула соединений из подраздела 6.5. Правило «поднимая `DB_MAX_OPEN_CONNS`, поднимайте и `max_connections`» действует и в обратную сторону: если оставить пул в значении 50 при уменьшенном до 30 пределе базы, пул займёт его целиком, и всё, что ходит к базе мимо пула - консольные команды, резервное копирование, проверка восстановимости - начнёт получать отказы. Держите пул примерно в трети предела базы, как в поставке.

Первые пять параметров задаются в файле описания рабочего сервера `docker-compose.prod.yml`, в разделе `command` службы базы данных и в переменных окружения прикладного сервера. Пул соединений задаётся в файле параметров `.env`. После правки система перезапускается: `sudo make deploy-up`.

6.7. Почтовые уведомления

Как устроена отправка

Собственный почтовый сервер система не поднимает. Она подключается к чужому почтовому серверу как обычный почтовый клиент: под учётной записью ящика, по защищённому соединению, теми же параметрами, которые задают в любой почтовой программе. Подойдёт ящик у поставщика услуг связи, ящик в корпоративном сервисе или почтовый сервер самой организации.

Письмо отправляется не сразу в момент события. Оно записывается в очередь в базе данных, а фоновая задача раз в 15 секунд забирает накопившееся и передаёт почтовому серверу. Промежуточная очередь нужна, чтобы письмо не пропало: событие и задание на письмо сохраняются вместе, и сбой сети или недоступность почтового сервера откладывают отправку, а не отменяют её. Непринятое сервером письмо повторяется с нарастающей паузой: через минуту, пять минут, пятнадцать минут, дальше раз в час. Исчерпав установленное число попыток (`MAIL_RETRY_ATTEMPTS`, по умолчанию пять), система перестаёт пытаться, помечает письмо недоставленным и записывает причину отказа в журнал прикладного сервера.

Что будет, если не настраивать

Отправку включает единственный параметр: непустой `SMTP_HOST`. Пока он пуст, почта выключена целиком, и это допустимое состояние системы, а не ошибка. Фоновая задача разбора очереди при запуске не стартует и сообщает об этом в журнал, очередь не наполняется, а возможности, которым требуется письмо, отказываются выполняться с явным сообщением о ненастроенной почте. Видимости отправленных писем система при этом не создаёт.

> **ВАЖНО.** Почта выключается пустым адресом сервера, а не отдельным выключателем. Если в файле параметров `SMTP_HOST` заполнен, а остальные значения оставлены пустыми, прикладной сервер откажется запускаться: полуготовая настройка обнаружилась бы только на первом письме. Проверяются режим защиты соединения, номер порта, наличие и правильность адреса отправителя, парность логина и пароля и положительность числовых параметров.

Куда записываются параметры

Параметры почты записываются в файл параметров `.env` рядом с остальными, см. подраздел 5.4, и попадают в прикладной сервер обычным порядком. После правки файла систему перезапускают.

Листинг 6.4 — Применение изменений

```
make deploy-up
```

Ожидаемый результат: контейнер прикладного сервера пересоздан. Проверить, что значения дошли, можно командой из подраздела 13.6, подставив в неё `SMTP_HOST`.

Настройка почтового сервера

Значения выдаёт администратор почтовой службы. Не переносите настройки другого сервиса без проверки. Полный перечень параметров с умолчаниями приведён в приложении Б.

Таблица 6.9 — Параметры почтового подключения

Параметр	Что указать
`SMTP_HOST`	Адрес почтового сервера
`SMTP_PORT`	Порт, разрешённый почтовой службой
`SMTP_TLS_MODE`	`starttls` для перехода к шифрованию или `tls` для шифрования с начала соединения; режим должен соответствовать порту сервера
`SMTP_USERNAME`	Учётная запись ящика в формате почтовой службы
`SMTP_PASSWORD`	Пароль SMTP или отдельный пароль приложения, если его требует служба
`SMTP_FROM`	Адрес отправителя, разрешённый этой учётной записи

Порядок настройки:

1. Завести отдельный ящик для системы, а не использовать личный ящик работника. Ящик, привязанный к человеку, перестаёт работать при его увольнении вместе со сменой пароля.
2. Убедиться, что ящику разрешена отправка через SMTP. Это отдельное разрешение, и по умолчанию оно бывает выключено.
3. Записать параметры из таблицы в файл параметров и передать их прикладному серверу, как показано выше.
4. Перезапустить систему и убедиться, что она поднялась: неверное значение останавливает запуск с сообщением о том, какой параметр не принят.
5. Отправить проверочное письмо и получить его.

Разрешение на отправку у ящика. Второй шаг пропускают чаще всего, а выглядит его пропуск как неверный пароль. У поставщиков почтовых услуг отправка через почтовые программы нередко включается отдельной отметкой при создании ящика, а у ящика, созданного без неё, панель управления предлагает обратиться в поддержку вместо того, чтобы включить отправку по требованию. В пробном режиме услуги отправка через SMTP бывает выключена целиком, и тогда работает только веб-интерфейс поставщика.

Признак именно этой причины: соединение с почтовым сервером устанавливается, защищённое соединение поднимается, сервер предъявляет

способы проверки подлинности - и отклоняет верную пару логина и пароля отказом 535. Проверить пару в отрыве от системы можно с самого сервера.

Листинг 6.5 — Проверка учётных данных ящика с сервера системы

```
python3 - <<'PY'
import smtplib
s = smtplib.SMTP("<адрес почтового сервера>", 587, timeout=25)
s.ehlo(); s.starttls(); s.ehlo()
s.login("<полный адрес ящика>", "<пароль>")
print("вход принят, отправка ящику разрешена")
s.quit()
PY
```

Ожидаемый результат: строка о принятом входе. Отказ `SMTPAuthenticationError (535)` при заведомо верном пароле означает, что отправка этому ящику не разрешена, и вопрос решается у поставщика услуги, а не правкой параметров. Проверка занимает минуту и избавляет от поиска ошибки в настройках системы, где её нет.

Особенности отдельных случаев.

Внешняя почтовая служба. Уточните адрес, порт, режим шифрования и способ авторизации. Пароль для входа в веб-почту может отличаться от пароля SMTP: некоторые службы требуют отдельный пароль приложения.

Почтовый сервер организации. Единственный доступный вариант для контура без выхода в сеть общего пользования. Адрес, порт и режим защиты соединения берутся у администратора почтового сервера. Если сервер стоит во внутренней сети и не требует входа по учётной записи, `SMTP_USERNAME` и `SMTP_PASSWORD` оставляют пустыми обоими: заполненный логин без пароля система считает ошибкой. Режим `none` без шифрования допустим только внутри закрытого контура, где защиту канала обеспечивает сама сеть; для сервера в сети общего пользования он недопустим, поскольку логин, пароль и содержимое писем передавались бы открытым текстом.

ВАЖНО. Адрес в `SMTP_FROM` обязан совпадать с ящиком, под которым система входит на почтовый сервер. Отправку от чужого адреса почтовые серверы отвергают ошибкой 550, причём настройка выглядит рабочей ровно до первого письма: подключение и вход проходят успешно, а отказ приходит на этапе передачи письма.

Проверочное письмо

Проверочное письмо отправляется мимо очереди и отвечает сразу: система дожидается ответа почтового сервера и возвращает либо подтверждение отправки, либо разбор причины отказа. Это единственный способ убедиться в правильности настройки, не дожидаясь первого настоящего письма.

Отправку выполняет метод программного интерфейса `POST /api/settings/mail/test`. Обращение к нему выполняется с маркером доступа работника, у которого есть право на раздел настроек.

Листинг 6.6 — Получение маркера доступа

```
curl -s -X POST https://<адрес>/api/login \
  -H 'Content-Type: application/json' \
  -d '{"username":"<логин>","password":"<пароль>"}'
```

Ожидаемый результат: ответ содержит поле `token` - это и есть маркер доступа. Значение поля подставляется в следующую команду.

Листинг 6.7 — Отправка проверочного письма

```
curl -s -X POST https://<адрес>/api/settings/mail/test \
  -H 'Authorization: Bearer <маркер доступа>' \
  -H 'Content-Type: application/json' \
  -d '{"to":"<адрес получателя>"}'
```

Ожидаемый результат: ответ `{"success":true,"data":{"sent":true,"to":"<адрес получателя>"}}`, а на указанный адрес приходит письмо с темой «Проверка почтовой рассылки». При неудаче ответ содержит `"success":false` и объяснение причины: непринятый логин, несовпадающий отправитель, истёкшее ожидание или ошибка защищённого соединения. Разбор ответов приведён в подразделе 13.13.

Проверить, считает ли система почту настроенной, можно методом `GET /api/settings/mail/status`: он отвечает признаком `configured`, который равен `true` только при заданном адресе почтового сервера.

Записи SPF и DKIM в описании домена

Отправить письмо мало, его нужно ещё доставить. Почтовые службы получателей проверяют, разрешено ли этому серверу отправлять письма от имени домена организации, и письмо без такого подтверждения попадает в папку со спамом либо отклоняется молча.

Таблица 6.10 — Записи, требуемые в описании домена отправителя

Запись	Что подтверждает	Кто выдаёт значение
SPF	Список серверов, которым разрешено отправлять письма от имени домена	Поставщик почтовой услуги
DKIM	Подпись письма ключом домена, подтверждающая отправителя и неизменность письма	Поставщик почтовой услуги

Значения записей выдаёт поставщик почтовой услуги, а вносит их в описание домена тот, кто этим доменом управляет. Порядок создания записей зависит от почтовой службы и управления доменом.

ВАЖНО. Отсутствие записей SPF и DKIM не даёт никакой видимой ошибки. Проверочное письмо уйдёт успешно, система отчитается об отправке, а получатель писем не увидит: они лягут в папку со спамом или будут отброшены его почтовой службой. Именно поэтому проверку настройки завершают не ответом системы, а полученным письмом во входящих.

Исходящий порт и пределы отправки

На некоторых площадках исходящие SMTP-соединения ограничены. Уточните разрешённый порт у администратора почтовой службы и проверьте соединение с самого сервера системы, а не с рабочего места.

Листинг 6.8 — Проверка доступности почтового сервера

```
curl -v --connect-timeout 10 telnet://<почтовый сервер>:<порт>
```

Ожидаемый результат: соединение установлено. Для режима `tls` обычное текстовое приветствие до установки шифрования не ожидается; эта команда проверяет доступность порта, а не авторизацию и доставку. Окончательная проверка — получение проверочного письма.

У почтовой службы могут быть ограничения числа писем. Уточните действующие пределы для своей учётной записи; значения зависят от службы и тарифа.

Собственный предел системы задаётся параметром `SMTP_RATE_PER_HOUR` (по умолчанию 400 писем в час) и должен оставаться заведомо ниже предела поставщика. Достигнув его, система не теряет письма, а откладывает остаток очереди до следующего часа. Растянутая рассылка безобиднее упора в чужой предел: превысив его, ящик получает отказ на все письма подряд, а при повторных нарушениях поставщик услуг ограничивает отправку целиком.

7. Защищённое соединение и сертификат

7.1. Зачем это нужно

Система работает только по защищённому соединению. Причина не формальная: через неё передаются пароли и персональные данные, которые по незащищённому соединению видны любому, кто имеет доступ к сети между рабочим местом и сервером.

Кроме того, продление сеанса технически работает только по защищённому соединению: браузеру запрещено сохранять маркер продления, полученный по обычному протоколу. Поэтому без сертификата вход в систему не заработает вовсе.

> **ПРИМЕЧАНИЕ.** Сертификат нужен серверу, а не пользователям. Сотрудникам выдаются обычные имя и пароль. Никаких персональных сертификатов сотням работников выдавать не требуется.

7.2. Выбор способа получения сертификата

Сертификат должен быть подписан стороной, которой доверяет браузер. Отсюда три возможных способа, различающихся тем, кто подписывает.

Таблица 7.1 — Способы получения сертификата

Способ	Когда подходит	Что потребуется на рабочих местах
Собственный удостоверяющий центр	Система работает внутри сети, внешнего доменного имени нет	Однократная установка одного корневого сертификата, централизованно групповой политикой
Общедоступный удостоверяющий центр	У организации есть доменное имя, которым она владеет	Ничего, доверие настроено в браузерах изначально
Удостоверяющий центр организации	В организации развёрнута собственная служба сертификатов	Ничего, корневой сертификат уже установлен на машинах домена

ПРИМЕЧАНИЕ. Второй способ часто оказывается удобнее первого, и он применим, даже когда сервер стоит внутри сети и недоступен извне. Владение доменом подтверждается записью в системе доменных имён, а не доступом к серверу. Если у организации есть свой домен, этот путь избавляет от установки чего-либо на рабочие места.

7.3. Выпуск сертификата

Собственный удостоверяющий центр. Применяется при работе внутри сети организации.

Листинг 7.1 — Выпуск сертификата собственным центром

```
./scripts/init-tls.sh self <адрес системы> <IP-адрес сервера>
```

Сценарий создаёт корневой сертификат удостоверяющего центра и подписанный им сертификат сервера. Указание сетевого адреса вторым аргументом позволяет обращаться к системе и по имени, и по адресу.

Ожидаемый результат: выведены сведения о сертификате и путь к корневому сертификату `nginx/certs/ca/ca.crt`.

ВАЖНО. Обратный прокси сценарий перезагружает сам, но обращается для этого к Docker. Запущенный без `sudo`, он не отличает отказ в доступе от незапущенного контейнера и печатает «Контейнер nginx не запущен - перезагрузка не требуется», хотя тот работает. Сертификат при этом заменён, а прокси продолжает отдавать прежний, и предупреждение о недоверенном сертификате никуда не денется. Поэтому на работающей системе прокси перезагружают отдельной командой, как в подразделе 7.5.

Общедоступный удостоверяющий центр. Применяется при наличии доменного имени, доступного из сети общего пользования, и открытого порта 80.

Листинг 7.2 — Выпуск сертификата общедоступным центром

```
./scripts/init-tls.sh self <домен>
sudo make deploy-up
./scripts/init-tls.sh acme <домен> <адрес электронной почты>
```

Порядок двухшаговый: сначала создаётся временный сертификат, чтобы прокси смог запуститься, затем выпускается настоящий. Без первого шага прокси не стартует, а без работающего прокси невозможно подтвердить владение доменом.

Готовый сертификат. Применяется, если сертификат приобретён либо выпущен удостоверяющим центром организации.

Листинг 7.3 — Установка готового сертификата

```
./scripts/init-tls.sh import <файл цепочки> <файл ключа>
```

Сценарий сверяет соответствие ключа сертификату и отказывается устанавливать несовпадающую пару. Это защищает от типичной ошибки, при которой прокси перестаёт запускаться уже после перезапуска, когда разбираться некогда.

7.4. Установка корневого сертификата на рабочие места

Шаг обязателен **только** при выпуске сертификата собственным удостоверяющим центром.

> **ВАЖНО.** Без этого шага работники увидят предупреждение о недоверенном сертификате и не смогут его пропустить. Кнопки «перейти всё равно» не будет: система требует обязательного защищённого соединения, а это запрещает браузеру такой обход. Установка корневого сертификата снимает предупреждение полностью.

Файл для установки: `nginx/certs/ca/ca.crt`. Он одинаков для всех рабочих мест.

Централизованно в домене. Групповая политика: «Конфигурация компьютера» -> «Политики» -> «Конфигурация Windows» -> «Параметры безопасности» -> «Политики открытого ключа» -> «Доверенные корневые центры сертификации», импорт файла. Политика применится на рабочих местах при очередном обновлении.

Отдельная машина Windows. Из командной строки с правами администратора:

Листинг 7.4 — Установка корневого сертификата в Windows

```
certutil -addstore -f Root ca.crt
```

Рабочее место Linux.

Листинг 7.5 — Установка корневого сертификата в Linux

```
sudo cp ca.crt /usr/local/share/ca-certificates/buro-ca.crt
sudo update-ca-certificates
```

ОПАСНО. Приватный ключ удостоверяющего центра `nginx/certs/ca/ca.key` позволяет выпустить сертификат на любое имя, которому будут доверять все рабочие места организации. Он не должен покидать сервер и попадать в резервные копии общего доступа.

7.5. Продление сертификата

Сертификат, выпущенный собственным центром, действует 825 суток. За месяц до истечения выпустите новый той же командой: корневой сертификат переиспользуется, поэтому повторно устанавливать его на рабочие места не потребуется.

Листинг 7.6 — Продление сертификата собственного центра

```
./scripts/init-tls.sh self <адрес системы> <IP-адрес сервера>
sudo docker compose -f docker-compose.base.yml -f docker-compose.prod.yml \
  exec nginx nginx -s reload
```

Сертификат общедоступного центра действует 90 суток и требует регулярного продления. Настройте задание в планировщике суперпользователя, `sudo crontab -e`: продление ведёт `certbot`, а он

работает с каталогом `/etc/letsencrypt`, куда обычному пользователю доступа нет.

Листинг 7.7 — Задание на автоматическое продление

```
30 3 * * * cd /opt/systemburo && ./scripts/init-tls.sh renew \
  >> /var/log/systemburo-tls.log 2>&1
```

Команда безопасна для ежедневного запуска: она обращается за новым сертификатом только когда до истечения остаётся менее 30 суток, и перезагружает прокси лишь при фактическом обновлении файла.

Листинг 7.8 — Проверка срока действия

```
openssl x509 -in nginx/certs/fullchain.pem -noout -dates
```

Ожидаемый результат: две строки с датами начала и окончания действия.

Продление готового сертификата. Сертификат, полученный со стороны - приобретённый или выпущенный удостоверяющим центром организации, - система не продлевает и о его сроке не напоминает: она про этот срок ничего не знает. Отслеживает его тот, кто сопровождает сервер. За месяц до истечения возьмите у выдавшей стороны новую пару файлов и установите её той же командой, что при первой установке, после чего перезагрузите прокси.

Листинг 7.9 — Замена готового сертификата

```
./scripts/init-tls.sh import <файл цепочки> <файл ключа>
sudo docker compose -f docker-compose.base.yml -f docker-compose.prod.yml \
  exec nginx nginx -s reload
openssl x509 -in nginx/certs/fullchain.pem -noout -dates
```

Ожидаемый результат: сценарий сообщает об установке, последняя команда показывает новые даты. Заведите напоминание о сроке в том же месте, где ведутся остальные регламентные работы (подраздел 9.2): истёкший сертификат закрывает вход в систему всем сразу, а заголовок обязательного защищённого соединения не оставляет пользователю возможности пройти дальше предупреждения браузера.

7.6. Сервер за внешним входящим прокси

Внешний прокси может завершать защищённое соединение посетителя перед сервером системы. Сначала определите оба участка: браузер → внешний прокси и внешний прокси → nginx системы. Наличие сертификата на первом участке не определяет протокол второго.

Рабочий сервер. Штатный `nginx/nginx.conf` принимает рабочие обращения по HTTPS; HTTP перенаправляется на HTTPS. Внешний прокси должен обращаться к HTTPS-входу nginx, а сертификат этого входа обслуживается по разделу 7. Если прокси обращается только по HTTP, возможен цикл перенаправлений. Не заменяйте штатный конфиг примером, доверяющим произвольному заголовку `X-Forwarded-Proto`: такая топология требует отдельного согласованного решения.

Проверочный стенд. В `docker-compose.staging.yml` nginx слушает HTTP, а внешнее HTTPS-соединение обеспечивает входящий прокси. Ограничьте доступ к внутреннему HTTP-входу доверенной точкой входа средствами сети площадки. Рабочую схему нельзя получить простым переносом стендового конфига на production.

Адрес посетителя. В `EDGE_TRUSTED_PROXY_CIDRS` задают только подтверждённые адреса непосредственного внешнего прокси с длиной префикса. Не определяйте доверенный адрес по заголовку произвольного запроса и не открывайте доверие всей частной сети. Внешний прокси должен корректно формировать сведения об исходном посетителе. Подробности параметров и внутренней сети nginx → backend приведены в подразделе 4.3 и приложении Б.

При прямом подключении посетителей к ВМ внешний прокси отсутствует: `EDGE_TRUSTED_PROXY_CIDRS` оставляют пустым. Внутренняя сеть контейнеров и доверие backend к nginx настраиваются Compose независимо от этого.

Проверка. Выполните проверки подраздела 5.9 через тот адрес, которым будут пользоваться сотрудники. Подтвердите загрузку страницы и

вход, отсутствие цикла перенаправлений, действующий сертификат и корректный адрес посетителя в журнале при обычном запросе через рабочий маршрут. Ответ `/health` не подтверждает эти свойства.

Изменение файла параметров или конфигурации применяют пересозданием соответствующего контейнера. Перед публикацией проверьте итоговую конфигурацию, сохранив защитные заголовки и ограничения доступа. Реальные адреса и секреты в общую документацию не включают.

8. Проверочный стенд

8.1. Назначение

Проверочный стенд представляет собой второй экземпляр системы, развёрнутый отдельно от рабочего сервера. Он нужен, чтобы проверять изменения и настройки до их применения на рабочем сервере, обучать сотрудников без риска для данных и воспроизводить обращения пользователей.

> **ВАЖНО.** Не разворачивайте стенд на том же сервере, где работает рабочий экземпляр. Они разделят память и дисковую подсистему, и проверка нагрузки на стенде скажется на работе сотрудников.

8.2. Отличия от рабочего сервера

pgAdmin запускается только по требованию в профиле tools, с пределами 384 МиБ памяти и 0,25 CPU и политикой restart: unless-stopped. Обычный автоматический Deploy останавливает и удаляет только его контейнер, сохраняя том данных; следующий ручной запуск применяет актуальный образ и ограничения. Перед первым запуском новой версии поверх существующего тома сохраните отдельную копию каталога данных pgAdmin: обновление его SQLite требует собственного плана возврата. Проверка нового образа не доказывает совместимость конкретного старого тома. В рабочем Compose сервис pgAdmin не добавляется. Внутри образа

pgAdmin нет pip; дополнительные исправления и обновления выполняются пересборкой образа, а не установкой пакетов в работающий контейнер.

Таблица 8.1 — Отличия стенда от рабочего сервера

Свойство	Стенд	Рабочий сервер
Доступ	Закрыт запросом имени и пароля на входе, кроме программного интерфейса и проверки состояния, см. подраздел 8.5	Открыт, разграничение внутри системы
Подробность журнала	`debug`	`info`
Панель управления базой данных	Поднимается по требованию командой `sudo docker compose -f docker-compose.base.yml -f docker-compose.staging.yml --profile tools up -d pgadmin`, затем доступна по адресу `/pgadmin/`	Не разворачивается
Наполнение	Допустимы демонстрационные данные	Только рабочие данные
Обновление	Автоматическое: забирает из GHCR три готовых образа одного SHA — прикладной сервер, веб-приложение и обратный прокси	Вручную, со сборкой образов на сервере

ПРИМЕЧАНИЕ. Запрос имени и пароля на входе в стенд решает одну задачу, не пустить посторонних. Он не заменяет разграничение прав внутри системы.

8.3. Развёртывание стенда

Автоматическое выкладывание проверяет успешный Security для точного SHA, соответствие ветке dev и текущей вершине этой ветки. SSH-подключение требует `VPS_HOST_FINGERPRINT` из секретов окружения GitHub Actions. Отпечаток получают по независимому доверенному каналу, например через консоль управления виртуальной машиной, и сопоставляют с алгоритмом ключа, который согласует SSH-клиент поставки. Другой публичный ключ того же узла не заменяет ожидаемый ключ согласованного алгоритма. При смене ключа сначала подтверждают причину и новый отпечаток через независимый доступ, затем меняют секрет и проверяют новое подключение. Отключение проверки ключа не является восстановлением доступа. Перед Deploy выполняется отдельное SSH-подключение только с VPS_SSH_KEY и проверенным отпечатком: оно проверяет возможность выполнить id и обратиться к Docker. Поле пароля в

этих шагах отсутствует, резервного перехода на пароль нет. Такая проверка готовности ключа не меняет конфигурацию sshd и сама по себе не означает запрета парольного входа или входа root: политика SSH и сохранённый аварийный доступ принимаются отдельно.

Порядок совпадает с рабочим сервером, отличаются вызываемые команды.

Листинг 8.1 — Развёртывание стенда

```
make init-staging DOMAIN=<адрес стенда>
sudo make staging-build
sudo make staging-up
sudo make staging-seed PASS=<пароль администратора>
```

Ожидаемый результат: стенд доступен по указанному адресу, при открытии запрашивает имя и пароль, выведенные сценарием подготовки параметров.

> **ПРИМЕЧАНИЕ.** Приведённые команды собирают образы на самом стенде и требуют доступа к исходникам и базовым образам. Автоматические обновления используют готовые образы; их длительность зависит от сети и состояния сервера.

Для автоматического обновления CI собирает и проверяет шесть конечных образов одного SHA: прикладной сервер, веб-приложение, прокси стенда, прокси рабочего сервера, PostgreSQL и pgAdmin. На стенде постоянно запускаются четыре из них: прикладной сервер, веб-приложение, прокси стенда и PostgreSQL. pgAdmin остаётся инструментом профиля tools и запускается отдельно по требованию. Перед публикацией выполняется сканирование образов, проверка непривилегированного запуска прикладного сервера, наличия OpenSSL в прокси стенда, запуска и перезапуска PostgreSQL с сохранением данных. На стенде выбираются файлы той же ревизии, скачиваются её готовые образы и выполняется запуск с --no-build.

Переменные `BACKEND_IMAGE`, `FRONTEND_IMAGE`, `NGINX_IMAGE`, `POSTGRES_IMAGE` и `PGADMIN_IMAGE` задаёт процесс автоматического выкладывания. Это выбор готовых образов одного SHA; отсутствие

переменной сохраняет предусмотренное в Compose локальное имя и ручную сборку. PostgreSQL и pgAdmin также собираются и публикуются в GHCR, а первоначальная и автономная установка требуют подготовки доступных площадке образов. Наличие метки dev не подтверждает ревизию — проверяют точный SHA и фактический идентификатор образа.

ПРИМЕЧАНИЕ. Обратный прокси стенда принимает соединения по незащищённому протоколу и сертификата не требует - выпускать его командой из раздела 7 не нужно. Если стенд должен быть доступен по защищённому соединению, оно обеспечивается внешним прокси перед ним.

Демонстрационное наполнение добавляется отдельно:

Листинг 8.2 — Добавление демонстрационных данных

```
sudo make staging-seed-demo PASS=<пароль администратора>
```

8.4. Наполнение стенда вымышленными данными

Демонстрационный набор из предыдущего раздела невелик и неизменен. Чтобы проверять поведение системы на объёме - списки, поиск, отбор, показатели работы, печатные формы - стенд наполняют вымышленными данными: организациями, работниками, машинами, заявками и отметками проезда. Записи заводятся тем же путём, что и при работе руками, поэтому связаны между собой: у заявки есть организация, заявитель, согласующие, вложения с людьми и машинами из справочников, история обработки и отметки на постах.

ВАЖНО. Наполнение предназначено только для стенда. На рабочем сервере команда для него не предусмотрена, а сама она отказывается работать, пока экземпляр не отмечен как проверочный стенд.

Отметка ставится один раз:

Листинг 8.3 — Отметка экземпляра как проверочного стенда

```
sudo make staging-fake ARGS="-mark-stand"
```

Ожидаемый результат: сообщение о том, что экземпляр отмечен, с указанием имени базы данных.

Перед наполнением команда показывает, что будет создано, и ничего не записывает:

Листинг 8.4 — Предварительный показ

```
sudo make staging-fake ARGS="-profile=medium"
```

Наполнение выполняется отдельным запуском:

Листинг 8.5 — Наполнение стенда

```
sudo make staging-fake ARGS="-profile=medium -apply"
```

Ожидаемый результат: перечень созданного по видам записей, пароль заведённых работников и значение источника случайности. Повторный запуск с тем же значением источника даёт тот же набор данных - это позволяет воспроизвести обстоятельства, при которых обнаружилось замечание.

Отметки проезда и прохода в этот перечень не входят и показываются отдельной таблицей: они ставятся на уже созданных машинах и работниках, новых записей не добавляют и вместе с партией не удаляются отдельно - машина и работник принадлежат заявке и уходят вместе с ней. В предварительном показе их число приблизительное. Через пост проходят только те, чью заявку приняли в работу и чей пропуск на тот момент ещё действовал, зато отмечаются машины и работники в заявках, а одна запись справочника попадает в несколько заявок, поэтому отмеченных бывает и меньше обещанного, и больше.

Таблица 8.2 — Объём наполнения по заготовкам

Заготовка	Организаций	Работников	Заявок	Период дат
`small`	3	10	30	30 суток
`medium`	10	50	300	180 суток
`large`	50	500	5000	365 суток

Отдельные значения заменяются флагами: `-applications`, `-users`, `-cars`, `-employees`, `-days-back`. Полный перечень флагов выводит `make staging-fake ARGS="-help"`.

> **ПРИМЕЧАНИЕ.** Работники заводятся с общим паролем, который печатается в конце наполнения. Он предназначен для входа на стенде и не должен сохраняться в обращениях и постоянных журналах.

За каждого заведённого работника наполнение сразу расписывается в согласии на обработку персональных данных - той редакции текста, которая на стенде действует сейчас. Без этого стенд с включённым обязательным согласием при входе встречал бы каждого вымышленного работника окном согласия и дальше не пускал: данные в списках есть, а войти под ними нельзя. В записи согласия видно, что его проставило наполнение, а не человек из браузера.

Каждое наполнение образует партию с меткой. Партии перечисляются и удаляются целиком:

Листинг 8.6 — Перечень и удаление партий

```
sudo make staging-fake ARGS="-list"
sudo make staging-fake ARGS="-purge=<метка партии>"
sudo make staging-fake ARGS="-purge=<метка партии> -apply"
```

Ожидаемый результат: удаление снимает записи партии и сведения об их изменении. Данные, заведённые на стенде руками, остаются нетронутыми. Справочники и образцы вложений, которыми пользуются другие партии, тоже сохраняются и показываются в отчёте отдельным столбцом.

8.5. Ограничение доступа к стенду

Стенд доступен из сети, и его содержимое показывается только после ввода имени и пароля: браузер запрашивает их при первом открытии адреса. Проверку выполняет обратный прокси стенда. До прикладного сервера этот запрос не доходит, с учётными записями системы он никак не связан, и войти в систему всё равно потребуется отдельно. На рабочем сервере такого запроса нет.

Как устроено. Имя и пароль задаются параметрами `BASIC_AUTH_USER` и `BASIC_AUTH_PASS` в файле параметров стенда. При запуске прокси сценарий `nginx/staging-entrypoint.sh` создаёт `/etc/nginx/.htpasswd` с

необратимой свёрткой пароля. Проверка настроена в `nginx/staging.conf`, средство расчёта свёртки включено в `nginx/Dockerfile.staging`. Отдельно сценарий создаёт криптографически случайное значение признака `staging_auth` и записывает его в `/etc/nginx/staging-auth.conf`. Оно не выводится из имени и пароля, не хранится в исходном коде и меняется при каждом запуске контейнера.

Начальные значения создаёт сценарий подготовки параметров: имя `admin`, пароль случайный. Значения записываются в `.env`; вывод `make init-staging` содержит команду их просмотра, а не сами секреты. Просматривайте их только в доверенном сеансе, не копируйте в журналы и документацию.

ВАЖНО. Compose подставляет известные значения из поставки при отсутствующем или пустом значении параметров, поэтому проверьте случайный пароль в `.env` до первого запуска. При прямом запуске контейнера с пустым `BASIC_AUTH_PASS` сценарий прокси отказывает в запуске.

ВАЖНО. Браузер передаёт этот пароль в каждом запросе, и защищает его только само соединение. Пароль стенда не должен совпадать ни с одним паролем рабочего сервера.

Что закрыто, а что нет. Пароль спрашивается при обращении к веб-приложению и к панели управления базой данных. Часть адресов отвечает без него.

Таблица 8.3 — Адреса стенда, отвечающие без запроса пароля

Адрес	Почему открыт
`/health`	Проверка состояния мониторингом и конвейером сборки, выполняется без учётных данных
`/api/`	Программный интерфейс: обращения к нему предъявляют маркер доступа, полученный при входе в систему
`/api/events`	Поток событий; браузер получает одноразовый пропуск защищённым запросом `POST /api/events/ticket`, маркер доступа в адрес потока не помещается

ВАЖНО. Запрос пароля закрывает интерфейс, но не программный интерфейс. Данные стенда защищены входом в систему ровно так же, как на рабочем сервере, поэтому переносить на стенд рабочие персональные данные нельзя.

Описание программного интерфейса по адресу `/swagger/` пароль спрашивает наравне с самим интерфейсом. Прежде оно отвечало без пароля, и перечень всех методов вместе с версией сборки доставался любому желающему; поблажка ничего не давала, потому что пароль есть у каждого, кому описание нужно. Отдаётся оно по-прежнему только при `SWAGGER_ENABLED=true`.

Сколько действует однажды введённый пароль. После верного ввода прокси выдаёт браузеру признак `staging_auth` с предельным сроком семь суток. Он передаётся только по HTTPS, недоступен JavaScript страницы и имеет `SameSite=Strict`. При неверном вводе признак не выдаётся. При каждом запуске контейнера прокси создаётся новое случайное значение, поэтому все ранее выданные признаки перестают подходить даже без смены пароля: браузеру может снова потребоваться Basic Auth. Наличие доступа в другом браузере или старом сеансе не доказывает, что новый изолированный браузер уже авторизован.

> **ПРИМЕЧАНИЕ.** Признак помечен как передаваемый только по защищённому соединению. Если стенд открывается по незащищённому адресу, браузер его не сохранит, и пароль будет запрашиваться при каждом обращении.

Смена имени и пароля. Значения правятся в файле параметров стенда.

Листинг 8.7 — Строки файла параметров

```
BASIC_AUTH_USER=<имя>
BASIC_AUTH_PASS=<пароль>
```

Листинг 8.8 — Применение изменений

```
sudo make staging-up
```

Ожидаемый результат: контейнер прокси пересоздан, прежняя пара больше не принимается.

Листинг 8.9 — Проверка на сервере стенда

```
curl -sI -u <имя>:<пароль> http://localhost/
curl -sI http://localhost/
```

Ожидаемый результат: первая команда отвечает кодом 200, вторая - кодом 401 с заголовком `WWW-Authenticate`.

> **ВАЖНО.** Если стенд обходится автоматической проверкой страниц из конвейера сборки, новые значения нужно внести и в её настройки. Иначе проверка остановится на запросе пароля и начнёт сообщать о недоступности стенда.

Отключение и обратное включение. Отдельного параметра для этого нет, запрос пароля снимается правкой конфигурации прокси. В файле `nginx/staging.conf` в блоке `server` строка `auth_basic $staging_realm;` заменяется на `auth_basic off;`.

Листинг 8.10 — Перезапуск прокси стенда

```
sudo docker compose -f docker-compose.base.yml -f docker-compose.staging.yml \
  restart nginx
```

Ожидаемый результат: `curl -sI http://localhost/` отвечает кодом 200 без запроса пароля. Обратное включение - вернуть прежнюю строку и повторить перезапуск. Файл конфигурации подключён к контейнеру напрямую, пересобирать образ не требуется.

> **ВАЖНО.** Снимать запрос пароля допустимо, только если стенд закрыт от посторонних сетевыми средствами. Иначе открытым окажется всё, включая панель управления базой данных.

Проверка защиты стенда в изолированной среде

scripts/test-staging-auth.sh проверяет Basic Auth и заголовки на синтетическом локальном стенде. Скрипт требует STAGING_TEST_BASE_URL с localhost и портом, тестовый логин/пароль и STAGING_TEST_COMPOSE; проверяет локальный Docker socket. Он выполняет запросы и действия с изолированной средой, не предназначен для запуска на рабочем staging/production. Не подставляйте реальные реквизиты в пример или журнал. Проверка этого скрипта не заменяет браузерную приёмку фактически выложенной ревизии.

9. Эксплуатация рабочего сервера

9.1. Основные команды

Таблица 9.1 — Команды управления системой

Действие	Команда
Запуск	`sudo make deploy-up`
Остановка	`sudo make deploy-down`
Пересборка образов	`sudo make deploy-build`
Просмотр журналов	`sudo make deploy-logs`
Состояние служб	`sudo docker compose -f docker-compose.base.yml -f docker-compose.prod.yml ps`
Резервная копия сейчас	`sudo make deploy-backup`
Состояние копий	`sudo make deploy-backup-status`
Обзор занятого места	`sudo make deploy-storage`
Очистка накопленных данных	`sudo make deploy-cleanup`
Настройка файлового архива	`sudo make deploy-archive ARGS="show"`
Работа с данными одной организации	`sudo make deploy-entity ARGS="show -type=organization -id=N"`
Проверка живости сейчас	`sudo make deploy-health-check`

Команды выполняются под учётной записью, заведённой в подразделе 5.1, и требуют `sudo`: доступ к Docker выдан через него, а не членством в группе. Пароль спрашивается на первой команде и затем четверть часа не запрашивается.

Остановка прерывает работу пользователей. Прикладной сервер завершается корректно: перестаёт принимать новые запросы, даёт до 10 секунд на завершение текущих и только затем останавливается, поэтому незавершённых операций записи при штатной остановке не остаётся.

Команды, которые спрашивают подтверждение. Пароль, не запрашиваемый четверть часа, снимает последнюю преграду перед опечаткой: набранное не то имя команды выполняется молча, а двум командам из перечня ниже для порчи данных не нужно ни одного аргумента. Поэтому перед работой они спрашивают слово. Пока его набирают, читают, что именно произойдёт и почему этого не отменить.

Таблица 9.2 — Команды, требующие подтверждения

Команда	Что произойдёт	Слово
`sudo make deploy-seed`	Перезаписывается пароль супер-администратора, прежний перестаёт работать	`ПЕРЕЗАПИСАТЬ`
`sudo make deploy-seed-demo`	На рабочий сервер наливаются вымышленные организации, заявки, работники и машины	`НАЛИТЬ`
`sudo make deploy-cleanup ARGS="... -apply"`	Записи, попавшие под условия очистки, удаляются безвозвратно	`УДАЛИТЬ`
`sudo make deploy-entity ARGS="purge ... -apply"`	Данные организации удаляются из базы и с диска физически и необратимо	`СНЕСТИ`

Слово сверяется целиком. Любой другой ответ, в том числе пустой, прекращает работу строкой «Отменено, ничего не выполнено», и до самой операции дело не доходит. Запуск без человека за клавиатурой отвечает пустой строкой, то есть тоже прекращается. Очистка и снос данных организации спрашивают только в режиме применения: предварительный показ ничего не удаляет, а вопрос на безопасном вызове приучал бы отвечать не глядя. Прочие подкоманды `deploy-entity` вопроса не задают - они либо только читают, либо обратимы. Восстановление из копии устроено так же и спрашивает слово `ВОССТАНОВИТЬ`, подраздел 11.7.

Подтверждение снимается переменной `CONFIRM=yes`, заданной перед командой, - это нужно для запусков по расписанию и из сценариев. На рабочем сервере воспользоваться этим под учётной записью сопровождения не выйдет: `sudo` очищает окружение, поэтому запись `CONFIRM=yes sudo make deploy-seed` до цели не дойдёт, а задать переменную после `sudo` разрешение на две команды не позволяет. Обойтись без вопроса можно только с полными правами суперпользователя, где `sudo` в самой команде не участвует.

ВАЖНО. Подтверждение защищает от опечатки и невнимательности, но не от умысла. То же самое делается обращением к контейнеру напрямую, минуя `make`, и там никто ничего не спрашивает. Тот, кто вошёл под учётной записью сопровождения и знает её пароль, по-прежнему может всё, о чём сказано в подразделе 5.1.

9.2. Повседневные операции

Таблица 9.3 — Регламент повседневных работ

Операция	Периодичность	Кто выполняет
Контроль состояния служб	Ежедневно	Подразделение ИТ; поставленная на расписание проверка живости делает это сама и пишет при отказе, подраздел 12.2
Просмотр журнала на ошибки	Ежедневно	Подразделение ИТ
Просмотр журнала отказов в доступе	Еженедельно	Подразделение ИТ либо оператор системы с правом просмотра
Свежесть резервной копии	Ежедневно	Подразделение ИТ; полных прав на сервере для этого не требуется, подраздел 11.5
Контроль свободного места	Еженедельно	Подразделение ИТ
Контроль срока действия сертификата	Ежемесячно	Подразделение ИТ
Проверка восстановления из резервной копии	Ежемесячно	Подразделение ИТ

ПРИМЕЧАНИЕ. Наблюдение за журналами системы и разбор обращений пользователей доступны не только подразделению ИТ, но и операторам бюро пропусков, если им выдано соответствующее право. Журналы открываются прямо в интерфейсе системы, доступ к серверу для этого не нужен.

9.3. Управление учётными записями

Заведение и блокировка работников выполняются в интерфейсе системы и описаны в руководстве администратора. С точки зрения эксплуатации существенны три обстоятельства.

Блокировка закрывает служебные действия и чтение чужих данных, действующие сеансы отзываются. Предусмотренные системой операции с собственными данными рассматриваются отдельно; блокировку не используют как способ удалить эти данные.

Блокировку входа после неудачных попыток пароля не следует путать с блокировкой работника: первая временная и снимается сама по истечении срока. Срок растёт с каждой новой серией из пяти неудач - минута, пять,

пятнадцать, тридцать минут, час, - поэтому работник, который перебирает забытый пароль, ждёт всё дольше. Чтобы не заставлять его ждать, администратор снимает такую блокировку кнопкой в карточке работника; обращение к серверу для этого не нужно.

При проверке прав учитывайте серверный кэш до 30 секунд. Интерфейс обновляет состояние отдельно; если экран не обновился, перечитайте страницу и повторите допустимое действие. Видимая кнопка не подтверждает серверное разрешение.

Заведение учётной записи и первый вход

Пароль новому работнику придумывает система, а не человек, который его заводит. В форме создания поле пароля можно оставить пустым: тогда система придумает пароль по действующим требованиям и вышлет его работнику письмом вместе с логином и адресом системы. Тот, кто заводит учётную запись, пароля не видит и передавать его не должен.

Пустое поле допустимо не всегда, и отказ объясняет причину. Без указанного адреса почты придуманный пароль доставить нечем - система откажет и попросит либо заполнить адрес, либо задать пароль вручную. То же самое при ненастроенной почте: отправлять некуда, даже если адрес заполнен. Настройка почты описана в разделе 6.7.

Пароль, заданный администратором вручную, работает и остаётся способом завести учётную запись человеку без адреса почты - например, охраннику на посту. Передать такой пароль работнику придётся лично.

Свой пароль работник задаёт при первом входе - в обоих случаях. И придуманный системой, и заданный администратором пароль прошёл через чужие руки, а высланный письмом ещё и лежит в почтовом ящике открытым текстом. До тех пор, пока работник не задаст свой, система отвечает отказом на всё, кроме самой формы смены пароля: войти он войдёт, но работать не начнёт.

Порядок первого входа, когда включено требование согласия на обработку персональных данных (раздел 13.2): сначала работник соглашается с текстом, затем задаёт свой пароль, и только после этого система открывается. Оба окна появляются сами, идти за ними никуда не нужно.

Таблица 9.4 — Что происходит при заведении учётной записи

Адрес почты	**Поле пароля**	**Что делает система**
Указан, почта настроена	Оставлено пустым	Придумывает пароль, шлёт работнику письмо с логином и паролем, требует сменить пароль при первом входе
Указан, почта настроена	Заполнено	Сохраняет заданный пароль, шлёт работнику письмо с логином и этим паролем, требует сменить при первом входе
Указан, почта не настроена	Оставлено пустым	Отказывает: доставить пароль нечем
Не указан	Оставлено пустым	Отказывает: прочитать пароль работнику будет негде
Не указан	Заполнено	Сохраняет пароль, письма нет, требует сменить при первом входе

Срок действия паролей

Система умеет требовать от работников смены пароля по истечении заданного срока. В поставке эта возможность выключена: включать её должен человек осознанно. Паролей она не меняет и по почте не рассылает - работник, у которого срок вышел, входит своим прежним паролем, после чего система просит задать новый и до этого никуда его не пускает.

Включение. Раздел «Настройки», вкладка «Безопасность», блок «Срок действия паролей». Настроенная почта для включения не требуется: без неё не уйдут только предупреждения о скором истечении, а сама проверка сроков работает. Настроек четыре.

Таблица 9.5 — Настройки срока действия паролей

Настройка	**Допустимые значения**	**Что задаёт**
Требовать смену пароля по истечении срока	Включено, выключено	Общий выключатель. По умолчанию выключен

Продолжение таблицы 9.5

Настройка	Допустимые значения	Что задаёт
Срок действия пароля, суток	От 30 до 120, по умолчанию 90	Через сколько суток после последней смены пароль считается истёкшим. Верхняя граница - требование приказа ФСТЭК России N 21 для информационных систем персональных данных
Предупреждать заранее, суток	От 0 до 30, по умолчанию 7	За сколько суток до истечения работник получает письмо и уведомление. Ноль - не предупреждать. Требует настроенной почты и указанного у работника адреса
Требовать сменить пароль при первом входе	Включено, выключено	Относится к паролям, которые придумывает сама система: обновлению паролей всем работникам и смене из карточки. Такой пароль годится ровно на один вход - до тех пор, пока работник не задаст свой, система в остальном ему отказывает. Включено по умолчанию, выключать не следует: это единственное, что ограничивает срок жизни пароля, лежащего в почтовом ящике открытым текстом. По истечении срока действия смена требуется всегда, независимо от этой настройки

Проверка сроков выполняется ежесуточно в 04:00 по часовому поясу из параметра `RESET_TIMEZONE`. Отдельный планировщик в операционной системе для этого не нужен. Повторный проход того же дня никого не выберет заново: уже помеченные из отбора исключены.

Что показывает блок состояния. Числа в блоке считаются по тем же условиям, по которым отбирает работников сама проверка, поэтому увиденное на экране совпадает с тем, что произойдёт.

Таблица 9.6 — Показания блока состояния

Показание	Толкование
Ближайшая проверка	Когда планировщик проснётся в следующий раз
Срок действия пароля вышел у работников	У скольких работников срок уже истёк, то есть скольким система при следующем входе покажет форму смены пароля. Часть из них могла быть помечена в прошлые сутки, поэтому это не размер ближайшего прогона
Учётных записей с почтой	Действующие незаблокированные записи с указанным адресом. Столько писем уйдёт при обновлении паролей всем работникам
Без почты	Действующие незаблокированные записи без указанного адреса. Проверки сроков это не касается, но предупредить их заранее не получится

Обновление паролей всем работникам. Кнопка «Обновить пароль всем работникам» в том же блоке - отдельный инструмент, к сроку действия отношения не имеющий. Она придумывает новый пароль **всем** действующим незаблокированным работникам с указанным адресом почты, а не только тем, у кого срок вышел, и каждому высылает его письмом. Заблокированных и перенесённых в архив прогон не затрагивает. Кнопка показывается только при настроенной почте: доставить придуманный пароль без неё нечем. Прогон закрыт отдельным правом «Смена паролей всем работникам»: право на раздел настроек его не включает, поэтому работник, которому разрешено править контактный телефон бюро, сбрасывать пароли всей организации может быть не вправе. Администраторы получают это право вместе с полным доступом. Перед выполнением система показывает окно подтверждения с числом затрагиваемых работников.

ВАЖНО. Это исключительная мера, а не способ планового обновления. Прогон обрывает работу всех, кого он затронул: смена пароля аннулирует маркеры продления сеанса на всех устройствах, поэтому работники теряют открытые сеансы и входят заново - уже новым паролем из письма, а при включённом требовании смены сразу задают свой. Уместен он при подозрении на утечку паролей, при первичной раздаче паролей после ввода системы в эксплуатацию и после переноса на другой сервер. В рабочее время его выполнять не следует. Для повседневного обновления паролей служит срок действия, описанный выше: он ничего не рассылает и работу не прерывает.

ПРИМЕЧАНИЕ. Письма уходят очередью и с оглядкой на предел отправки (`SMTP_RATE_PER_HOUR`, по умолчанию 400 писем в час), поэтому при большом числе работников рассылка растягивается на часы: остаток очереди откладывается до следующего часа, но не теряется. Пароль при этом сменён сразу, а письмо к нему может прийти позже - учитывать это при выборе времени прогона.

Смена пароля одному работнику выполняется без общего прогона: в разделе «Пользователи», в карточке работника, кнопка «Сменить пароль и отправить письмом». Система придумает пароль по действующим требованиям и отправит его на указанный в карточке адрес. Это порядок для

случая «работник потерял пароль», и он тоже исключение, а не повседневность: пароль, придуманный системой, уходит по почте открытым текстом.

Работники без адреса почты. Проверке сроков адрес не нужен, и такие учётные записи она берёт наравне со всеми. Пропускают их только два действия выше - выслать придуманный пароль некуда, - и предупреждение о скором истечении: оно приходит письмом. Полный список открывается в разделе «Пользователи» переключателем режима списка в положение «Без почты». По итогам каждого прогона работники с правом на раздел настроек получают уведомление внутри системы, а при настроенной почте ещё и письмо с итогом. После проверки сроков в нём число помеченных истёкшими; после обновления паролей всем - сколько паролей сменено, сколько пропущено без адреса и перечень таких логинов (в письме первые двадцать, полный список смотрят в разделе «Пользователи»).

Свой пароль работник меняет сам в личном кабинете кнопкой «Сменить пароль», подтверждая действие текущим паролем. Сменив пароль заранее, работник начинает отсчёт срока заново, и требование смены его не застанет - именно к этому и подталкивает предупреждение, приходящее за несколько суток до истечения.

Запрет повторного использования паролей

Вернуться к паролю, которым учётная запись уже пользовалась, система не даёт. Проверяются десять последних паролей работника; попытка задать любой из них отклоняется сообщением «Этот пароль уже использовался. Придумайте пароль, которым вы раньше не пользовались». Действующий пароль при отклонённой попытке не меняется.

Правило одинаково для всех путей смены и обойти его, зайдя с другой стороны, нельзя: оно работает и когда работник меняет пароль сам, и когда пароль задаёт администратор из карточки, и при обновлении паролей всем работникам. Пароль, придуманный системой, тоже запоминается - иначе

работник, получив письмо, смог бы «сменить» пароль на тот же самый. Первый пароль запоминается вместе с заведением учётной записи.

Глубина в десять паролей - осознанный предел, а не круглое число. Пароли хранятся не в открытом виде, а хешами, и сравнение с перечнем означает вычисление хеша заново на каждую запись. Проверка всех паролей за всю историю работника заставила бы форму смены думать секундами; на десяти она укладывается в доли секунды. Пароль, ушедший за десятую позицию, снова разрешён.

После обновления запрет действует сразу, а не с первой смены. Перечень пополняется в момент смены пароля, поэтому у учётных записей, заведённых до появления запрета, он был бы пуст: первая смена запомнила бы новый пароль, а прежний - тот самый, от которого и уходят, - остался бы разрешённым. Чтобы такого промежутка не было, при первом запуске обновлённой системы действующий пароль каждой учётной записи вносится в её перечень. Отдельной команды это не требует и выполняется один раз.

Перечень прежних паролей закрыт: он не показывается ни в истории работника, ни в выгрузках, и не отдаётся ни одним методом. Хранятся только хеши - восстановить из них пароль нельзя. Записи глубже десятой удаляются сразу при смене пароля, поэтому перечень не растёт и отдельной уборки по расписанию не требует. Вместе с удалением учётной записи исчезает и её перечень.

> **ПРИМЕЧАНИЕ.** Запрет распространяется на пароли одной учётной записи, а не на все пароли системы. Совпадение паролей у разных работников система не отслеживает и отследить не может - хеши считаются с разной солью и между собой не сравниваются.

Вход в систему от имени работника

Работник жалуется, что раздел не открывается, список пуст или кнопка не работает, а на экране администратора всё в порядке. Чтобы увидеть систему его глазами, знать его пароль не нужно: администратор входит от его имени, а система пишет в журнал, что за этими действиями стоял он.

Как войти. Раздел «Пользователи», карточка работника, кнопка «Войти как пользователь» рядом с кнопкой «Права доступа». Открывается «Обзор и новости» - уже от имени выбранного работника: меню, разделы и списки те же, что видит он. Кнопка показывается только при праве «Вход от имени пользователя»; её нет для самого себя, для архивных и заблокированных учётных записей и для тех, от чьего имени входить не положено по полномочиям: от имени супер-администратора не входит никто, от имени администратора - только супер-администратор.

Отказ при нажатии означает одно из двух. Либо у выбранного работника есть право, которого нет у самого администратора, - войти от имени более полномочного система не даёт. Либо работнику назначена обязательная смена пароля: до неё его учётная запись закрыта и для него самого, и войти от его имени значило бы упереться в то же окно смены пароля. Так бывает у работников с истёкшим сроком действия пароля, а сразу после обновления паролей всем работникам - у каждого, кому пароль сменили, до тех пор, пока он не задаст свой.

Что на экране. Внизу постоянно висит полоса «Вы работаете от имени» с фамилией и именем работника, остатком времени сеанса и кнопкой «Вернуться в свою учётную запись». Полоса видна на любом экране и ничем не перекрывается - это единственный признак, отличающий чужой экран от своего.

Как вернуться. Кнопкой на полосе; открывается «Обзор и новости» уже под своей учётной записью. Права при возврате пересчитываются, поэтому раздел, открытый глазами работника, может оказаться недоступен - это ожидаемо. Сеанс заканчивается и сам: маркер действует 30 минут и не продлевается, по истечении система сообщает, что режим истёк, и возвращает администратора в его учётную запись. Если разбор не завершён, в режим входят заново.

Что в режиме не работает. Смотреть можно всё, что видит работник, менять - не всё. Закрыты смена паролей (своего, чужого и рассылкой всем),

общесистемные настройки, перенос учётных записей в архив и восстановление, смена типа, роли, признака администратора и групп прав, правка прав и ролей, блокировка и разблокировка, снятие блокировки входа, подтверждение и отзыв согласия на обработку персональных данных, а также вход от имени третьего работника. На такую попытку система отвечает отказом и предлагает вернуться в свою учётную запись. Перечень с обоснованием приведён в техническом описании системы, подраздел 13.3.

Что остаётся в журнале. Вход в режим и выход из него записываются в историю той учётной записи, от чьего имени работали, с указанием администратора, открывшего сеанс. Действия, выполненные внутри режима, дополнительно несут в журнале отметку об инициаторе - по ней они отличимы от действий самого работника.

> **ВАЖНО.** Режим не заменяет смену пароля и не предназначен для неё. Работнику, потерявшему пароль, меняют пароль кнопкой «Сменить пароль и отправить письмом» в той же карточке - система придумает пароль сама и отправит письмом. Заходить от имени работника, чтобы «проверить пароль» или задать ему новый, бессмысленно: пароль в режиме не проверяется, а смена пароля в нём запрещена.

9.4. Восстановление доступа супер-администратора

Супер-администратор в системе один. При утрате доступа к нему восстановление выполняется повторным запуском команды создания учётной записи: она не создаёт вторую запись, а обновляет пароль существующей.

Листинг 9.1 — Смена пароля супер-администратора

```
sudo make deploy-seed PASS=<новый пароль>
```

Прежде чем что-либо менять, команда спрашивает слово `ПЕРЕЗАПИСАТЬ` и показывает, что именно перезапишет. Вопрос задаётся до обращения к базе, поэтому запуск, прекращённый на этом месте, не меняет ничего.

Ожидаемый результат: команда завершается без ошибок, вход выполняется с новым паролем. Прочие данные не затрагиваются.

9.5. Режим технических работ

Режим включается супер-администратором в разделе системного управления. При включённом режиме все работники, кроме супер-администратора, попадают на страницу технических работ и не могут выполнять действия.

Режим следует включать перед обновлением и любыми работами, затрагивающими целостность данных. Он не останавливает фоновые задачи и не прерывает уже выполняющиеся запросы.

При включении заполняются четыре поля. Все они видны пользователю на странице технических работ, поэтому заполнять их следует так, как если бы вы отвечали на вопрос «когда всё заработает и к кому обращаться».

Таблица 9.7 — Данные, объявляемые при включении режима

Поле	Назначение
Сообщение	Что именно делается и чего ждать пользователю. Показывается крупно, первым
Начало работ	Дата и время начала. Отображается как срок, не как момент включения
Окончание работ	Дата и время окончания. Задаёт срок и одновременно служит предохранителем
Почта и телефон поддержки	Куда обращаться, пока система недоступна. Незаполненное поле не показывается

Начало и окончание задаются целиком: система не примет одну половину окна и не примет окончание раньше начала. По объявленному окну страница считает, сколько времени осталось, и показывает долю прошедшего времени.

Режим снимается сам, когда объявленное окончание прошло. Проверка выполняется при любом обращении к признаку режима, отдельная фоновая задача для этого не нужна. Снятие записывается в журнал событий как `maintenance_auto_disabled`. Поэтому даже забытый включённым режим отпускает пользователей в назначенный срок.

Вход супер-администратора при включённом режиме. Пока режим активен, обычного пользователя уводит на страницу технических работ с

любого адреса, включая форму входа. Для супер-администратора на странице технических работ есть ссылка «Вход для администратора» (адрес `/?admin`) - она открывает форму входа. Обычному работнику эта ссылка не поможет: его учётной записи система откажет во входе, пока идут работы.

Аварийное снятие режима. Если войти в систему не удаётся вовсе - утрачен пароль супер-администратора или недоступен интерфейс, - режим снимается на сервере одной командой. Прикладной сервер перезапускать не нужно: признак режима хранится не дольше десяти секунд.

Листинг 9.2 — Снятие режима технических работ на рабочем сервере

```
sudo make deploy-maintenance-off
```

Ожидаемый результат: команда печатает текущее значение признака (`maintenance.enabled | false`) и сообщение «Флаг режима выключен», пользователи возвращаются в систему в течение десяти секунд. На системе, где режим ни разу не включали, признака в настройках ещё нет, поэтому команда печатает только сообщение, а таблица значений выходит пустой - это тоже успешное выполнение, а не сбой. Для проверочного стенда команда называется `make staging-maintenance-off`, для локальной установки - `make maintenance-off`.

Восстановление самого доступа супер-администратора выполняется отдельно, порядок описан в подразделе 9.4.

9.6. Файловый архив бланков

Задания выгрузки и курсор сверки сохраняются в PostgreSQL (`archive_export_jobs`, `archive_export_scan_states`) и переживают перезапуск процесса. В затронутых операциях задание ставится в той же транзакции, что и изменение исходных данных. Исполнитель проверяет ревизию задания перед подтверждением результата; завершение старого задания не закрывает более новое. Сверка идёт ограниченными страницами и продолжает незавершённый обход после перезапуска. Выгрузка согласована с удалением и обезличиванием, чтобы старое задание не создало сведения

заново после их удаления. Это техническое свойство очереди, а не новая политика сроков хранения или разрешение читать исторический оригинал.

В папке каждой заявки лежат её бланки, машиночитаемое описание и подпапка «Приложения» с файлами, которые заявитель приложил при подаче. Имена приложенных файлов приходят от заявителя, поэтому они отделены от бланков: иначе одноимённый файл затёр бы печатную форму.

Система выкладывает сформированные бланки в каталог на диске сервера. Путь задаётся параметром `ARCHIVE_HOST_PATH`, раскладка внутри каталога и пределы занимаемого места - командой `server archive` на сервере. Раздел администрирования «Файловый архив» действующие значения показывает, но не меняет.

ВАЖНО. После установки выгрузка бланков выключена, и включает её администратор командой `sudo make deploy-archive ARGS="on"`. Ни установка, ни заполнение начальных данных её не включают, поэтому на новой площадке каталог архива остаётся пустым, сколько бы заявок ни прошло, а состояние бланков в карточках заявок показывается как «Пропущено». Первое, что стоит посмотреть при жалобе «бланки не появляются», - строку «Выгрузка бланков» в выводе `sudo make deploy-archive ARGS="show"`.

ОПАСНО. Каталог архива должен лежать вне каталога загруженных файлов, а каталог загруженных файлов - вне архива. Прикладной сервер проверяет это при запуске и с вложенным каталогом не стартует. Архив и загрузки обслуживаются разными механизмами доступа и хранения; их каталоги должны оставаться независимыми. Раздача загрузок проверяет действующий сеанс, разрешённый раздел хранения, а для приложений к заявкам — также доступ к заявке. Эти проверки не заменяют политику выдачи исторических бланков. Проверка ведётся по фактическому пути с разворачиванием символических ссылок, поэтому подложить каталог ссылкой не получится.

Каталог создаёт администратор до включения выгрузки и отдаёт его владельцу, под которым работает прикладной сервер: процесс не имеет права менять владельца файлов и на чужом каталоге остановится. Система создаёт подкаталоги правами 0750, файлы - 0640, поэтому содержимое читают только этот пользователь и его группа.

Листинг 9.3 — Подготовка каталога архива

```
sudo mkdir -p /srv/buro-archive
sudo chown 1001:1001 /srv/buro-archive
sudo chmod 0750 /srv/buro-archive
```

Ожидаемый результат: каталог существует, владелец совпадает с пользователем прикладного сервера.

ВАЖНО. Если `ARCHIVE_HOST_PATH` не задан, каталог создаётся сам при первом запуске - рядом с файлами системы и от имени администратора машины. Владелец у него окажется чужой, и запись не пойдёт: строки очереди уйдут в состояние ошибки с сообщением о запрете доступа. Проверить владельца и при необходимости выдать каталог прикладному серверу нужно и в этом случае, командами из листинга выше с путём фактического каталога.

Проверить, что каталог виден прикладному серверу под нужным владельцем, можно не заходя в файловую систему хоста:

Листинг 9.4 — Проверка владельца изнутри контейнера

```
sudo make deploy-archive ARGS="show"
sudo docker compose -f docker-compose.base.yml -f docker-compose.prod.yml exec backend ls -ld /app/archive
```

Ожидаемый результат: первая команда печатает каталог архива, вторая - строку, в которой владелец и группа совпадают с пользователем прикладного сервера.

Шифрование архива

При штатной подготовке параметров сценарий создаёт ключи архива и включает `REQUIRE_ENCRYPTION=true`: новые бланки сохраняются зашифрованными, запуск без необходимых ключей отклоняется. Незашифрованные архивы возможны на прежних установках, где требование было явно отключено; это не рекомендуемый режим новой установки. Ниже описана настройка получателей и чтения зашифрованного архива.

Порядок такой. Принимающая сторона создаёт пару ключей командой `age-keygen -o receiver.key`; открытый ключ (строка вида `age1...`) передаётся вам, закрытый остаётся у неё и никуда не пересылается. На сервере создаётся вторая пара - для самой системы: `age-keygen -o`

`/opt/systemburo/archive.key`. Затем в файле параметров заполняются `ARCHIVE_AGE_RECIPIENT` открытым ключом принимающей стороны и `ARCHIVE_AGE_IDENTITY` закрытым ключом системы, после чего службы перезапускаются.

Оба параметра задаются только вместе: открытый ключ без ключа системы служба отвергает при запуске, потому что иначе стала бы писать файлы, которые сама не прочитает, и выгрузка архива по кнопке в карточке заявки перестала бы работать.

После включения новые файлы получают имя с окончанием `.age` и открываются командой `age -d -i receiver.key -o бланк.xlsx 'бланк.xlsx.age'`. Ранее записанные файлы остаются как были и продолжают читаться.

Ключ получателя позволяет расшифровать файл вне приложения, но не подтверждает допустимость его выдачи. Через приложение выдача исторического оригинала подчиняется действующим ограничениям подраздела 9.6; при разрешённом скачивании файл расшифровывается на лету и не требует ручного ввода ключа. На диске он остаётся зашифрованным. Из карточки заявки получают актуальный бланк, а не неограниченный доступ к сохранённому оригиналу.

Перевести их в шифрованный вид можно двумя способами, и один другой не заменяет. Выгрузка задним числом за нужный период из раздела администрирования пересобирает бланки заново, и пересобранный файл пишется уже зашифрованным. Замороженные файлы она не трогает: их содержимое окончательно и повторно не формируется. Такие файлы закрывает отдельная команда - она берёт лежащий на диске файл и шифрует его как есть, не меняя содержимого.

```
sudo make deploy-archive ARGS="encrypt"
sudo make deploy-archive ARGS="encrypt -apply"
```

Ожидаемый результат: первая команда только считает и печатает, сколько открытых файлов найдено; вторая закрывает их и сообщает итог. Без

ключа `-apply` не изменяется ничего. Повторный запуск безопасен: уже закрытые файлы в работу не берутся, поэтому прерванный прогон достаточно повторить.

ВАЖНО. Команда нужна один раз - когда ключи задали позже, чем начали писать архив. Если ключи заполнены до первого запуска системы, открытых файлов в архиве не появляется вовсе. Чтобы такое не прошло незамеченным, при `REQUIRE_ENCRYPTION=true` служба отказывается запускаться с незаполненными ключами архива: требование шифрования, заявленное однажды, распространяется и на архив.

ОПАСНО. Закрытый ключ системы хранится там же, где ключ шифрования персональных данных, и на тех же условиях - вне резервных копий архива. При его утрате ранее записанные файлы читает только принимающая сторона своим ключом; если утрачены оба, содержимое не восстанавливается.

Шифрование файлов не отменяет защиты носителя. Раздел, на котором лежит архив, рекомендуется шифровать средствами операционной системы (для Linux это LUKS): это закрывает не только сами файлы, но и имена, размеры и структуру каталогов, по которым видно, сколько заявок и от каких организаций проходит через бюро.

Место сторожат три настройки, задаваемые командой `server archive set` на сервере. Порог предупреждения задаёт долю заполнения раздела, после которой ответственным за архив уходит уведомление; по умолчанию это 80 процентов. Наименьший остаток свободного места (по умолчанию два гигабайта) и предельный объём самого архива (по умолчанию не ограничен) - жёсткие пределы, при которых запись останавливается. Действующие значения показывает `server archive show` и раздел администрирования «Файловый архив».

Настройки архива задаются на сервере, а не через интерфейс: раскладка каталогов определяет, где физически лежат документы со сведениями работников, и менять её должен тот, кто отвечает за сервер. Команда выполняется в контейнере прикладного сервера, запускает её `make deploy-archive`, а подкоманда и её ключи передаются в `ARGS`.

Таблица 9.8 — Настройка файлового архива

Команда	Назначение
`sudo make deploy-archive ARGS="show"`	Действующие настройки, каталог из переменных окружения и пример готового пути
`sudo make deploy-archive ARGS="preview -dir Ш -file Ш"`	Путь по новым шаблонам на примере последней заявки, перечень плейсхолдеров, замечания к ошибочным
`sudo make deploy-archive ARGS="set ..."`	Изменение настроек: `-dir`, `-file`, `-quota`, `-min-free`, `-warn`, `-recheck`, `-freeze`, `-zip-max`
`sudo make deploy-archive ARGS="on"` и `ARGS="off"`	Включение и выключение выгрузки
`sudo make deploy-archive ARGS="encrypt"`	Подсчёт файлов, записанных до включения ключей шифрования; с ключом `-apply` - их закрытие

Размеры принимаются числом байт либо с единицей: `2G`, `512M`, `750K`. Перед изменением шаблона стоит посмотреть результат командой предпросмотра: шаблон без номера заявки сложит все заявки дня в один каталог, а неизвестный плейсхолдер команда отклонит. Каждое изменение записывается в журнал изменений с прежним и новым значением.

ВАЖНО. Путь ограничен по длине, чтобы файл открывался по сети с рабочего места. Русские буквы занимают в этом пределе вдвое больше латинских, поэтому длинные значения помещаются не всегда: наименование организации и в имени папки, и в имени файла предел исчерпывает. Когда места не хватает, система укорачивает имя самой глубокой папки - в предпросмотре это видно сразу, обрезанным окончанием. Одно и то же значение дважды в путь не ставят: организацию оставляют либо в папке, либо в имени файла. Справку по всем подкомандам печатает `make deploy-archive ARGS="help"`; на проверочном стенде та же операция называется `make staging-archive`.

Пример: увеличить срок заморозки до двух месяцев и поднять запас свободного места.

```
sudo make deploy-archive ARGS="set -freeze 60 -min-free 4G"
```

Ожидаемый результат: команда печатает сохранённые настройки и пример пути.

Уведомление о заполнении отправляется один раз на переход через порог, а не при каждой проверке. Когда место освободится и заполнение опустится ниже порога, отметка снимается, и следующее приближение снова вызовет уведомление.

ВАЖНО. При достижении жёсткого предела очередь выгрузки останавливается: строки переходят в состояние остановки и повторяются каждые пятнадцать минут, пока место не появится. Уже выложенные файлы не трогаются. Подача и обработка заявок не затрагиваются вовсе: архив не должен доедать место, нужное базе данных. Состояние видно в разделе администрирования.

Запись идёт фоновой очередью, отдельно от работы пользователя, поэтому файл появляется в каталоге не мгновенно, а в течение нескольких секунд после изменения заявки. Сорвавшаяся запись повторяется сама, а ночью система сверяет свой реестр с содержимым каталога и дописывает недостающее. Периодичность разбора очереди и повторов задают параметры `ARCHIVE_WORKER_TICK` и `ARCHIVE_SWEEP_INTERVAL`; менять их в обычной работе не требуется.

Файлы закрытой заявки через заданный срок замораживаются: они считаются окончательными и больше не переписываются, что бы ни правили в системе. Изначально это месяц с того дня, когда заявка перестала действовать; срок задаётся флагом `-freeze` той же команды.

Состояние архива смотрят в разделе администрирования «Файловый архив». Вкладка «Обзор» показывает занятое место, отдельно число заявок, бланков и машиночитаемых описаний, свободное место на разделе, момент последней записи, число ошибок записи и число вложений, оставшихся без бланка; ниже архив разложен по месяцам и по типам вложений. Вкладка «Лента» перечисляет записи целиком с отбором по состоянию и счётчиком очереди - по ней видно, что уже записано и что ещё ждёт разбора. Заявка в строке названа своим номером, вложение - наименованием типа, поэтому запись читается без обращения к базе. Оттуда же разбирают ошибки и запускают выгрузку задним числом за выбранный период - она понадобится после первого включения архива, чтобы выложить ранее поданные заявки, и после правки шаблона бланка, чтобы пересобрать печатные формы этого типа.

У каждого бланка есть состояние выгрузки. Оно видно и в ленте раздела администрирования, и рядом с вложением в карточке заявки, но подписи там короче - в таблице они приведены обе.

Первым столбцом идёт подпись в ленте, в скобках - подпись у вложения в карточке заявки, если она короче.

Таблица 9.9 — Состояния выгрузки бланка

Состояние	Что означает	Что делать
Ждёт очереди (в карточке «В очереди»)	Заявка поставлена в очередь, обработчик до неё ещё не дошёл	Ничего: файл появляется в течение нескольких секунд. Очередь, стоящая часами, - повод посмотреть журнал прикладного сервера
Записан (в карточке «В архиве»)	Файл записан, размер и контрольная сумма отвечают содержимому	Ничего
Ошибка выгрузки (в карточке «Ошибка»)	Запись сорвалась по временной причине: занятый файл, отказ диска, сбой формирования	Повторяется само с растущей паузой - от минуты до шести часов; срок следующей попытки подписан в строке ленты. Если состояние держится, разобрать причину: она записана там же
Нет шаблона	У типа вложения нет действующего Excel-бланка, формировать нечего	Приложить шаблон типу вложения и запустить выгрузку задним числом. Это пробел в архиве, а не решение администратора, поэтому он показывается отдельным счётчиком
Остановлено местом (в карточке «Нет места»)	Выгрузка остановлена нехваткой места на разделе или исчерпанной квотой	Освободить место либо поднять квоту. После этого очередь пойдёт дальше сама, повторять вручную не нужно
Вложение удалено	Вложения в системе больше нет, а файл в каталоге остался	Система такие файлы не удаляет: решение за человеком, с оглядкой на срок хранения документов
Пропущено	Выгрузка выключена целиком либо у этого типа вложения выключено автосохранение	Ничего, если так задумано. Выгрузка включается командой `sudo make deploy-archive ARGS="on"`, автосохранение типа - тумблером в карточке типа вложения

Выгрузка исторических оригиналов из файлового архива разрешается только действующему незаблокированному суперадминистратору с выполнением остальных проверок. Перед выгрузкой за период доступна оценка объёма; предел по умолчанию — два гигабайта. Архив собирается потоком, отдельная ZIP-копия на диске не создаётся. Выдача закрыта и

суперадминистратору, если включено действующее требование согласия: исторические файлы не позволяют надёжно проверить всех представленных в них пользователей. При выключенном требовании неурегулированный отзыв согласия или возражение также блокирует выдачу. Ошибка проверки не разрешает скачивание. В карточке заявки формируется актуальный бланк по текущим правам и маскированию; авторство не даёт отдельного доступа к историческому оригиналу. Сам факт хранения архива не подтверждает допустимость его выдачи. Обращения через приложение учитываются в журнале доступа к персональным данным.

ВАЖНО. Не удаляйте и не правьте файлы каталога мимо системы. Система считает выложенным то, что записала сама, и после ручного вмешательства её перечень перестаёт отвечать содержимому каталога. Ночная сверка перепроверяет заявки за последние 30 суток (глубина задаётся командой `server archive set -recheck`) и записывает недостающие файлы заново; для более ранних заявок то же делает выгрузка задним числом за нужный период. Замороженные бланки не восстанавливает ни то, ни другое: их содержимое окончательно и повторно не формируется, поэтому удалённый замороженный файл возвращается только из резервной копии.

9.7. Защита каталога архива

Каталог архива — служебное хранилище персональных данных. Обычный доступ к бланкам организуйте через приложение: так применяются текущие ограничения выдачи и ведётся аудит. Прямое чтение файлов, их расшифровка или выдача сетевой папкой проходят вне этих проверок. Общий ресурс не создаётся приложением и не нужен для штатной работы пользователей.

ОПАСНО. Не выдавайте сетевой доступ к каталогу как замену недоступному скачиванию из приложения. Наличие ключа расшифровки или права операционной системы не отменяет ограничения обработки и выдачи сведений после отзыва.

Права операционной системы должны оставлять доступ только службам и уполномоченному эксплуатационному персоналу в пределах

необходимых работ. Приложение выставляет права новых файлов при записи, но не исправляет автоматически ранее открытые каталоги или внешние общие ресурсы. При приёмке установки проверьте их отдельно; не меняйте владельца, группу или права действующего архива без оценки последствий.

Каталог не размещается в публичной файловой раздаче. При включённом шифровании расшифрованные бланки и слепки всё равно содержат открытые персональные данные; ключи хранятся отдельно. Обращения вне приложения не попадают в его журнал: если эксплуатационная операция требует такого доступа, её основание и учёт обеспечиваются отдельно. Текущая сверка документации не означает проверки или изменения существующих резервных копий и исторических каталогов.

9.8. Переименование базы данных

Если имя базы требуется изменить уже после запуска системы, порядок такой.

Листинг 9.5 — Переименование базы данных

```
sudo make deploy-down
sudo docker compose -f docker-compose.base.yml -f docker-compose.prod.yml \
  up -d db
sudo docker compose -f docker-compose.base.yml -f docker-compose.prod.yml \
  exec db psql -U postgres \
  -c 'ALTER DATABASE auto_registry RENAME TO buropropuskov;'
```

Затем измените в файле параметров `DB_NAME` и имя базы в конце строки `DATABASE_URL`, после чего запустите систему.

Листинг 9.6 — Запуск после переименования

```
sudo make deploy-up
```

Ожидаемый результат: система запускается, данные на месте. Операция мгновенная, данные не копируются.

ВАЖНО. Переименование выполняется при остановленном прикладном сервере. При активных подключениях база откажется переименовываться.

9.9. Очистка накопленных данных

Часть служебных записей система убирает сама, ежесуточно и без участия человека: недействительные маркеры продления сеанса, прочитанные и давние непрочитанные уведомления, подписки устройств на доставку вне системы, которым долго не удавалось доставить ни одного сообщения. Сроки задаются параметрами `REFRESH_TOKEN_RETENTION_DAYS`, `READ_NOTIFICATION_RETENTION_DAYS`, `NOTIFICATION_RETENTION_DAYS` и `PUSH_SUBSCRIPTION_RETENTION_DAYS`. У первых двух по умолчанию 30 суток, у третьего 90: непрочитанное уведомление человек ещё может открыть, поэтому оно живёт дольше прочитанного. У подписок устройств срок 180 суток: за полгода без единой успешной доставки устройство считается забытым. Менять сроки обычно не требуется: уборка касается записей, которыми уже нельзя воспользоваться.

> **ВАЖНО.** Параметры перечисленных сроков передаются через environment в docker-compose.base.yml. Их можно задать в .env и применить пересозданием службы по подразделу 6.1. Для иной конфигурации сначала проверить, что Compose действительно передаёт нужную переменную; не добавлять дубли environment без необходимости.

Автоматическая уборка также обрабатывает отработанные записи почтовой очереди (MAIL_MESSAGE_RETENTION_DAYS, по умолчанию 30 суток). Это не удаление писем из почтового ящика получателя.

История сущностей убирается автоматически только при явно заданном AUDIT_RETENTION_MONTHS больше нуля; ноль означает, что автоматическое удаление выключено. Сохраняются исключения для корзины и последних отметок входа/выхода. Срок ручной команды audit и умолчание фоновой задачи — разные настройки.

APPLICATION_RETENTION_MONTHS больше нуля включает необратимое обезличивание завершённых, отозванных и отказанных заявок. Заявки в работе не обрабатываются. Опорная дата берётся из завершения, затем отзыва, затем подачи; порция ограничена 200 заявками. Проверка

выполняется при запуске задачи и затем раз в сутки. Сохраняются заявка, даты, статус, решения и факт прохода; персональные сведения участников и связанные материалы обрабатываются механизмом обезличивания. До включения нужно согласовать основание и период хранения, резервные копии и повторное применение обезличивания после восстановления старой копии. Ноль оставляет эту автоматическую обработку выключенной.

Другие группы обслуживаются штатной командой очистки. Удаление необратимо, срок и выборку сначала проверяют без применения.

Таблица 9.10 — Группы данных, доступные команде очистки

Группа	Что удаляется	Срок по умолчанию
`tokens`	Недействительные маркеры продления сеанса	30 суток
`notifications`	Прочитанные уведомления	30 суток
`unread-notifications`	Непрочитанные уведомления	90 суток
`push-subscriptions`	Подписки устройств на доставку вне системы без единой успешной отправки	180 суток
`email-messages`	Отработанные записи очереди почты	30 суток
`audit`	История действий над сущностями	36 месяцев
`snapshots`	Суточные слепки таблиц постов	12 месяцев
`request-aggregates`	Суточные итоги журнала запросов	24 месяца

Команда без указания групп берёт две первые. Значение `all` берёт все восемь. Срок задаётся видом `30d` для суток и `12m` для месяцев и заменяет значение по умолчанию для всех выбранных групп.

Ключ `-except` вычитает группы из выбранных. Он избавляет от перечисления: чтобы почистить всё, кроме истории действий, достаточно `-targets=all -except=audit`.

Выборку внутри группы сужают три ключа.

Таблица 9.11 — Ключи сужения выборки

Ключ	Что задаёт	Где применим
`-from`	Начало периода в виде `2023-01-01`. Вместе со сроком хранения задаёт отрезок: всё, что старше начала, остаётся нетронутым	Во всех группах
`-entity`	Тип сущности: `car`, `employee`, `application` и другие	Только в группе `audit`
`-table`	Номер таблицы поста	Только в группе `snapshots`

Ключ, неприменимый к выбранной группе, команда отклоняет и объясняет почему. Это сделано намеренно: если бы такой ключ молча игнорировался, оператор просил бы сузить удаление, а получил бы удаление всей группы. По той же причине проверяется написание типа сущности, а при ошибке выводится перечень допустимых значений.

Без явного разрешения на удаление команда ничего не трогает и только показывает, сколько записей попало бы под условие. Это основной режим: сначала смотрят числа.

Разрешением служит ключ `-apply`, и с ним на рабочем сервере команда спрашивает подтверждение - слово `УДАЛИТЬ`, порядок в подразделе 9.1. Предварительный показ подтверждения не спрашивает: удалять ему нечего, а вопрос на безопасном вызове приучал бы отвечать не глядя. На проверочном стенде `make staging-cleanup` не спрашивает ничего и в режиме применения.

В отчёте по каждой группе видно, сколько в ней всего записей, сколько места она занимает и сколько освободит удаление. Оценка освобождаемого объёма приблизительная: записи внутри одной группы разного размера.

Листинг 9.7 — Что будет удалено

```
sudo make deploy-cleanup ARGS="-targets=all"
```

Ожидаемый результат: таблица с числом записей по каждой группе и напоминание, что ничего не удалено.

Листинг 9.8 — Удаление истории старше трёх лет

```
sudo make deploy-cleanup ARGS="-targets=audit -older-than=36m -apply"
```

Ожидаемый результат: та же таблица и строка с числом удалённых записей.

Листинг 9.9 — Очистка всего, кроме истории действий

```
sudo make deploy-cleanup ARGS="-targets=all -except=audit -apply"
```

Ожидаемый результат: в таблице семь групп вместо восьми, история действий не затронута.

Листинг 9.10 — История только по транспорту, за отрезок времени

```
sudo make deploy-cleanup ARGS="-targets=audit -entity=car -from=2023-01-01 -older-than=24m"
```

Ожидаемый результат: под удаление попадают записи о машинах за отрезок от начала 2023 года до границы срока. История людей и заявок не затрагивается, записи старше начала отрезка остаются.

Листинг 9.11 — Слепки одного поста

```
sudo make deploy-cleanup ARGS="-targets=snapshots -table=3 -apply"
```

Ожидаемый результат: удаляются суточные слепки только указанного поста. Номер поста виден в адресной строке при открытии его таблицы.

Справка по всем ключам выводится командой `make deploy-cleanup ARGS="-help"`. На проверочном стенде та же операция называется `make staging-cleanup`.

ВАЖНО. Удаление строк не возвращает место операционной системе. Освободившийся объём остаётся в файлах базы и используется под новые записи, поэтому после очистки размер каталога с данными сразу не уменьшится. Это нормально: место не потеряно, оно просто отдано базе, а не диску.

ВАЖНО. Удаление необратимо. Выполняйте его после создания резервной копии, иначе при ошибке в сроке восстанавливать будет нечего.

ПРИМЕЧАНИЕ. Историю сущностей команда чистит с оговорками. Записи об удалении строк таблиц постов сохраняются независимо от срока: по ним работает корзина. Последние отметки въезда и выезда каждой машины и каждого человека тоже сохраняются: из них берётся время последнего выезда в карточке, и у редко приезжающей машины такая отметка бывает старше любого разумного срока хранения.

ПРИМЕЧАНИЕ. Слепки таблиц, снятые вручную кнопкой в интерфейсе, команда не удаляет никогда. Под срок попадают только суточные, снимаемые системой перед сбросом статусов.

9.10. Архивация, обезличивание и удаление по ID

Отдельно от повседневной очистки (подраздел 9.9) система умеет работать с данными одной организации по её идентификатору: собрать всё, что ей принадлежит - заявки, вложения, машины, сотрудников, приложенные к заявкам файлы - в единый переносимый пакет, обратимо вывести организацию из активного оборота, необратимо обезличить её персональные данные или необратимо снести их из рабочей базы по уже снятому и проверенному пакету. Все операции выполняются подкомандой `entity` того же служебного бинаря, что и `cleanup`/`archive`/`storage`.

ВАЖНО. У подкоманды `entity` нет ни веб-интерфейса, ни отдельной кнопки в администрировании. Это сделано намеренно, тем же решением, что и для очистки данных: доступ к операции равен доступу к серверу, а не к учётной записи в системе. Кнопка «обезличить организацию» или «снести данные организации» в интерфейсе означала бы, что компрометация единственной учётной записи супер-администратора превращается в необратимую потерю или зачистку персональных данных.

Таблица 9.12 — Подкоманды `entity`

Команда	Назначение	Меняет данные
`show`	Показать состав данных, связанных с организацией	Нет
`export`	Снять переносимый зашифрованный пакет	Пишет пакет на диск и запись в журнал системы
`verify`	Проверить целостность и пригодность уже снятого пакета	Нет
`import`	Развернуть проверенный пакет на этот стенд	Да, только с ключом `-apply`. В базу, где организация ещё есть, не разворачивается: конфликт останавливает разворот
`retire`	Обратимо вывести организацию и её работников из активного оборота	Да, только с ключом `-apply`
`restore`	Вернуть организацию и работников, выведенных последним `retire`	Да, только с ключом `-apply`
`anonymize`	Необратимо затереть персональные поля организации	Да, только с ключом `-apply`, отката нет
`purge`	Физически снести данные организации по проверенному пакету	Да, только с ключом `-apply`, необратимо

У команд `show`, `export`, `retire`, `restore`, `anonymize` цель задаётся ключами `-type=organization -id=N` (тип сущности в этой редакции только один - организация). У `verify` обязателен ключ `-pkg` с путём к каталогу пакета; `-type`/`-id` необязательны и лишь дополнительно сверяют личность пакета. У `import` обязателен только `-pkg` - в отличие от `verify`, ключей `-type`/`-id` у него нет: сверить личность пакета перед разворотом можно только отдельным заранее выполненным вызовом `entity verify -type=... -id=...`. У `purge` обязательны сразу все три: `-type`, `-id` и `-pkg`. Полная справка по ключам каждой подкоманды - `sudo make deploy-entity ARGS="help"`.

entity show. Печатает, сколько строк в каждой связанной таблице принадлежит организации: заявки, вложения, машины, работники и так далее. Общие справочники (гражданства, марки, форматы номеров) и посты, которыми организация лишь пользуется, в подсчёт не входят - они принадлежат не организации, а системе целиком.

Листинг 9.12 — Состав данных организации

```
sudo make deploy-entity ARGS="show -type=organization -id=42"
```

Ожидаемый результат: таблица «имя таблицы - число строк» и общий итог. Если у организации нет связанных данных - строка «Связанных записей не найдено».

entity export. Собирает тот же состав данных в каталог `<тип>-<id>-<время>` внутри параметра `ENTITY_EXPORT_PATH` и в журнал системы. Без ключа `-apply` команда только считает и печатает состав пакета, ничего не записывая. С ключом `-apply` пакет пишется на диск: опись `manifest.json`, строки таблиц в `tables/*.jsonl`, приложенные к заявкам файлы в `files/`.

ВАЖНО. `ENTITY_EXPORT_PATH` обязателен и по умолчанию пуст - в пакете лежат все персональные данные организации разом, поэтому место хранения задаётся осознанно, а не подставляется каталогом рядом с приложением. Без него команда отказывает даже в пробном прогоне без `-apply`. Каталог обязан лежать вне `UPLOAD_PATH`: служебные пакеты не относятся к файлам, выдаваемым через веб-интерфейс загрузок. Доступ к каталогу пакетов ограничивают средствами операционной системы; проверка независимости путей сохраняется даже при защищённой раздаче загрузок.

На сервере этому параметру соответствует `ENTITY_EXPORT_HOST_PATH` - каталог хоста, тем же порядком, что и у файлового архива бланков (подраздел 9.6). Каталог нужно создать и передать во владение пользователю, под которым работает прикладной сервер, ДО первой выгрузки: без этого шага docker создаёт каталог сам, от суперпользователя, и запись пакета откажет ошибкой доступа.

Листинг 9.13 — Подготовка каталога пакетов выгрузки

```
sudo mkdir -p /srv/systemburo/entity-export
sudo chown 1001:1001 /srv/systemburo/entity-export
sudo chmod 0750 /srv/systemburo/entity-export
```

Ожидаемый результат: каталог существует, владелец совпадает с пользователем прикладного сервера - та же проверка, что и для архива в подразделе 9.6.

Файлы пакета закрываются age-конвертами на ключах `ARCHIVE_AGE_RECIPIENT` и `ARCHIVE_AGE_IDENTITY` - тех же, что использует файловый архив бланков (подраздел 9.6). Без этих ключей команда отказывает писать пакет, пока явно не передан ключ `-plaintext` - молчаливая запись пакета с персональными данными открытым текстом не предусмотрена.

Листинг 9.14 — Снятие пакета

```
sudo make deploy-entity ARGS="export -type=organization -id=42"
sudo make deploy-entity ARGS="export -type=organization -id=42 -apply"
```

Ожидаемый результат первой команды: таблица состава пакета и предупреждение, что ничего не записано. Второй - тот же состав, путь к

каталогу пакета, признак шифрования и, если снятие успешно, запись в журнале системы.

Реальное (не пробное) снятие пакета всегда оставляет в журнале системы запись: каталог пакета, число строк и файлов, признак шифрования, отпечаток описи. Персональных данных в записи нет - только метаданные, по которым позже можно отличить подменённый пакет от настоящего. Если запись в журнал не удалась, пакет с диска удаляется, а команда завершается ошибкой: пакет без следа в журнале не считается штатным результатом.

ВАЖНО. Паспортные и патентные поля внутри строк таблиц, если они зашифрованы ключом системы (`DATA_ENCRYPTION_KEY`), уезжают в пакет как лежат в базе - шифротекстом. Развернуть их можно только с тем же значением `DATA_ENCRYPTION_KEY`, что действовало на этой установке в момент снятия пакета. Об этом же команда предупреждает при выводе.

ПРИМЕЧАНИЕ. Таблица `employee_files` числится в схеме базы, но ни один путь кода в неё не пишет. Если в ней всё же оказались строки, команда выгрузит их как данные таблицы, но предупредит, что самих файлов взять неоткуда.

entity verify. Проверяет уже снятый пакет, не изменяя ни его, ни базу: годен ли он к развороту на этом или другом стенде. Ключ `-pkg` обязателен, ключи `-type`/`-id` необязательны и, если заданы, дополнительно сверяют, что пакет принадлежит именно ожидаемой организации, а не подменён или перепутан с другим.

Таблица 9.13 — Что проверяет `entity verify`

Проверка	Что означает несовпадение
Манифест на диске ровно в одном виде (открытый либо зашифрованный, не оба сразу)	Оба сразу - признак подмены или повреждения пакета, проверка отказывает сразу
Версия формата пакета	Пакет новее системы - обновите систему; пакет старше - разверните на стенде прежней версии
Отпечаток, размер и (для таблиц) число строк каждого файла пакета против описи	Файл повреждён, подменён или не дописан
Каждая строка `tables/*.jsonl` разбирается как объект	Файл таблицы повреждён
Файлы в каталоге пакета, которых нет в описи	Предупреждение, не отказ: пакет ещё может быть годен, но лишний файл нужно осмыслить

Продолжение таблицы 9.13

Проверка	Что означает несовпадение
Колонки каждой таблицы против схемы текущей базы	Колонка есть в пакете, а в схеме нет - разворачивать некуда, отказ; обратное - предупреждение, поле получит значение по умолчанию при развороте
Личность пакета (если заданы `-type`/`-id`)	Тип или идентификатор в манифесте не совпадают с ожидаемыми - явный отказ «это не тот пакет»

Листинг 9.15 — Проверка пакета

```
sudo make deploy-entity ARGS="verify -pkg=/app/entity-export/organization-42-20260811-140502"
sudo make deploy-entity ARGS="verify -pkg=/app/entity-export/organization-42-20260811-140502 -type=organization
-id=42"
```

Ожидаемый результат: таблица проверенных файлов с состоянием («ок» или причина отказа), затем предупреждения, затем причины отказа, и в конце - итоговая строка «Пакет годен.» либо «Пакет не годен.».

ВАЖНО. `verify` расшифровывает пакет только собственным ключом системы (`ARCHIVE_AGE_IDENTITY`) той установки, где команда запущена. Для проверки пакета на другом стенде на нём должен быть заведён тот же самый `ARCHIVE_AGE_IDENTITY`, что и на исходной установке.

ОПАСНО. У пакета, снятого с ключом `-plaintext` (без шифрования), опись сверяется сама с собой: вырезанную из неё запись и удалённый файл `verify` не заметит. У зашифрованного пакета такой лазейки в теории нет - конверт защищает содержимое от правки без переупаковки. Пакет, на который будут опираться при последующем удалении данных из рабочей системы, должен быть зашифрованным, а не снятым с `-plaintext`.

entity import. Разворачивает проверенный пакет на текущий стенд - обратная операция к `export`. Сама команда начинает с той же проверки, что и `verify`, по тому же каталогу: пакет не прошёл проверку - разворот не начинается вовсе. Дальше состав манифеста сверяется с картой данных организации (а не только с тем, что такие таблицы существуют в базе), и для каждой таблицы пакета проверяется, не заняты ли на этом стенде её идентификаторы: пакет разворачивается только на стенд, где ни один из этих идентификаторов ещё не встречается. При конфликте команда отказывает и печатает список задетых таблиц с примерами занятых номеров - слияние с уже существующими данными не поддерживается.

Без ключа `-apply` команда доходит до этой точки и печатает результат, ничего не записывая. С `-apply`: файлы заявок кладутся на диск, все таблицы пакета вставляются одной операцией с исходными идентификаторами, после чего правятся внутренние счётчики новых идентификаторов, а сам факт разворота попадает в журнал системы.

Листинг 9.16 — Разворот пакета

```
sudo make deploy-entity ARGS="import -pkg=/app/entity-export/organization-42-20260811-140502"
sudo make deploy-entity ARGS="import -pkg=/app/entity-export/organization-42-20260811-140502 -apply"
```

Ожидаемый результат первой команды: состав пакета по таблицам, список конфликтов идентификаторов (если есть) и напоминание повторить с `-apply`. Второй - тот же состав и строка «Импорт выполнен.».

Файлы заявок при развороте заново закрываются ключом целевой установки (`DATA_ENCRYPTION_KEY`, если он здесь задан) - их содержимое защищено правильно независимо от ключа исходного стенда. Паспортные и патентные поля внутри строк таблиц устроены иначе: они переносятся как есть, тем шифротекстом, каким их выгрузил исходный стенд, без перешифровки. Если манифест пакета отмечен таким шифрованием, команда предупреждает проверить совпадение `DATA_ENCRYPTION_KEY` на обеих установках - без совпадения эти поля лягут в базу назначения нечитаемым набором байт, и сам разворот об этом не узнает.

> **ВАЖНО.** `entity import` не сверяет личность пакета сам - в отличие от `verify`, у него нет ключей `-type`/`-id`. Если важно быть уверенным, что разворачивается пакет именно ожидаемой организации, эту проверку нужно сделать заранее отдельным вызовом `entity verify -type=... -id=...`.

entity retire и entity restore. Обратимый вывод организации из активного оборота: `retire` гасит саму организацию и всех её действующих работников одним действием, `restore` возвращает ровно то, что погасил последний `retire`. Строки из базы не удаляются, файлы не затрагиваются - меняется только признак активности.

Действующий супер-администратор организации гашению не подлежит - то же правило, что запрещает заблокировать владельца системы и в остальной части интерфейса; если такой работник есть, команда называет его отдельно и оставляет активным. Активные сеансы погашенных работников прекращаются, как при обычной блокировке работника из интерфейса; при возврате им потребуется войти заново.

Эта операция отдельна от архивации организации в интерфейсе: та блокируется, если у организации есть действующие работники, - `entity retire` такой блокировки не имеет и гасит организацию с работниками одним действием, что и требуется при завершении сотрудничества с контрагентом.

Листинг 9.17 — Вывод организации из оборота и возврат

```
sudo make deploy-entity ARGS="retire -type=organization -id=42"
sudo make deploy-entity ARGS="retire -type=organization -id=42 -apply"
sudo make deploy-entity ARGS="restore -type=organization -id=42 -apply"
```

Ожидаемый результат: без `-apply` - список того, что будет погашено или возвращено, с `-apply` - то же самое как факт, с напоминанием, что откат делается командой `restore` с теми же `-type`/`-id`. Без предшествующего `retire` (или если он уже возвращён) `restore` отказывает - подряд возвращать всё неактивное он не умеет и не должен.

ОПАСНО. Повторный `retire` над уже погашенной организацией ничего не гасит и не создаёт новой записи в журнале - иначе `restore` перестал бы находить список первого, настоящего вывода из оборота. Разбирать это как ошибку команды не нужно: сообщение «организация уже погашена» так и означает - возвращать нужно тем же `restore`, а не повторным `retire`.

entity anonymize. Необратимо затирает персональные поля сотрудников и пользователей организации. В отличие от `retire`, ничего не гасит и не архивирует: связи между записями, история, должности, номера машин и работа системы иначе не меняются - заявки и учётные записи продолжают существовать и обрабатываться как есть, но теряют идентифицирующие поля.

Таблица 9.14 — Поля, которые затирает `entity anonymize`

Таблица	Что затирается
Сотрудники заявок, реестр уникальных сотрудников организации, сотрудники в теле уже поданной заявки (три таблицы с одинаковым набором полей)	Фамилия, имя, отчество; серия и номер паспорта; номер патента (вместе с их отпечатками для проверки на повтор); основание пребывания
Заявки	Фамилия, имя, отчество и телефон инициатора подачи из шапки заявки - это может быть не сам отправитель, а другой указанный им человек
Предупреждения о совпадении с чёрным списком (только у элемента-сотрудника, не у элемента-машины)	Нормализованное имя своего сотрудника
Пользователи организации	Фамилия, имя, отчество, электронная почта, телефон, логин (заменяется псевдонимом `deleted_<id>` - обнулить нельзя, на логине держатся уникальность записи и вход)

Действующего супер-администратора организации команда не трогает - тот же запрет, что у `retire`, только здесь он строже: обезличивание меняет логин, и вход под прежним становится невозможен сразу, а это необратимо. Пропуск виден в результате команды и до, и после выполнения. Обезличенные пользователи теряют вход немедленно: их активные сеансы (маркеры продления) отзываются той же командой, тем же порядком, что и у `retire`.

Команда честно предупреждает про то, чего она НЕ трогает: файлы, приложенные к заявкам (сканы паспортов и патентов), - это тоже персональные данные, только в другом виде; слепки заявок в файловом архиве бланков (заявка.json, подраздел 9.6), если для заявок этой организации выпускались бланки, - там те же паспорт, патент, а также ФИО и телефон инициатора открытым текстом на момент выпуска бланка; и значение самой записи чёрного списка, с которой сравнили элемент, - это данные чужого человека, занесённого в список не этой организацией, обезличивание её не касается.

Листинг 9.18 — Обезличивание персональных данных

```
sudo make deploy-entity ARGS="anonymize -type=organization -id=42"
sudo make deploy-entity ARGS="anonymize -type=organization -id=42 -apply"
```

Ожидаемый результат первой команды: перечень полей под затирание и число строк, которые оно затронет. Второй - тот же перечень как свершившийся факт и предупреждения о том, чего команда не коснулась.

> **ОПАСНО.** У `anonymize` нет команды отката - действие необратимо. Перед его применением стоит убедиться, что организация действительно прекращает работу с системой, а не подавать команду по одному подозрению.

entity purge. Физически удаляет данные организации из рабочей базы и файлы заявок с диска - единственная команда раздела, которая по-настоящему освобождает систему от данных, а не переводит их в архив или в отключённое состояние. Снос идёт только по уже снятому и проверенному пакету: без него команда не работает вовсе.

У сноса три самостоятельные проверки, и отказ любой из них останавливает всё до единой строки удаления:

- пакет обязан пройти проверку `verify` С УКАЗАНИЕМ `-type` и `-id` цели - команда вызывает её сама; пакет другой организации или не прошедший проверку отклоняется сразу;
- пакет обязан быть зашифрован - у `export` есть обходной ключ `-plaintext`, у `purge` такого нет ни при каких условиях: опись открытого пакета сверяется сама с собой, и подмену такая проверка не увидит;
- пакет обязан ПОКРЫВАТЬ ТЕКУЩЕЕ состояние данных организации - счётчики строк каждой таблицы описи сверяются с фактическим состоянием и до начала, и повторно внутри самого удаления; расхождение в любую сторону (данные добавили или убрали после снятия пакета) останавливает снос, потому что копия устарела: он либо уничтожил бы то, чего в пакете нет, либо описывал бы состояние, которого уже нет.

У активной организации последняя проверка будет срабатывать часто и по мелочам - открыли заявку, появилась отметка прочтения, - это ожидаемо и не повод искать обход. Рабочий порядок для необратимого сноса:

Листинг 9.19 — Порядок необратимого сноса

```
sudo make deploy-entity ARGS="retire -type=organization -id=42 -apply"
sudo make deploy-entity ARGS="export -type=organization -id=42 -apply"
sudo make deploy-entity ARGS="verify -pkg=/app/entity-export/organization-42-20260811-140502 -type=organization
-id=42"
sudo make deploy-entity ARGS="purge -type=organization -id=42 -pkg=/app/entity-export/organization-42-20260811-
140502"
sudo make deploy-entity ARGS="purge -type=organization -id=42 -pkg=/app/entity-export/organization-42-20260811-
140502 -apply"
```

Сначала `retire`: погашенная организация и её работники больше не действуют, поэтому её данные перестают меняться и покрытие пакета не разойдётся с базой в промежутке между снятием пакета и сносом. Затем свежий `export` уже по погашенной организации, `verify` для дополнительной уверенности в личности пакета и, наконец, `purge` - сначала без `-apply` как последняя проверка, затем с ним.

Ожидаемый результат первых команд: состав пакета, число строк и файлов, предупреждения, если есть. Последней, с `-apply`, - строка «Снос выполнен. Данные удалены физически и необратимо.».

ОПАСНО. Перед сносом команда спрашивает слово `СНЕСТИ` - так же, как очистка накопленных данных спрашивает `УДАЛИТЬ` (подраздел 9.9), а создание администратора `ПЕРЕЗАПИСАТЬ`. Вопрос задаётся только на `purge` с ключом `-apply`; предварительный показ, `retire`, `restore` и остальные подкоманды его не спрашивают. Вопрос - последний предохранитель, а не проверка: к моменту его появления команда уже сверила пакет и покрытие. Строка с `-apply` отличается от безопасного предварительного показа одним ключом, то есть одной стрелкой вверх в истории командной оболочки, поэтому прочитайте строку целиком, прежде чем ответить.

Удаление строк и запись в журнал системы идут одной операцией: строки без следа в журнале не считаются удалёнными. Файлы заявок снимаются с диска только после того, как эта операция зафиксирована. Если снос строк прошёл и записан в журнале, а часть файлов убрать с диска не удалось, команда сообщает об этом отдельно и просит убрать их вручную - сам снос при этом уже состоялся, повторного запуска не требуется.

Если среди сносимых пользователей есть авторы общих (доступных не только автору) шаблонов отчётов, они не сносятся вместе с автором, а обрабатываются иначе. Перед удалением строк команда снимает с таких

шаблонов владельца - шаблон остаётся в системе и виден всем, кто им пользовался, но изменить или удалить его через раздел отчётов больше не может никто, то же состояние, что и у системных пресетов. Личных (не общих) шаблонов это не касается - они по-прежнему уходят вместе с автором как обычная часть графа. Число отвязанных таким образом шаблонов команда считает отдельно от удалённых строк - строка физически осталась в базе, удалением её считать нельзя, - и предупреждает о них ещё в пробном прогоне, до применения `-apply`.

> **ОПАСНО.** У `purge` нет отката и нет исключений из перечисленных проверок. Единственный способ вернуть снесённые данные - развернуть тот же пакет командой `import`: в рабочую систему, где организация уже снесена, он развернётся тоже, потому что её идентификаторы там больше не заняты, но это уже новая операция восстановления из копии, а не отмена сноса.

Устройство пакета

Пакет `entity export` - каталог на диске, а не файл-архив.

Листинг 9.20 — Устройство пакета

```
organization-42-20260811-140502/
  manifest.json[.age]          опись пакета
  tables/
    applications.jsonl[.age]
    attachments.jsonl[.age]
    ...
  files/
    000123[.age]
    000124[.age]
    ...
```

Опись (`manifest.json`) несёт версию формата, тип и идентификатор организации, время снятия, признак шифрования, а по каждой таблице - её имя, число строк, столбцы, отпечаток и размер файла; по каждому файлу вложения - исходную заявку, оригинальное имя и отпечаток. Таблица, у которой не нашлось ни одной строки, в опись не попадает вовсе - пустой файл в описи читался бы как «данные были и потерялись», а не «данных нет».

Имена файлов внутри пакета безличные, вида `files/000123` - настоящее имя (например, «Паспорт Иванова.pdf») лежит в описи, то есть под тем же шифрованием, а не в открытом виде на файловой системе.

В пакет не входят общие справочники (организации они не принадлежат) и файловый архив бланков - у него собственное хранилище и собственная выгрузка (подраздел 9.6).

ПРИМЕЧАНИЕ. При заданных `ARCHIVE_AGE_RECIPIENT`/`ARCHIVE_AGE_IDENTITY` пакет шифруется на ту же пару ключей, что и файловый архив бланков, отдельного ключа под эту выгрузку не заводится.

Перенос организации между контурами

Листинг 9.21 — Снятие и проверка пакета на исходной установке

```
sudo make deploy-entity ARGS="show -type=organization -id=42"
sudo make deploy-entity ARGS="export -type=organization -id=42"
sudo make deploy-entity ARGS="export -type=organization -id=42 -apply"
sudo make deploy-entity ARGS="verify -pkg=/app/entity-export/organization-42-20260811-140502 -type=organization
-id=42"
```

Ожидаемый результат последней команды: «Пакет годен.». Сверка личности пакета по `-type`/`-id` здесь не для полноты: сам `import` (см. ниже) её не делает, и это единственная точка, где можно убедиться, что переносится пакет именно этой организации.

Каталог пакета передаётся на целевой контур доступным вне системы способом (например, `rsync`/`scp` через доверенный канал) - сам перенос между машинами система не выполняет.

ВАЖНО. Если пакет зашифрован, на целевом контуре должен быть заведён тот же `ARCHIVE_AGE_IDENTITY`, что и на исходной установке - и `verify`, и `import` расшифровывают пакет только собственным ключом системы той установки, где запущены.

Листинг 9.22 — Разворот на целевом контуре

```
sudo make deploy-entity ARGS="import -pkg=/app/entity-export/organization-42-20260811-140502"
sudo make deploy-entity ARGS="import -pkg=/app/entity-export/organization-42-20260811-140502 -apply"
```

Ожидаемый результат первой команды - состав пакета и отсутствие конфликтов идентификаторов (при конфликте разворот на этот стенд невозможен, см. описание команды выше). Второй - «Импорт выполнен.».

Офбординг организации с возможностью вернуть

Листинг 9.23 — Снятие пакета, вывод из оборота, при необходимости - возврат

```
sudo make deploy-entity ARGS="show -type=organization -id=42"
sudo make deploy-entity ARGS="export -type=organization -id=42 -apply"
sudo make deploy-entity ARGS="retire -type=organization -id=42"
sudo make deploy-entity ARGS="retire -type=organization -id=42 -apply"
```

Снятие пакета перед выводом из оборота не обязательно для самого гашения - `retire` данные из базы не убирает, - но даёт дополнительную страховочную копию для хранилища компании.

Если решение отменяется:

```
sudo make deploy-entity ARGS="restore -type=organization -id=42 -apply"
```

Если же решение о прекращении сотрудничества окончательное, следующий шаг после `retire` и проверенного пакета - необратимый снос командой `purge`, описанный выше.

Ограничения

Команды `show`, `export`, `verify`, `import`, `retire`, `restore` объём базы не меняют. `anonymize` заменяет содержимое затронутых строк, но не удаляет их - объём базы остаётся тем же. Единственная команда, физически освобождающая место, - `purge`, но и она не заменяет очистку накопленных данных из подраздела 9.9: `purge` сносит целиком данные ОДНОЙ организации по решению о прекращении сотрудничества с ней, тогда как подраздел 9.9 выборочно убирает устаревшие технические записи (историю действий, маркеры продления, суточные слепки) у всех организаций разом, не трогая ни одну из них саму по себе.

Резервные копии и снапшоты команды `entity` не трогают ни при каких условиях.

ПРИМЕЧАНИЕ. Снятый пакет - это перенос данных в защищённый архив, а не их уничтожение. Пока пакет цел и проверен, данные восстановимы; фактическое уничтожение исходных данных в рабочей системе выполняет отдельная команда `purge`, а не сама выгрузка.

В этой редакции поддерживается только один тип сущности - организация (`-type=organization`).

9.11. Уход с системы

Если организация отказывается от системы целиком - решает сделать паузу или перейти на другую систему, - предусмотренного отдельной командой пути нет. Уход выполняется тем же порядком, что и любое прекращение эксплуатации сервера: снятием проверенной резервной копии, остановкой служб и удалением томов и каталогов с данными.

Шаг 1. Снимите резервную копию и проверьте её пригодность к восстановлению - без этого шага стирание необратимо в буквальном смысле.

Листинг 9.24 — Копия перед уходом с системы

```
make deploy-backup ARGS=pered-uhodom
AGE_IDENTITY=/путь/к/buro-backup.key make deploy-backup-verify
```

Ожидаемый результат: сообщение об успешном снятии копии, затем строка «копия пригодна для восстановления» с числами восстановленных записей.

ОПАСНО. Если проверка не прошла, стирать данные нельзя. Разберитесь с копированием по подразделу 11.5, повторите шаг 1 и только затем переходите дальше.

Шаг 2. Заберите копию с сервера защищённым каналом и сохраните вне его - на своём носителе или во внешнем хранилище (подраздел 11.6). После ухода с сервера копия окажется единственным местом, где остаются данные.

Шаг 3. Остановите службы.

Листинг 9.25 — Остановка

```
make deploy-down
```

Шаг 4. Удалите тома с данными и каталоги, лежащие на диске сервера вне томов Docker: файловый архив бланков по пути `ARCHIVE_HOST_PATH`, каталог пакетов выгрузки по идентификатору сущности по пути `ENTITY_EXPORT_HOST_PATH` (подраздел 9.10), файл параметров `.env`.

Листинг 9.26 — Удаление томов и файла параметров

```
docker compose -f docker-compose.base.yml -f docker-compose.prod.yml down -v
sudo rm -rf "$ARCHIVE_HOST_PATH"
sudo rm -rf "$ENTITY_EXPORT_HOST_PATH"
rm -f .env
```

Ожидаемый результат: тома `pgdata` и `uploads` удалены командой `docker volume ls`, каталог архива бланков, каталог пакетов выгрузки и файл параметров с сервера убраны.

ВАЖНО. Если снятые пакеты `entity export` ещё нужны, унесите их с сервера ДО этого шага - подраздел 9.10 описывает перенос каталога пакета доверенным каналом (`rsync`/`scp`). После удаления `ENTITY_EXPORT_HOST_PATH` снятые ранее пакеты восстанавливаются только из резервной копии, если она их застала - штатное резервное копирование этот каталог не охватывает (подраздел 9.10, «Ограничения»).

ВАЖНО. Файл параметров содержит пароли и ключ шифрования персональных данных. Если снятая на шаге 1 копия должна оставаться читаемой, сохраните файл параметров вместе с ней, отдельно от самого сервера, прежде чем удалять его здесь.

Возврат. Если решение об уходе отменяется, система поднимается заново на этом или на другом сервере по разделу 5 «Установка» либо по разделу 14 «Перенос системы на другой сервер», а данные восстанавливаются из снятой на шаге 1 копии командой `make deploy-restore` (подраздел 11.7).

9.12. Ключ шифрования персональных данных

Параметром `DATA_ENCRYPTION_KEY` зашифрованы сведения документов, удостоверяющих личность: серия и номер паспорта, номер патента. Резервная копия базы содержит те же зашифрованные значения, поэтому без ключа она бесполезна, а сам ключ в копию намеренно не входит - иначе он лежал бы вместе с данными, которые защищает.

Порядок хранения ключа устанавливает владелец установки. Система не предписывает ни способа, ни носителя: подойдут сейф, отдельный носитель, хранилище секретов организации, запечатанный конверт у ответственного лица. Обязательных требований два, и оба следуют из устройства защиты, а не из предпочтений: ключ хранится **отдельно от сервера** и **отдельно от резервных копий**, а доступ к нему имеет более одного человека, иначе отпуск или увольнение единственного держателя равнозначны утрате данных.

Проверять восстановимость нужно вместе с копиями: развернуть копию на отдельной машине, задать ключ из хранилища и убедиться, что сведения документов читаются. Проверка копии без ключа подтверждает только целостность файлов.

Система сама сверяет ключ при запуске. При первом запуске в служебной таблице заводится контрольная запись, зашифрованная действующим ключом. При каждом следующем запуске она расшифровывается и сверяется. Если ключ не соответствует данным - подставлен чужой, утрачен и создан заново, либо убран с зашифрованной базы, - прикладной сервер не поднимается и записывает в журнал причину. Данные при этом не изменяются.

> **ВАЖНО.** Отказ запуска здесь - защита, а не сбой. Раньше система в этом случае поднималась, показывала операторам вместо серии паспорта нечитаемый набор символов, и первое же сохранение такой записи шифровало её повторно - после чего не помогал уже и прежний ключ. Увидев в журнале сообщение о несовпадении ключа, не создавайте параметры заново: верните прежний ключ из хранилища либо переведите данные на новый порядком ниже.

Смена ключа. Правка `DATA_ENCRYPTION_KEY` в файле параметров данные не переводит - она делает их нечитаемыми. Смена выполняется командой `server reencrypt`, которая расшифровывает значения прежним ключом, зашифровывает новым и пересчитывает служебные значения поиска.

Порядок работы: включить режим технических работ в разделе системного управления (подраздел 9.5), снять резервную копию, прописать новый ключ в файл параметров и выполнить перевод.

Листинг 9.27 — Смена ключа шифрования

```
sudo make deploy-backup ARGS=pered-smenoy-klyucha
sudo docker compose -f docker-compose.base.yml -f docker-compose.prod.yml stop backend
# прописать новый ключ в DATA_ENCRYPTION_KEY файла параметров
# прежний ключ вводится скрыто; значение не включается в историю команд оболочки
read -rs -p "Прежний ключ: " OLD_KEY; echo
sudo docker compose -f docker-compose.base.yml -f docker-compose.prod.yml \
  run --rm --no-deps backend /app/server reencrypt -old-key "$OLD_KEY"
# продолжать только после успешной проверки объёма
sudo docker compose -f docker-compose.base.yml -f docker-compose.prod.yml \
  run --rm --no-deps backend /app/server reencrypt -old-key "$OLD_KEY" -apply
unset OLD_KEY
# продолжать только после успешного перевода
sudo docker compose -f docker-compose.base.yml -f docker-compose.prod.yml up -d backend
```

Отдельный контейнер команды получает новое значение из `.env`; `exec` в старом контейнере его не перечитает. На время перевода приложение остановлено, база остаётся запущенной. Команда с ключом доступна привилегированному наблюдателю процессов: выполняйте её только в доверенном административном сеансе. При любой ошибке оставьте приложение остановленным, сохраните оба ключа и разберите результат; не подменяйте ключ повторной генерацией. По окончании перевода запустите приложение, проверьте карточку и снимите режим технических работ там, где включали.

Без флага `-apply` команда ничего не пишет, а показывает, сколько значений подлежит переводу; выполнять её первой строкой стоит всегда. Прежний ключ сверяется с контрольной записью до первой записи в базу: ошибка в ключе, обнаруженная на середине прогона, оставила бы половину строк переведёнными. Весь перевод идёт одной транзакцией, и прерывание

отменяет его целиком - частично переведённую базу не открыл бы полностью ни один из двух ключей.

Пустое значение `-old-key` означает, что данные лежат открытыми: этим же порядком шифрование включается на установке, которая работала без него. Обратный случай - пустой `DATA_ENCRYPTION_KEY` при заданном `-old-key` - снимает шифрование и оставляет сведения документов в базе открытыми.

ОПАСНО. Прежний ключ нужен для перевода целиком. Начинать смену, не имея его на руках, бессмысленно: восстановить исходные значения будет нечем.

10. Порядок внесения изменений в систему

Раздел описывает полный путь изменения: от получения исходного кода до его появления на рабочем сервере.

10.1. Общая схема

Таблица 10.1 — Этапы внесения изменения

Этап	Где выполняется	Результат
1. Получение исходного кода	Рабочее место разработчика	Локальная копия проекта
2. Внесение правки	Рабочее место разработчика	Изменённый код в отдельной ветке
3. Автоматическая проверка	Конвейер сборки	Отчёт о прохождении проверок
4. Проверка на стенде	Проверочный стенд	Подтверждение работы вживую
5. Выкладывание на рабочий сервер	Рабочий сервер	Изменение доступно пользователям

10.2. Получение исходного кода и внесение правки

Исходный код хранится в системе контроля версий Git, в репозитории `git@github.com:buropropuskov/systemburo.git`. Порядок выдачи доступа описан в подразделе 4.1; на рабочем месте разработчика он выдаётся на его личную учётную запись, а не ключом развёртывания сервера. Работа ведётся в отдельной ветке, а не в основной: это позволяет проверить изменение до того, как оно попадёт к другим.

Листинг 10.1 — Получение кода и создание ветки

```
git clone git@github.com:buropropuskov/systemburo.git systemburo
cd systemburo
git checkout -b fix/kratkoe-opisanie
```

Локальный запуск для разработки поднимает систему с отдельной базой и не затрагивает ни стенд, ни рабочий сервер.

Листинг 10.2 — Локальный запуск для разработки

```
make up
make seed
```

Ожидаемый результат: интерфейс доступен на порту 8081, программный интерфейс на 8080, вход под учётной записью, созданной командой заполнения.

10.3. Проверка изменения до выкладывания

Перед отправкой изменения прогоняются тесты. Локально достаточно прогнать те, что относятся к изменённой части.

Листинг 10.3 — Прогон проверок

```
make test                           # бэкенд-тесты
cd frontend && npm run test:run     # фронтенд-тесты
cd frontend && npx playwright test # E2E-сценарии
```

Ожидаемый результат: все тесты завершаются успешно. Упавший тест выводит своё имя и расхождение ожидаемого с фактическим.

После отправки изменения конвейер выполняет применимые проверки, сборку и сканирование образов. До публикации сканируются шесть конечных образов: backend, frontend, nginx стенда, nginx рабочего сервера, PostgreSQL и pgAdmin. Найденные исправимые уязвимости высокого или критического уровня останавливают публикацию. Для PostgreSQL отдельно проверяются запуск, перезапуск и сохранность данных. Автоматическая поставка дополнительно проверяет статус обязательных процессов для точного SHA и текущую вершину целевой ветки. Ручная установка не обходит ответственность за эти проверки: её результат проверяют отдельно. Еженедельный Security обнаруживает новые известные проблемы, но не

устанавливает обновления; покрытие новых Docker-контекстов и закреплённых дополнительных пакетов механизмом обновления проверяют при сопровождении.

Загрузка страниц после обновления

Основные страницы поставляются отдельными хешированными JS-файлами и загружаются по переходу, с общей сессией и правами. JSZip загружается при скачивании архива. Это не отдельные приложения и не отдельные копии данных. Контроль состава загрузки выполняет frontend/build/check-bundle-size.js по Vite manifest; расчётный gzip не заменяет проверку Content-Encoding реального ответа.

В nginx JS/CSS обрабатываются отдельно от HTML: assets сохраняют сжатие и immutable-кэш, HTML проходит подстановку CSP nonce и не кэшируется как неизменный файл. Не переносите запрет upstream-сжатия HTML на /assets/. На время обновления используйте согласованный порядок технических работ; старые выпуски frontend специально не сохраняются. Если открытая вкладка запросила уже отсутствующий файл, приложение предлагает безопасное обновление с проверкой несохранённых изменений.

10.4. Проверка на стенде

После прохождения тестов и включения изменения в основную ветку оно автоматически выкладывается на проверочный стенд.

Убедитесь, что этап выкладывания завершился успешно для нужного SHA. Сопоставьте эту ревизию с рабочей копией на стенде, ссылками и идентификаторами фактически запущенных образов всех четырёх служб: backend, frontend, nginx и PostgreSQL. У образов nginx и PostgreSQL дополнительно проверьте метку `org.opencontainers.image.revision`. Подвижная метка `dev`, один лишь номер версии рабочей копии или ответ `/health` не доказывают совпадение всей поставки. После проверки образов выполните реальный запрос к затронутому API и проверьте соответствующий экран и действие под нужной ролью.

На стенде проверяются сценарии, которые автоматика не покрывает: внешний вид, удобство, поведение на реальных данных.

Таблица 10.2 — Проверяемые на стенде сценарии

Сценарий	**Что подтверждает**
Вход администратора и рядового работника	Проверку подлинности и выдачу прав
Подача заявки со всеми типами вложений	Основной рабочий процесс
Приём в работу, согласование, завершение	Переходы состояний и запись решений
Открытие таблицы поста, отметка въезда и выезда	Работу постов
Формирование печатной формы по заявке	Работу шаблонов бланков
Выгрузка отчёта	Формирование электронных таблиц
Открытие панели показателей	Расчёт аналитики

10.5. Выкладывание на рабочий сервер

Автоматический процесс `deploy-production.yml` работает с веткой prod. Ручной запуск требует подтверждения deploy; перед подключением проверяются успешные CI и Security для точного SHA, принадлежность ветке и её текущая вершина, а также доверенный `VPS_HOST_FINGERPRINT`. На сервер поступают готовые frontend/backend/nginx-prod/PostgreSQL того же SHA; сборка на сервере и перезапись общих меток dev в этом процессе не выполняются. Права чтения GHCR и независимый аварийный доступ к виртуальной машине подготавливают до запуска. Описанный ниже порядок ручной сборки применяется отдельно и не заменяет проверки автоматической поставки.

ОПАСНО. Перед выкладыванием создайте и проверьте резервную копию базы данных. Возврат только образов допустим, если прежняя версия совместима с текущей схемой и данными. При несовместимом изменении нужен согласованный возврат совместимого комплекта базы, файлов, конфигурации и ключей; восстановление может потерять изменения после точки копирования. Порядок и ограничения приведены в подразделе 10.7.

Порядок ручного обновления из исходников на рабочем сервере.

1. Предупредить пользователей, при длительном обновлении включить режим технических работ (подраздел 9.5).

2. Получить новую версию исходного кода в каталог системы.

Листинг 10.4 — Обновление исходного кода на рабочем сервере

```
cd /opt/systemburo
git fetch origin
git checkout <версия>
```

ВАЖНО. Если в каталоге системы есть местные правки файлов описания - пересчитанные параметры базы, настройка прокси, - переключение версии либо перенесёт их дальше, либо откажется выполняться. Порядок в подразделе 10.7, врезка про местные правки; посмотреть, что правлено на этой площадке, помогает `git status`.

3. Пересобрать образы и перезапустить систему.

Листинг 10.5 — Пересборка и перезапуск

```
sudo make deploy-build
sudo make deploy-up
```

4. Проследить за журналом запуска.

Листинг 10.6 — Наблюдение за запуском после обновления

```
sudo make deploy-logs
```

Ожидаемый результат: в журнале нет сообщений уровня `error`. Схема базы приводится в соответствие автоматически именно на этом шаге, и здесь же проявятся несовместимости, если они есть.

5. Выполнить проверки из подраздела 5.9.
6. Снять режим технических работ.

Пользователи не смогут работать с момента перезапуска до успешного старта, обычно это одна-две минуты.

Обновление на площадке без доступа к репозиторию. Если система развёрнута из архива, второй шаг выполняется иначе: команды `git` в таком каталоге не работают, потому что рабочей копии репозитория там нет. Новая версия передаётся архивом и распаковывается поверх прежнего каталога, после чего порядок продолжается с третьего шага без изменений.

Листинг 10.7 — Обновление кода из архива

```
cd /opt/systemburo
tar -xzf systemburo-<новая версия>.tar.gz --strip-components=1
```

ВАЖНО. Распаковка поверх заменяет файлы кода, но не удаляет лишние. Файл параметров `.env`, каталог сертификатов `nginx/certs` и каталог `nginx/acme-webroot` в архиве отсутствуют и остаются нетронутыми - именно поэтому распаковывают поверх, а не в очищенный каталог. Обратная сторона в том, что файлы, удалённые в новой версии, остаются на диске. Если есть основания думать, что это мешает, сохраните `.env` и `nginx/certs` в стороне, разверните новую версию в пустой каталог и верните их на место.

10.6. Изменения схемы базы данных

Схема приводится в соответствие автоматически при каждом запуске прикладного сервера, отдельной команды не требуется. Практические следствия:

- изменения применяются в момент запуска, и если они объёмны, запуск затянется;
- изменения схемы носят добавляющий характер: создаются таблицы, столбцы и индексы, столбцы и таблицы не удаляются;
- вместе со схемой при запуске выполняются разовые правки данных, если новая возможность их требует: новый столбец или таблица наполняются по уже имеющимся сведениям, а сведения, которые больше нельзя хранить, стираются. Каждая такая правка выполняется один раз и при повторном запуске ничего не делает;
- при ошибке приведения схемы прикладной сервер завершает работу и не запускается в частично обновлённом состоянии.

10.7. Возврат к предыдущей версии

Возврат кода выполняется получением прежней версии и повторной сборкой. Прежнюю версию находят по журналу изменений рабочей копии: он лежит в самом каталоге системы, поэтому перечень доступен и при недоступном репозитории.

Листинг 10.8 — Перечень последних версий

```
cd /opt/systemburo
git log --oneline -10
```

Ожидаемый результат: десять последних изменений, у каждого свой определитель в начале строки. Его и подставляют в команду возврата.

Листинг 10.9 — Возврат к предыдущей версии кода

```
cd /opt/systemburo
git checkout <предыдущая версия>
sudo make deploy-build
sudo make deploy-up
```

ВАЖНО. В каталоге системы обычно лежат местные правки файлов описания - пересчитанные параметры базы (подраздел 6.6) и настройка обратного прокси под площадку (подразделы 7.6, 13.3). Переключение версии их не выбрасывает: если возвращаемая версия те же файлы не меняла, правки переносятся вместе с переключением, и `git status` показывает их на месте. Но если версия эти файлы меняет, переключение не выполнится вовсе - Git откажется затирать несохранённое. Тогда правки сначала убирают в сторону (`git stash`) либо копируют файлы рядом, переключаются, и после сборки возвращают значения обратно. Перед возвратом версии полезно посмотреть `git status`, чтобы знать, какие файлы правлены на этой площадке.

ВАЖНО. На площадке без доступа к репозиторию возврата этими командами нет: рабочей копии в каталоге не существует, а вместе с ней и перечня прежних версий. Возврат там выполняется распаковкой прежнего архива поверх каталога, как описано в подразделе 10.5. Поэтому предыдущий архив поставки хранят до тех пор, пока новая версия не отработает без замечаний.

ВАЖНО. Приведение схемы базы выполняется только вперёд. Изменения схемы, внесённые новой версией, при откате кода не отменяются. Поскольку они носят добавляющий характер, прежняя версия обычно продолжает работать. Но если обновление изменило смысл существующих данных, потребуется восстановление базы из копии, созданной до обновления.

Для автоматически обновляемого стенда возврат готовых образов выполняют отдельно от приведённой выше сборки рабочего сервера. Выберите ранее успешно поставленный SHA и проверьте наличие согласованного комплекта четырёх runtime-образов — прикладного сервера, веб-приложения, прокси стенда и PostgreSQL — вместе с файлами

конфигурации этой ревизии. Из шести собираемых CI образов прокси рабочего сервера не используется на стенде, а pgAdmin остаётся отдельным инструментом tools. Уборка может удалить прежние локальные метки; dev обозначает текущую поставку, а не предыдущую. До запуска проверьте совместимость прежних версий приложения и PostgreSQL с текущей схемой, данными и томом БД. Не подменяйте образ PostgreSQL прежней версией автоматически: возврат контейнерного образа не выполняет обратное преобразование формата данных и не отменяет миграции схемы. При несовместимости нужен отдельный согласованный план восстановления совместимого комплекта базы, файлов, конфигурации и ключей из проверенной копии; он может потерять изменения после точки копирования. Только после проверки совместимости запускайте выбранный комплект без пересборки и повторите проверку фактических образов, revision-меток nginx и PostgreSQL, API и экрана из подраздела 10.4.

11. Резервное копирование и восстановление

11.1. Устройство

Копирование выполняется средствами системы: набор команд в составе поставки снимает копию, раскладывает её по срокам хранения, удаляет устаревшие и проверяет, что копия пригодна для восстановления. Запускается по расписанию, участия человека не требует.

Резервное копирование, предоставляемое как услуга хостинга, работает на уровне образа сервера целиком. Оно позволяет вернуть сервер в прежнее состояние, но не заменяет копию базы: из образа нельзя извлечь отдельную таблицу, отменить последствия ошибочной операции или перенести данные на другой сервер. Одно другому не мешает: образ защищает от отказа сервера, копия базы - от ошибки в данных.

11.2. Что входит в копию

Копирования одной базы данных недостаточно.

Таблица 11.1 — Состав данных для резервного копирования

Элемент	**Где расположен**	**В копии**	**Последствия утраты**
База данных	Том `pgdata`	Ежедневно	Полная утрата заявок, реестров, журналов, учётных записей
Загруженные файлы	Том `uploads`	По расписанию файлов	Утрата фотографий, шаблонов бланков, приложений к заявкам. Записи в базе останутся со ссылками на отсутствующие файлы
Файловый архив бланков	Каталог `ARCHIVE_HOST_PATH`	По расписанию файлов	Утрата сформированных бланков, доступных снаружи системы
Файл параметров	`.env` в каталоге системы	**Не входит**	Утрата ключа шифрования персональных данных
Сертификат удостоверяющего центра	`nginx/certs/`	**Не входит**	Потребуется повторный выпуск и повторная установка корневого сертификата на все рабочие места

База выгружается каждый раз, а файлы - раз в неделю: полный архив файлов, снимаемый ежедневно, умножается на число хранимых копий и занимает место несоразмерно своей ценности. Если объём файлов невелик, режим переключается параметром `BACKUP_UPLOADS_MODE` в значение `daily`.

Сначала выгружается база, затем архивируются файлы. Порядок существенен: при обратном файл, загруженный между двумя шагами, попал бы в базу, но не в архив, и после восстановления заявка ссылалась бы на отсутствующий файл.

ОПАСНО. Файл параметров в копию не входит намеренно. В нём ключ шифрования персональных данных, и хранение его вместе с копией базы означает, что кража архива равносильна краже персональных данных в открытом виде. Скопируйте `.env` отдельно, в хранилище секретов организации, и обновляйте эту копию при каждом изменении параметров, соблюдая правило хранения секретов из подраздела 5.4. Без ключа восстановленная база не отдаст ни одного паспорта.

11.3. Сроки хранения

Таблица 11.2 — Сроки хранения копий

Каталог	Что попадает	Сколько хранится
`daily`	Каждая копия	7 последних
`weekly`	Копия за воскресенье	4 последних
`monthly`	Копия за первое число	6 последних

Одна и та же копия попадает в несколько каталогов и занимает место один раз: используется жёсткая ссылка, а не второй экземпляр файла. Числа задаются параметрами `BACKUP_KEEP_DAILY`, `BACKUP_KEEP_WEEKLY`, `BACKUP_KEEP_MONTHLY`.

11.4. Настройка

Настройка выполняется один раз, на самом сервере системы. Все команды подраздела вводятся в каталоге `/opt/systemburo`. Установка расписания ставит службы и таймеры в систему, поэтому требует полных прав суперпользователя: разрешение на две команды, выданное в подразделе 5.1, её не покрывает. На проверочном стенде порядок тот же, отличаются только имена команд: вместо `deploy-backup` вводится `staging-backup`.

Шаг 1. Создайте пару ключей шифрования. Открытая часть остаётся на сервере и шифрует копии, закрытая нужна только для восстановления и на сервере не хранится.

Шаг обязателен. Пока открытый ключ не записан в параметр `BACKUP_AGE_RECIPIENT`, копирование отказывается работать: оно останавливается до первого обращения к базе, копию не создаёт вовсе и

печатает, чего ему не хватает. Проверка стоит именно в начале, чтобы незашифрованной выгрузки не появилось даже на минуту.

ОПАСНО. В выгрузке базы и в архиве бланков фамилии, сведения документов, удостоверяющих личность, и номера патентов. Без шифрования кража архива копий равносильна краже самой базы, причём незаметной: копии никто не открывает, пока не понадобится восстановление. Поэтому отсутствие ключа останавливает копирование, а не отмечается предупреждением, которое читают уже при разборе аварии.

Листинг 11.1 — Создание ключей шифрования копий

```
age-keygen -o buro-backup.key
```

Ожидаемый результат: выведена строка вида `Public key: age1...`, создан файл `buro-backup.key`.

Если средство `age` на сервере не установлено, ставить его не требуется: та же пара ключей создаётся одноразовым контейнером.

Листинг 11.2 — Создание ключей без установки средства

```
sudo docker run --rm alpine:3.20 sh -c \
  'apk add --no-cache -q age && age-keygen' > buro-backup.key
grep 'public key' buro-backup.key
```

Ожидаемый результат: в файле две строки - открытый ключ в комментарии и закрытый ключ. Открытый ключ выводится второй командой.

ПРИМЕЧАНИЕ. Сами копии тоже шифруются без установки `age` на сервер - скрипт запускает его в одноразовом контейнере. Установка средства в систему лишь ускоряет работу и убирает зависимость от доступа к реестру образов.

Файл `buro-backup.key` содержит обе части пары. Строка `# public key: age1...` - открытый ключ, он же выводится на экран при создании; строка `AGE-SECRET-KEY-1...` - закрытый. Открытый шифрует копии и остаётся на сервере в параметре `BACKUP_AGE_RECIPIENT`, закрытый нужен только для восстановления.

ОПАСНО. Закрытый ключ хранится вне сервера. Скопируйте `buro-backup.key` на рабочее место, положите в хранилище секретов организации и удалите файл с сервера. Если он останется рядом с копиями, шифрование теряет смысл: тот, кто получит доступ к серверу, получит и архивы, и ключ к ним. Без закрытого ключа копии не восстановить - утратив его, вы утратите и все копии. Держите не менее двух копий ключа в разных местах, как требует правило хранения секретов из подраздела 5.4.

Листинг 11.3 — Перенос закрытого ключа на рабочее место

```
scp <пользователь>@<адрес сервера>:/opt/systemburo/buro-backup.key .
grep AGE-SECRET-KEY buro-backup.key
```

Обе команды выполняются на рабочем месте, не на сервере. Ожидаемый результат: файл скачан, вторая команда выводит строку закрытого ключа. После этого удалите файл на сервере командой `rm /opt/systemburo/buro-backup.key`.

ВАЖНО. Удаляйте ключ с сервера после того, как проверите восстановимость копии по подразделу 11.5 - проверке он нужен.

Если от шифрования отказываются осознанно. Такой отказ возможен, но требует явного разрешения: параметр `BACKUP_ALLOW_UNENCRYPTED` в значении `yes`, единственном, которое принимается. Тогда копия создаётся незашифрованной, и видно это сразу в трёх местах: в выводе команды копирования, в журнале строкой «копия НЕ шифруется по явному разрешению BACKUP_ALLOW_UNENCRYPTED=yes и содержит персональные данные в открытом виде» и в поле `encrypted` файла состояния, откуда его берёт команда состояния (подраздел 11.5).

Разрешение уместно там, где за сохранность копий отвечает само хранилище: зашифрованный носитель, изолированный сервер копий с ограниченным кругом допущенных. Там, где копии уезжают во внешнее хранилище или лежат на обычном диске рядом с системой, его применять не следует.

Шаг 2. Впишите открытую часть в файл параметров `/opt/systemburo/.env`. Строка `BACKUP_AGE_RECIPIENT=` в файле уже есть,

оставленная пустой сценарием подготовки параметров: значение подставляется в неё, а не дописывается второй строкой в конец файла.

Листинг 11.4 — Запись открытого ключа в параметры

```
cd /opt/systemburo
grep -n '^BACKUP_' .env
```

Ожидаемый результат: перечень строк резервного копирования с номерами; `BACKUP_AGE_RECIPIENT` среди них пуста. Откройте файл любым редактором и подставьте в неё значение, полученное на шаге 1, - строку вида `age1...` целиком.

Листинг 11.5 — Проверка записанного ключа

```
grep -c '^BACKUP_AGE_RECIPIENT=' .env
grep '^BACKUP_AGE_RECIPIENT=' .env
```

Ожидаемый результат: первая команда выводит `1`, вторая - строку с вашим ключом. Если первая вывела `2`, значение дописали второй строкой: лишнюю удалите. Копирование при двух строках отработает, потому что берёт последнюю, но при следующей правке легко исправить не ту, и копии будут шифроваться ключом, который считали заменённым.

ВАЖНО. Без этого параметра копирование не выполняется вовсе - скрипт останавливается до первого обращения к базе, чтобы незашифрованная выгрузка с персональными данными не появилась на диске. Если копии сознательно снимаются открытыми, это разрешается отдельным параметром `BACKUP_ALLOW_UNENCRYPTED=yes`, и тогда каждое копирование пишет об этом предупреждение в журнал.

Шаг 3. Поставьте копирование на расписание.

Листинг 11.6 — Постановка на расписание

```
cd /opt/systemburo
sudo ./scripts/backup-install.sh production
```

Для проверочного стенда контур указывается явно: `sudo ./scripts/backup-install.sh staging`.

Ожидаемый результат: сообщение об установке двух таймеров - ежедневного копирования в 03:30 и ежемесячной проверки восстановимости.

Время задаётся вторым аргументом, например `sudo ./scripts/backup-install.sh production 04:15`.

Доступ оператора к состоянию копирования. Копии закрыты наглухо: в выгрузке базы персональные данные, и открывать их вправе только суперпользователь. Но следить за тем, что копирование идёт, приходится ежедневно (подраздел 9.2), и делать это из-под полных прав необязательно. Поэтому каталогу копий выставляется право прохода для группы, названной параметром `BACKUP_OPERATOR_GROUP`: вместо 700 у него становится 710. Члену этой группы открываются ровно два файла - отчёт о последнем запуске `status.json` и журнал `backup.log`, оба на чтение. Перечислить содержимое каталога, войти в подкаталоги сроков хранения и открыть саму копию он по-прежнему не может: подкаталоги остаются с правами 700, файлы копий - с правами 600 у суперпользователя.

По умолчанию параметр указывает на группу `buro`, которая заводится вместе с учётной записью сопровождения в подразделе 5.1. Сценарий подготовки параметров эту строку в файл не пишет, и её отсутствие означает как раз умолчание. Чтобы от такого доступа отказаться, строку добавляют пустой: `BACKUP_OPERATOR_GROUP=` без значения - осознанный отказ, и тогда каталог остаётся закрытым целиком.

> **ВАЖНО.** Сказанное про закрытые подкаталоги и копии верно для каталога, который система завела сама. Права на подкаталоги и файлы выставляются в момент их создания, и то, что было создано раньше, ими не переставляется. Если копирование работало до того, как доступ оператора включили, проверьте права перед включением: `ls -l /var/backups/systemburo/daily`. Ожидается `drwx------` у подкаталогов и `-rw-------` у файлов копий. Если права шире, откройте каталог группе только после того, как приведёте их к ожидаемым: `chmod 700` на подкаталоги сроков хранения и `chmod 600` на файлы копий. Иначе право прохода в каталог откроет группе оператора и старые копии.

Права выставляет и установщик расписания, и каждое копирование, поэтому заведённая позже группа подхватывается сама, повторно ставить

расписание не нужно. Как настроен доступ, установщик сообщает строкой `оператор:` в конце вывода.

Таблица 11.3 — Строка «оператор» в выводе установщика расписания

Строка	Что означает
`состояние копирования открыто группе <имя>, сами копии ей недоступны`	Группа найдена, право прохода выставлено. Ниже установщик печатает две команды, которыми состояние читается без `sudo`
`группы <имя> на сервере нет, состояние копирования увидит только суперпользователь`	Такой группы на сервере не заведено. Каталог остался закрытым целиком; ошибкой это не считается, копирование работает как прежде
`доступ отключён параметром BACKUP_OPERATOR_GROUP`	Параметр задан пустым, то есть от доступа отказались осознанно

Выравнивание прав на существующей установке. Открытие корня группе оператора (710) само по себе не трогает то, что уже лежит внутри каталога. Пока корень оставался закрытым целиком, режим подкаталогов сроков хранения `daily`, `weekly`, `monthly` и файлов в них значения не имел - дойти до них мог только суперпользователь. Если каталог копий завели раньше, чем в `backup.sh` появилась маска `umask 077`, часть подкаталогов и файлов могла остаться с правами, которые выставлял оператор без этой маски, обычно 755 и 644, - и открытый проход внутрь корня пустил бы группу прямо к содержимому.

Установка сама приводит такой каталог в порядок: подкаталоги закрываются до прав 700, файлы копий в них - до 600. Сколько записей пришлось закрыть, видно в выводе строкой `выровнено`; если закрывать было нечего, строки нет.

Листинг 11.7 — Строка о выравнивании в выводе установщика

```
выровнено:    закрыты права 12 записей в каталоге копий (были шире обещанного до этой установки)
```

Повторный запуск ничего не переставляет - уже закрытые записи выравнивание не трогает. Выполняется оно тем же шагом, что и включение доступа оператора: на сервере, где доступ оператора уже включён, права выровняются при следующем запуске `sudo ./scripts/backup-install.sh`.

Само выравнивание проверяется сценарием `./scripts/backup-install_test.sh`. Он разворачивает каталог копий во временной песочнице, намеренно оставляет часть подкаталогов и файлов открытыми и убеждается, что установка их закрывает, а повторный запуск ничего не меняет. Настоящий сервер сценарий не трогает и прав суперпользователя не требует: обращения к systemd подменяются, временные файлы удаляются по окончании. Запускать его на рабочем сервере не нужно - он нужен при правке самого механизма установки.

Шаг 4. Проверьте, что копия снимается.

Листинг 11.8 — Проверка копирования вручную

```
sudo make deploy-backup
sudo make deploy-backup-status
```

Ожидаемый результат: копирование завершается сообщением об успехе, состояние показывает свежую копию и её объём.

Метка копии. Перед работами, после которых может понадобиться откат, копию удобно пометить: метка попадает в имя файла и избавляет от поиска нужной копии по времени.

Листинг 11.9 — Копия с меткой

```
sudo make deploy-backup ARGS=pered-obnovleniem
```

Ожидаемый результат: файл вида `buro-db-2026-08-01-1612-pered-obnovleniem.dump.age`. В метке допустимы латинские буквы, цифры, дефис и подчёркивание: имя файла попадает в команды восстановления и во внешнее хранилище, где пробелы и кириллица создают трудности.

> **ВАЖНО.** Пока внешнее хранилище не настроено, копии лежат на том же сервере, и отказ его диска уничтожит их вместе с системой. Скрипт предупреждает об этом при каждом запуске. Настройка выгрузки - подраздел 11.6.

11.5. Проверка и наблюдение

Копия, из которой ни разу не восстанавливались, резервной копией не является: повреждение обнаружится тогда, когда она окажется единственной надеждой. Поэтому раз в месяц система сама разворачивает последнюю копию во временную базу рядом с рабочей, сверяет наполнение ключевых таблиц и удаляет временную базу. Рабочая база при этом не затрагивается.

Листинг 11.10 — Проверка восстановимости вручную

```
AGE_IDENTITY=/путь/к/buro-backup.key make deploy-backup-verify
```

Ожидаемый результат: строка «копия пригодна для восстановления» и числа восстановленных учётных записей, заявок и таблиц постов.

ВАЖНО. Путь к закрытому ключу передаётся переменной окружения, а `sudo` окружение очищает. Запись `AGE_IDENTITY=... sudo make deploy-backup-verify` до сценария не дойдёт, а задать переменную после `sudo` разрешение на две команды не позволяет. Проверку зашифрованной копии выполняет с полными правами суперпользователя тот же, кто ставил расписание. Незашифрованной копии ключ не нужен, и для неё достаточно `sudo make deploy-backup-verify`.

Все команды раздела выполняются на сервере системы, в каталоге `/opt/systemburo`, с `sudo`: доступ к Docker выдан через него, подраздел 5.1. Исключение одно - чтение состояния копирования без полных прав, о нём сказано ниже.

Таблица 11.4 — Где смотреть состояние

Что нужно узнать	Где смотреть
Свежесть, объём и состав копий	`sudo make deploy-backup-status`
Что делал скрипт и на чём остановился	файл `/var/backups/systemburo/backup.log`
Итог последнего запуска в машинном виде	файл `/var/backups/systemburo/status.json`
Как отработал запуск по расписанию	`journalctl -u systemburo-backup -n 50`
Когда будут ближайшие запуски	`systemctl list-timers 'systemburo-*'`
Сами файлы копий	каталоги `/var/backups/systemburo/daily`, `weekly`, `monthly`

Пути даны для значения `BACKUP_DIR` по умолчанию. Если параметр изменён, каталог другой, а команда состояния всё равно покажет верный путь первой строкой.

Посмотреть можно и хранилище, к системе не относящееся: копии, принесённые с другого сервера, или примонтированный внешний диск. Каталог передаётся перед командой и на файл параметров не влияет.

Листинг 11.11 — Состояние копий в стороннем каталоге

```
BACKUP_DIR=/mnt/disk/systemburo bash scripts/backup-status.sh production
```

Ожидаемый результат: тот же вывод, но по указанному каталогу; первая строка называет его, чтобы не спутать с рабочим.

Что видно оператору. Ежедневный контроль свежести копии не требует полных прав, и команда состояния работает без `sudo` - показывает она при этом меньше. Каталог копий, настройку внешнего хранилища и шифрования, время и итог последнего запуска член группы оператора видит целиком, вместе с предупреждениями о незашифрованной копии и о копии старше двух суток. Дальше вместо перечня копий печатаются три строки.

Листинг 11.12 — Чем вывод оператора отличается от полного

```
Перечень копий доступен только суперпользователю: сами копии закрыты, оператору видны состояние и журнал.
Полный перечень:  sudo ./scripts/backup-status.sh production
Журнал:           /var/backups/systemburo/backup.log
```

Те же сведения читаются и напрямую, двумя командами - их печатает установщик расписания сразу после установки, подставляя действующий каталог копий.

Листинг 11.13 — Состояние копирования без полных прав

```
cat /var/backups/systemburo/status.json
tail /var/backups/systemburo/backup.log
```

Ожидаемый результат: первая команда выводит отчёт о последнем запуске - время окончания, итог, метку времени копии, объём, признак шифрования и причину сбоя, если он был. Вторая показывает конец журнала: по нему видно, до какого шага дошло последнее копирование.

Разница между командой и этими двумя в том, что команда состояния читает ещё и файл параметров `/opt/systemburo/.env`, закрытый от посторонних правами 600. Учётной записи сопровождения он принадлежит, поэтому команда ей доступна и без `sudo`; отдельной учётной записи, которую лишь включили в группу оператора, остаются две команды выше - они обращаются прямо к файлам и параметров не касаются.

Если каталог копий для учётной записи закрыт - группа не задана, не заведена или запись в неё не входит, - команда состояния говорит об этом прямо, вместо того чтобы промолчать.

Листинг 11.14 — Каталог копий закрыт для этой учётной записи

```
Каталог копий /var/backups/systemburo закрыт для учётной записи buro.
Состояние копирования читает суперпользователь:  sudo ./scripts/backup-status.sh production
Оператору доступ выдаётся группой BACKUP_OPERATOR_GROUP, её выставляет scripts/backup-install.sh.
```

Различать здесь есть что: закрытый каталог и каталог, которого нет вовсе, дают разные сообщения намеренно. Сведения о самом каталоге читаются по правам родительского, поэтому закрытый выглядел бы как «копирование ни разу не выполнялось» - то есть команда врала бы ровно в том случае, ради которого её и открывают оператору.

ВАЖНО. Копия старше двух суток означает, что запуск не состоялся. Команда состояния сообщает об этом строкой «свежей копии нет более двух суток». Разбираться следует сразу: пока копирование не работает, система живёт без страховки.

Про шифрование команда состояния говорит дважды, и это разные утверждения. Строка «Шифрование» в шапке описывает настройку на сейчас, а предупреждение под итогом последнего запуска - то, что фактически лежит на диске: ключ могли задать уже после снятия последней копии, и тогда шапка покажет «включено», а последняя копия останется незашифрованной.

Таблица 11.5 — Строки о шифровании в команде состояния

Строка	Что означает
`Шифрование: включено`	Ключ задан, новые копии шифруются

Продолжение таблицы 11.5

Строка	Что означает
`Шифрование: ВЫКЛЮЧЕНО по явному разрешению, копии содержат персональные данные в открытом виде`	Задано `BACKUP_ALLOW_UNENCRYPTED=yes`. Копии снимаются, но открытыми
`Шифрование: ключ не задан - копирование останавливается с ошибкой, пока не задан BACKUP_AGE_RECIPIENT`	Ни ключа, ни разрешения. Копий не будет вовсе, пока это не исправлено
`ВНИМАНИЕ: последняя копия снята БЕЗ шифрования`	Читается из файла состояния и относится к последней снятой копии, а не к настройке. У копий, снятых прежними версиями сценария, признака нет, и строка не печатается

Кроме свежести команда показывает состав каждой копии: вошли ли в неё загруженные файлы и файловый архив бланков. Копия без них выглядит в перечне так же благополучно, как полная, и разница обнаруживается в момент восстановления, поэтому состав вынесен отдельной колонкой, а сами архивы печатаются строками под своей выгрузкой базы. Подробно перечень разобран в подразделе 11.7.

При недельном режиме архивации состав «только база» у копий за остальные дни ожидаем - файлы снимаются по воскресеньям и первого числа. В ежедневном режиме такая копия означает, что архивация файлов не отработала, и команда пишет об этом отдельной строкой.

Таблица 11.6 — Сообщения о полноте копии

Сообщение команды состояния	Что означает
`Файлы архивируются раз в неделю`	Пояснение к составу `только база`: заданный режим, а не сбой
`ВНИМАНИЕ: режим BACKUP_UPLOADS_MODE=daily, но в последнюю копию файлы не вошли`	Архивация файлов не отработала при ежедневном режиме. Причина - в журнале копирования, порядок разбора ниже

Как выяснить причину, если копия не снялась. Скрипт не падает молча: он записывает причину в журнал строкой со словом ОШИБКА и дублирует её в файл состояния. Отказ выполнять копирование записывается так же, но словом ОТКАЗ: это не сбой, а сознательная остановка до начала работы. Порядок разбора - три шага.

Шаг 1. Посмотрите итог последнего запуска.

Листинг 11.15 — Итог последнего запуска

```
cd /opt/systemburo
sudo make deploy-backup-status
```

Ожидаемый результат при сбое: строка «Итог: СБОЙ» и текст причины.

Шаг 2. Откройте журнал: в нём видно, до какого шага скрипт дошёл.

Листинг 11.16 — Последние записи журнала копирования

```
tail -30 /var/backups/systemburo/backup.log
```

Шаг 3. Найдите причину в таблице ниже. Тексты в ней - те же, что выводит скрипт.

Таблица 11.7 — Причины сбоя копирования

Сообщение в журнале	Что произошло	Что делать
`не задан BACKUP_AGE_RECIPIENT: копия содержала бы персональные данные в открытом виде`	Копирование отказалось выполняться: ключ шифрования копий не задан. Копия не создавалась, старые не тронуты	Завести ключ по шагу 1 подраздела 11.4. Если копии сознательно снимаются без шифрования, задать `BACKUP_ALLOW_UNENCRYPTED=yes`
`не удалось выгрузить базу`	Служба базы данных не отвечает	Проверить состояние служб, подраздел 13.1
`выгрузка базы не читается: повреждена или пуста`	Выгрузка получилась битой, на диске могло кончиться место	Проверить свободное место `df -h`, повторить копирование
`не удалось зашифровать выгрузку базы`	Средство шифрования недоступно: нет `age` и не скачивается образ	Проверить доступ в сеть с сервера либо установить `age` в систему
`не удалось разложить копию по каталогам`	Нет прав на каталог копий либо он на другом разделе	Проверить владельца и права `ls -ld /var/backups/systemburo`
`не удалось выполнить ротацию`	Нет прав на удаление старых копий	Проверить права на подкаталоги `daily`, `weekly`, `monthly`
`не удалось заархивировать загруженные файлы`	Не прочитался том или каталог загруженных файлов	Проверить `sudo docker volume ls` и права на каталог; копия базы при этом снята
`не удалось зашифровать архив файлов`	Средство шифрования недоступно на шаге файлов	Так же, как при сбое шифрования выгрузки базы

Продолжение таблицы 11.7

Сообщение в журнале	Что произошло	Что делать
`не удалось заархивировать файловый архив бланков`	Не прочитался каталог архива бланков	Проверить наличие и права каталога, заданного `ARCHIVE_HOST_PATH`
`не удалось зашифровать файловый архив`	Средство шифрования недоступно на шаге архива бланков	Так же, как при сбое шифрования выгрузки базы
`не удалось выгрузить копии во внешнее хранилище`	Хранилище недоступно или неверны реквизиты	Проверить настройку rclone; копии на сервере при этом сняты
`не задан BACKUP_AGE_RECIPIENT: копия содержала бы персональные данные в открытом виде`	Копирование остановлено до обращения к базе: ключ шифрования не задан, а незашифрованные копии не разрешены	Задать ключ по подразделу 11.4 либо, если копии сознательно снимаются без шифрования, разрешить это параметром `BACKUP_ALLOW_UNENCRYPTED=yes`
`прервано на строке`	Непредусмотренный сбой	Прислать разработчику журнал и вывод `journalctl -u systemburo-backup -n 100`

Отдельно от сбоев скрипт пишет предупреждения - они не прерывают копирование, но означают, что защита неполная.

Таблица 11.8 — Предупреждения в журнале

Сообщение	Что означает
`копия НЕ шифруется по явному разрешению BACKUP_ALLOW_UNENCRYPTED=yes и содержит персональные данные в открытом виде`	Шифрование отключено осознанно: копии лежат открытыми, и доступ к каталогу копий равнозначен доступу к персональным данным
`ни тома, ни каталога загруженных файлов не найдено`	Загруженные файлы в копию не вошли: тома нет или его имя изменено. База при этом скопирована
`внешнее хранилище не настроено`	Копии лежат только на этом сервере и погибнут вместе с его диском
`каталог копий не открыт группе <имя> (проверьте, что такая группа заведена)`	Группа оператора из `BACKUP_OPERATOR_GROUP` на сервере не заведена. Копии снимаются как обычно, но состояние копирования увидит только суперпользователь
`не удалось открыть <файл> на чтение группе <имя>`	Отчёт или журнал не удалось открыть группе оператора, хотя каталог ей открыт. Копирование при этом выполнено; читать состояние до выяснения причины придётся с полными правами

Проверка восстановимости останавливается со своим сообщением - оно печатается в вывод команды.

Таблица 11.9 — Сообщения проверки восстановимости

Сообщение	Что произошло	Что делать
`копий для проверки не найдено`	В суточном каталоге нет ни одной копии	Разобраться, почему не снимается копия, - порядок выше
`копия зашифрована, но AGE_IDENTITY не задан`	Проверке не передан закрытый ключ, которым копия расшифровывается	Передать путь к ключу перед командой проверки, как показано выше
`в копии нет ни одной учётной записи`	Копия развернулась, но пуста: выгрузка снималась с пустой или чужой базы	Проверить имя базы в файле параметров и повторить копирование
`архив не соответствует контрольной сумме`	Содержимое архива отличается от контрольных данных	Проверить другую копию и выяснить, где изменился файл
`количество записей не совпало с контрольными данными`	Счётчики во временной базе расходятся с контрольным файлом	Проверить другую копию и журнал создания этой копии
`контрольный файл новой копии отсутствует`	В комплекте нет обязательного контрольного файла	Проверить, что его создали и перенесли вместе с копией
`неверный формат контрольного файла`	Скрипт не смог прочитать контрольные данные	Проверить целостность комплекта
`не удалось удалить временную базу`	Автоматическая очистка временной базы не удалась	Сверить её имя по журналу и завершить очистку
Сообщение самой базы об ошибке разбора файла	Копия повреждена или дописана не до конца: обрыв при снятии, сбой диска, неполная передача во внешнее хранилище	Проверить предыдущую копию той же командой. Прошла - неисправна только последняя, снять копию заново и следить за журналом. Не прошла ни одна - неисправно копирование, а не отдельный файл: разобрать причину по журналу, как описано выше

ОПАСНО. Копия, не прошедшая проверку, восстановлению не подлежит и до выяснения причины не годится как единственная. Пока причина не найдена, сохраняйте более старые копии: срок хранения удалит их по расписанию, и вместе с ними исчезнет последняя пригодная.

Если журнала нет вовсе. Значит скрипт ни разу не запускался: не установлено расписание либо таймер остановлен. Проверьте его и посмотрите вывод последнего запуска.

Листинг 11.17 — Проверка расписания

```
systemctl list-timers 'systemburo-*'
systemctl status systemburo-backup.timer
journalctl -u systemburo-backup -n 100 --no-pager
```

Ожидаемый результат: таймер в состоянии `active (waiting)`, в списке указано время следующего запуска. Состояние `inactive` означает, что таймер выключен: включается командой `sudo systemctl enable --now systemburo-backup.timer`.

Если backup-status сообщает «копии хранятся дольше, чем данные живут в системе», сравните срок резервных копий и автоматического обезличивания. Успешное создание копии не снимает этого предупреждения: восстановление старой копии может вернуть ранее обезличенные данные. Не сокращайте срок и не удаляйте копии автоматически без решения владельца.

11.6. Хранение вне сервера

Копия на том же диске не защищает от отказа диска. Выгрузка во внешнее S3-совместимое хранилище включается двумя параметрами: `BACKUP_S3_REMOTE` (имя настройки rclone) и `BACKUP_S3_BUCKET` (бакет и префикс). Пока они пусты, выгрузка пропускается, а в журнал пишется предупреждение.

Требования к хранилищу те же, что к самой системе: копии содержат персональные данные, пусть и зашифрованные. Размещение данных выбирается с учётом требований к обработке персональных данных на территории Российской Федерации.

11.7. Восстановление

> **ОПАСНО.** Восстановление заменяет текущее содержимое базы содержимым копии. Отменить это нельзя. Перед началом убедитесь, что выбрана нужная копия: имя файла содержит дату и время снятия.

Восстановление выполняется на сервере системы, в каталоге `/opt/systemburo`. Понадобится закрытый ключ шифрования: принесите

файл `buro-backup.key` из хранилища секретов на время работ и удалите его с сервера после.

Порядок действий выполняет одна команда: она включает режим технических работ, останавливает прикладной сервер, восстанавливает базу и файлы, проверяет наполнение до запуска приложения, поднимает систему и снимает режим.

Имя нужной копии смотрят в каталоге копий: в имени файла стоят дата и время снятия.

Листинг 11.18 — Перечень доступных копий

```
sudo make deploy-backup-status
```

Ожидаемый результат: после сводки по каталогам выводится перечень копий - дата снятия словами, размер, состав, метка и имя файла, которое передаётся команде восстановления. Под строкой выгрузки базы идут её спутники: архив загруженных файлов и архив бланков, снятые в тот же момент. Дата и метка у них те же, поэтому повторно не печатаются.

```
Копии, пригодные для восстановления
Снята                    Размер  Что входит       Метка          Файл
2 августа 2026, 06:00     9.2 МБ  база+файлы+архив -              buro-db-2026-08-02-0600.dump.age
                         41.5 МБ    файлы                        buro-uploads-2026-08-02-0600.tar.gz.age
                          3.4 МБ    архив бланков                buro-archive-2026-08-02-0600.tar.gz.age
1 августа 2026, 17:22     9.2 МБ  только база      pered-testom   buro-db-2026-08-01-1722-pered-testom.dump.age


Файлы архивируются раз в неделю (BACKUP_UPLOADS_MODE=weekly),
поэтому состав «только база» у копий за остальные дни - ожидаемый.
```

Копии перечислены от новых к старым. Одна и та же копия в нескольких каталогах показывается один раз: это один файл, на который стоят жёсткие ссылки.

Колонка «Что входит» отвечает на главный вопрос перед восстановлением - полная ли копия. Значение `база+файлы+архив` означает, что восстановить можно всё; `только база` - что загруженные файлы и бланки придётся брать из копии за другой день. Это не поломка, пока `BACKUP_UPLOADS_MODE` равен `weekly`: в этом режиме файлы архивируются по воскресеньям и первого числа, о чём команда и напоминает строкой под перечнем. В режиме `daily` та же строка заменяется предупреждением: файлы

должны были войти в каждую копию, и если их нет, причина ищется в журнале копирования.

Ниже перечня может появиться ещё один блок - «Архивы файлов без выгрузки базы за тот же срок». В него попадают архивы, пережившие свою копию базы: сроки хранения считаются по каждому виду файлов отдельно, поэтому при недельном режиме архив загруженных файлов остаётся в хранилище дольше, чем суточные выгрузки базы. Восстанавливать такой архив не с чем - его берут только вместе с базой за другую дату, понимая последствия из подраздела «Восстановление копии, снятой давно».

Листинг 11.19 — Восстановление из копии

```
cd /opt/systemburo
install -d -m 700 /var/lib/systemburo/restore-production
RESTORE_SOURCE=current RESTORE_STATE_DIR=/var/lib/systemburo/restore-production \
AGE_IDENTITY=/путь/к/buro-backup.key \
  make deploy-restore ARGS="/var/backups/systemburo/daily/<имя файла из перечня выше>"
```

На проверочном стенде используйте `make staging-restore` и отдельный каталог состояния, например `/var/lib/systemburo/restore-staging`. Каталог должен принадлежать тому, кто запускает восстановление, и иметь права `700`. Программа использует `flock` и `sha256sum` для блокировки и проверки сохранённых данных.

`RESTORE_SOURCE=current` подтверждает, что исходная база с актуальным журналом доступна или журнал уже сохранён в каталоге состояния этого запуска. Если исходная установка и журнал утрачены, не указывайте этот режим и не запускайте приложение из старого дампа. При замене базы сохранённый журнал добавляется к журналу из дампа, а зафиксированные удаления повторяются до запуска служб. Для этого обе базы должны иметь таблицу журнала совместимой схемы.

ВАЖНО. Закрытый ключ на сервере не лежит - его унесли в хранилище секретов при настройке копирования (подраздел 11.4). Значит перед восстановлением его приносят обратно защищённым каналом, указывают путь в `AGE_IDENTITY`, а после восстановления файл с сервера удаляют. То же относится к проверке копий: `make deploy-backup-verify` без ключа не работает. Ключ, оставленный «до следующего раза», обесценивает шифрование копий.

Скрипт выводит заголовок каждого шага и затраченное время. Если передан архив загруженных файлов, скрипт также выводит сообщение об их восстановлении.

ПРИМЕЧАНИЕ. Если в системе установлено средство `pv`, объём восстановления базы дополнительно показывается полосой с процентами. Устанавливать его необязательно, порядок работы от этого не меняется.

Команда запросит подтверждение: нужно ввести слово `ВОССТАНОВИТЬ`. Ожидаемый результат: выводятся числа восстановленных заявок и учётных записей, затем система поднимается и режим технических работ снимается сам.

Вторым аргументом можно передать архив загруженных файлов, если восстанавливать нужно и их. Оба имени берут из перечня выше: архив стоит строкой под своей выгрузкой базы, с подписью «файлы».

Листинг 11.20 — Восстановление вместе с файлами

```
RESTORE_SOURCE=current RESTORE_STATE_DIR=/var/lib/systemburo/restore-production \
  AGE_IDENTITY=/путь/к/buro-backup.key bash scripts/restore.sh production \
  /var/backups/systemburo/daily/<выгрузка базы> \
  /var/backups/systemburo/daily/<архив загруженных файлов>
```

ВАЖНО. База и архив файлов берутся за один и тот же момент - у них совпадает метка времени в имени. Копия базы от одного числа с файлами от другого даёт карточки заявок, ссылающиеся на отсутствующие вложения.

Восстановление из комплекта с независимым журналом

Если исходная база недоступна, используйте отдельный PostgreSQL с журналом уничтожений. Разместите его так, чтобы он оставался доступен при утрате основной установки. Владелец базы журнала применяет `scripts/independent-journal-schema.sql`, создаёт запись установки и назначает разные логины для записи и восстановления. Для backend стенда задайте `DESTRUCTION_JOURNAL_WRITER_DSN`, `DESTRUCTION_JOURNAL_INSTALLATION` и `DESTRUCTION_JOURNAL_KEY`. Храните ключ и доступ для восстановления отдельно от основной установки.

Подготовьте дамп, архив вложений и архив бланков. Рядом с дампом должен лежать файл `checkpoint-<имя дампа>`. Штатное копирование создаёт отметку для полного комплекта при неизменной позиции журнала без незавершённых операций. Подайте `DESTRUCTION_JOURNAL_RECOVERY_DSN` только на время восстановления. Для зашифрованного комплекта также нужен `AGE_IDENTITY`.

Листинг 11.21 — Восстановление стенда при недоступной исходной базе

```
RESTORE_SOURCE=independent RESTORE_STATE_DIR=/var/lib/systemburo/restore-staging \
  bash scripts/restore.sh staging <дамп> <вложения> <бланки>
```

Скрипт сверяет отметку и состав комплекта до замены базы. После блокировки старого писателя он получает подтверждённые операции из отдельного журнала и повторяет уничтожения до запуска служб. Если журнал неполон, результат операции неизвестен или повтор требует ручного действия, разберите причину перед запуском.

После проверки восстановленных данных заведите новый PostgreSQL-логин с членством в `restore_journal_writer`. Владелец журнала вызывает `restore_journal.replace_writer` с идентификатором установки, поколением и номером последней записи после блокировки, а также именем нового логина. При результате `true` обновите DSN писателя в backend. Проверьте отказ прежнему логину и успешную запись новым.

Если восстановление прервалось

Система останется остановленной, а режим технических работ - включённым. Это сделано намеренно: поднимать приложение до выяснения причины опасно, потому что база может быть неполной.

ОПАСНО. После прерывания восстановления база может быть пустой или восстановленной частично. Не запускайте приложение: оно может создать схему и начальные записи поверх неполного состояния. Сохранённая резервная копия от этого не исчезает; устраните причину и повторите восстановление из проверенной копии. Поднимайте службы только после успешного завершения и проверки данных.

Определите шаг по последнему заголовку и ошибке в выводе. При повторе используйте те же дамп, каталог состояния и контур. Сохранённый журнал в `pending` нужен для повторной попытки: не удаляйте и не редактируйте его. После успешного завершения программа очищает это состояние.

Таблица 11.10 — Что делать при прерывании восстановления

Остановилось на шаге	Состояние системы	Порядок действий
Расшифровка копии, ключ не подошёл	Замена базы и остановка служб ещё не начались	Проверить ключ, переданный в `AGE_IDENTITY`, и повторить команду
Восстановление базы данных	**База пуста или восстановлена частично**, службы остановлены	Устранить причину и повторить команду с тем же дампом и каталогом состояния
Восстановление загруженных файлов	База восстановлена, файлы частично	Устранить причину и повторить команду с тем же комплектом
Объединение журналов или повтор удалений	Службы остановлены, исходный журнал сохранён	Разобрать ошибку. До подтверждения журнала и успешного повтора удалений службы не запускать
Запуск системы	База восстановлена, службы не поднялись	Поднять службы вручную, см. ниже
Снятие режима технических работ	Система работает, пользователи видят заглушку	Снять режим вручную, см. ниже

Ручной запуск после прерывания. Используйте эти команды, только если восстановление базы и файлов завершилось, журналы объединены и повтор удалений завершился успешно, а сбой возник на запуске служб или снятии режима обслуживания. Обе команды выполняются в каталоге `/opt/systemburo`. Для проверочного стенда в первой команде вместо `docker-compose.prod.yml` указывается `docker-compose.staging.yml`, во второй вместо `production` - `staging`.

Листинг 11.22 — Запуск служб и снятие режима вручную

```
sudo docker compose -f docker-compose.base.yml -f docker-compose.prod.yml \
  start backend frontend nginx
bash scripts/maintenance-off.sh production
```

Ожидаемый результат: три службы запущены, страница входа открывается без заглушки о технических работах.

> **ВАЖНО.** Здесь используется `start`, поскольку после восстановления достаточно запустить уже созданные контейнеры без смены образа и конфигурации. Команда `up` может пересоздавать контейнеры и, в зависимости от политики загрузки, обращаться к реестру. При восстановлении не совмещайте возврат данных с обновлением версии.

Если службы не запускаются, причина не в восстановлении: порядок диагностики - в подразделе 13.1. Состояние базы в любой момент проверяется теми же счётчиками, что выводит проверка восстановимости из подраздела 11.5.

> **ВАЖНО.** После восстановления проверьте вход в систему и открытие любой заявки. Если числа в выводе отличаются от ожидаемых, повторите восстановление из другой копии - предыдущие лежат в том же каталоге.

Возврат файлового архива бланков

Команда восстановления работает с базой и загруженными файлами. Файловый архив бланков в копию входит отдельным файлом `buro-archive-<дата>.tar.gz` и разворачивается вручную - каталог архива лежит на диске сервера, а не в томе, и подменять его заодно с базой команда не берётся. Имя этого файла берут из перечня копий: он стоит под своей выгрузкой базы, с подписью «архив бланков».

Обычно этого и не требуется: перечень выложенного придёт из копии базы, а ночная сверка допишет недостающие файлы, см. подраздел 9.6. Разворачивать архив из копии нужно, когда каталог утрачен вместе с диском или из него пропали замороженные бланки - их система заново не формирует.

Листинг 11.23 — Возврат каталога архива из копии

```
age -d -i /путь/к/buro-backup.key \
  -o /tmp/buro-archive.tar.gz \
  /var/backups/systemburo/daily/buro-archive-2026-08-02-0600.tar.gz.age
sudo tar xzf /tmp/buro-archive.tar.gz -C /srv/buro-archive
sudo chown -R 1001:1001 /srv/buro-archive
rm /tmp/buro-archive.tar.gz
```

Ожидаемый результат: в каталоге архива снова лежит дерево заявок, владелец совпадает с пользователем прикладного сервера. Если копии не шифруются, первая команда пропускается, а имя файла оканчивается на `.tar.gz`. Каталог назначения берётся из `ARCHIVE_HOST_PATH`, в примере это `/srv/buro-archive`.

ВАЖНО. Распаковка добавляет файлы к тем, что уже лежат в каталоге, и переписывает совпадающие по имени. Если архив разворачивается на замену прежнему содержимому, каталог сначала очищают: иначе рядом останутся файлы заявок, которых в восстановленной базе нет.

Восстановление копии, снятой давно

Копия годичной давности на сегодняшнюю версию системы разворачивается той же командой: порядок действий не меняется. Отличаются последствия, и их надо знать заранее.

Схема базы приводится к текущей версии сама, при первом запуске системы после восстановления: недостающие таблицы, столбцы и индексы создаются. Обратного хода нет - схема правится только вперёд, поэтому вернуть базу к виду годичной давности после запуска новой версии уже нельзя.

ОПАСНО. Перед восстановлением старой копии снимите копию текущего состояния командой `make deploy-backup ARGS=pered-vosstanovleniem`. Без неё отменить восстановление будет нечем: данные, накопленные после снятия старой копии, исчезнут безвозвратно.

Таблица 11.11 — Что изменится при восстановлении давней копии

Что	Как поведёт себя
Учётные записи	Вернутся к состоянию копии. Работники, заведённые позже, исчезнут вместе со своими заявками и правами; пароли, изменённые позже, вернутся к прежним

Продолжение таблицы 11.11

Что	Как поведёт себя
Журнал запросов и журнал обращений к персональным данным	Из копии не попадут в интерфейс. Прежние таблицы сохраняются под именами `request_logs_legacy` и `pd_audit_logs_legacy`, а рабочие создаются заново пустыми: с тех пор они разделены по датам, и старый вид базе не подходит. Данные не теряются, разбираются запросом к базе
Персональные данные	Для зашифрованных сведений документов нужен ключ, действовавший на момент копии. При несовпадении ключа запуск отклоняется; прежний ключ возвращают из хранилища секретов вместе с восстановлением копии
Загруженные файлы	Должны восстанавливаться из архива той же даты, что и база. База от одного числа с файлами от другого даёт карточки, ссылающиеся на отсутствующие вложения
Файловый архив бланков	Предпочтительно восстановить его из копии того же момента. Если оставлен нынешний архив, его состояние может расходиться с восстановленной базой. Сохраните отдельную копию каталога и согласуйте сверку; не очищайте архив автоматически
Заявки в работе	Вернутся к состоянию на момент копии: согласования, сделанные позже, отменятся, а завершённые заявки снова окажутся в обработке

Порядок безопаснее проверить сначала на стенде: `make staging-restore` разворачивает ту же копию на проверочном контуре, где ошибка ничего не стоит. Перед такой проверкой сверьте версии системы и необходимые ключи: стенд и рабочий сервер могут отличаться. Не переносите рабочие персональные данные на обычный учебный стенд; используйте согласованный изолированный контур восстановления.

11.8. Перенос на другой сервер

Копии подходят и для переноса: выгрузка базы и архивы файлов не зависят от сервера, на котором сняты. Развёрнутый порядок переноса с проверками описан в разделе 14.

12. Мониторинг и журналы

12.1. Проверка состояния

Адрес `/health` возвращает признак работоспособности прикладного сервера. Проверка подключается к средствам мониторинга организации.

Листинг 12.1 — Проверка состояния

```
curl -s http://localhost/health
```

Ожидаемый результат: `{"status":"ok"}`.

ВАЖНО. Проверка подтверждает, что процесс запущен и отвечает, но к базе данных она не обращается. При недоступной базе проверка продолжит отвечать успешно, а система работать не будет. Полноценным контролем является периодическая проверка входа в систему, она описана в подразделе 12.2.

Контейнер прикладного сервера имеет встроенную проверку работоспособности с интервалом 30 секунд. Контейнер базы данных проверяется каждые 10 секунд, и прикладной сервер не запускается, пока база не сообщит о готовности.

12.2. Проверка живости с оповещением на почту

В поставку входит сценарий, который проверяет систему так, как её видит человек, и при отказе пишет письмо. Он закрывает разрыв, оставленный адресом `/health`: тот отвечает «в порядке» на любом запущенном процессе, не заглядывая ни в базу, ни во вход, поэтому система с отвалившейся базой отчитывается бодро, не пуская при этом ни одного работника.

Что проверяется. Три вещи, первые две - снаружи, тем же путём, что и у человека.

Таблица 12.1 — Состав проверки живости

Проба	Как выполняется	Что ловит
Сайт	Запрос к внешнему адресу системы	Упавший прокси, просроченный сертификат, пустую страницу при несобранном интерфейсе
Вход	Настоящий вход отдельной учётной записью с получением маркера доступа и последующим выходом	Отказ вида «страница открывается, а войти нельзя»
База	Запрос `SELECT 1` к базе данных	Базу, которая соединение принимает, но на запрос не отвечает

Почему вход проверяется настоящим входом. Напрашивается более простая проба: послать заведомо неверный пароль и убедиться, что система ответила отказом. Она доказывает мало: отказ система обязана возвращать

одинаковым и на несуществующее имя, и на неверный пароль, иначе по ответу перебирают имена учётных записей. Такая проба говорит лишь, что система на запрос отвечает, - а «сайт открывается, а войти нельзя» и есть тот отказ, ради которого проверка заводится. Успешный вход подделать нечем: он требует прочитать запись работника, сверить пароль и выдать пару маркеров, то есть живую базу и работающую выдачу маркеров разом.

Отсюда требование к учётной записи. Для проверки заводят отдельную запись обычного работника, без прав администратора и без заявок: её единственное дело - доказывать, что вход работает. Пароль к ней лежит в файле параметров в открытом виде, поэтому давать ей что-либо сверх входа нельзя. Если в организации принята плановая смена паролей, менять его нужно одновременно в системе и в файле параметров, иначе проверка объявит отказ на исправной системе. Проверка сама за собой убирает: после входа она выходит, чтобы не накапливать записи о сеансах.

Настройка. Параметры вписываются в файл `.env` руками, сценарий подготовки параметров их не создаёт. Образец с пояснениями к каждой строке лежит в `.env.example`, перечень - в приложении Б.

Листинг 12.2 — Наименьший рабочий набор параметров

```
HEALTH_CHECK_URL=https://buro.example.ru
HEALTH_CHECK_USERNAME=health
HEALTH_CHECK_PASSWORD=<пароль этой учётной записи>
HEALTH_ALERT_TO=dezhurny@example.ru
```

Отправка письма берёт адрес почтового сервера и учётные данные к нему из параметров почты того же файла, начинающихся с `SMTP_`. Пока не задан либо адрес почтового сервера, либо `HEALTH_ALERT_TO`, проверка работает и пишет итог в свой журнал, но об отказе никто не узнает, пока туда не заглянут; о каждом таком случае в журнал уходит отдельная строка.

Постановка на расписание. Выполняется один раз, отдельным сценарием. Он создаёт службу и таймер, включает их и печатает, что получилось: адрес, учётную запись, адресатов писем и каталог состояния. Чего не хватает - о том скажет отдельным предупреждением.

Листинг 12.3 — Постановка проверки на расписание

```
sudo ./scripts/health-install.sh production 5
```

Ожидаемый результат: строка «Расписание установлено» и сводка настроек. Второй аргумент - промежуток в минутах, допустимы значения от 1 до 60, по умолчанию 5. Повторный запуск сценария обновляет расписание, поэтому менять промежуток можно тем же вызовом.

Листинг 12.4 — Проверить сейчас, не дожидаясь расписания

```
sudo make deploy-health-check
```

Ожидаемый результат: строка с итогом и ответом каждой из трёх проб. Код возврата 0 означает, что система в порядке (либо объявлены технические работы), 1 - отказ системы, 2 - сама проверка выполниться не смогла.

Что приходит в письме. Письмо об отказе называет адрес системы, стенд, имя сервера, время и то, сколько отказ уже длится; дальше идут ответы всех трёх проб и предположение о причине, сведённое из них. «Страница открывается, а база не отвечает» и «сайт открывается, но войти нельзя» - разные аварии и разное начало разбора, поэтому проба базы выполняется отдельно от входа. Заканчивается письмо командами, которые смотрят на сервере.

Письмо уходит на смене состояния - когда система упала и когда поднялась, - а пока отказ длится, повторяется напоминанием не чаще `HEALTH_ALERT_REPEAT_MIN`. Письмо о восстановлении приходит обязательно: без него нельзя отличить кончившуюся аварию от того, что письма просто перестали доходить.

Отдельным письмом сообщается, что сломалась сама проверка: пропал `curl`, каталог состояния закрыт на запись, не задан адрес системы. Ослепшая проверка ничем не лучше отказавшей системы, поэтому такой запуск не выглядит успешным, а получает своё состояние и своё письмо.

Объявленный в системе режим технических работ тревогу снимает: проверка видит его по ответу системы, пропускает пробу входа и молчит.

ВАЖНО. Проверка идёт с того же сервера, где стоит система. Отказ самого сервера - выключенное питание, отказ диска, потеря сети - она не покажет: письмо просто не придёт, и тишина будет неотличима от исправной работы. Тем же ограничением объясняется, почему не приходит письмо при отказе почтового сервера. Наблюдение снаружи, с другой машины, эта проверка не заменяет; если оно требуется, его строят средствами организации поверх адреса `/health` или внешним сервисом.

Таблица 12.2 — Где смотреть состояние проверки

Что нужно узнать	Где смотреть
Итог каждого запуска	файл `/var/lib/systemburo/health/health.log`
Текущее состояние и время последнего письма	файл `/var/lib/systemburo/health/state.json`
Как отработал запуск по расписанию	`journalctl -u systemburo-health -n 50`
Когда будут ближайшие запуски	`systemctl list-timers 'systemburo-*'`
Снять проверку с расписания	`systemctl disable --now systemburo-health.timer`

Пути даны для значения `HEALTH_STATE_DIR` по умолчанию. Каталог закрыт правами 700 и читается суперпользователем: в журнале внешний адрес системы и имя проверяющей учётной записи, то есть половина пары для входа.

ПРИМЕЧАНИЕ. На проверочном стенде, закрытом базовой авторизацией прокси, проверка после входа не выходит - базовая авторизация занимает тот же заголовок запроса, что и маркер доступа. Записи о сеансах там будут накапливаться, и об этом сказано строкой в журнале. На рабочем сервере, который базовой авторизацией не закрыт, выход выполняется.

12.3. Журналы приложения

Для сопоставления записей прокси и приложения используйте идентификатор запроса. В прикладном и PD-аудите учитывается фактический итоговый HTTP-статус, в том числе отказ допуска и отмена до отправки ответа. Клиенту выдаётся безопасное сообщение; диагностические записи не должны включать тело запроса, SQL с данными, заголовки авторизации или секретные значения. Параметры адреса маскируются действующим закрытым перечнем разрешённых значений. Ответы API не

должны попадать в общий кэш; это правило не распространяют на неизменные хешированные JS/CSS.

Прикладной сервер пишет журнал в поток вывода и, если задан параметр `LOG_FILE_PATH`, дополнительно в файл. На рабочем сервере файловый журнал включён.

Смена файлов выполняется самим приложением, настраивать системное средство ротации не требуется: файл сменяется при достижении 100 МБ, хранится 30 суток, сохраняется до 14 файлов, старые сжимаются.

Листинг 12.5 — Просмотр журналов

```
# все службы, в реальном времени
sudo make deploy-logs

# только прикладной сервер, последние 200 строк
sudo docker compose -f docker-compose.base.yml -f docker-compose.prod.yml \
  logs --tail=200 backend

# только ошибки
sudo docker compose -f docker-compose.base.yml -f docker-compose.prod.yml \
  logs backend | grep -i error
```

ВАЖНО. Журналы контейнеров по умолчанию не ограничены по размеру и способны заполнить диск. Настройте ограничение при вводе в эксплуатацию.

Листинг 12.6 — Ограничение размера журналов контейнеров

```
# /etc/docker/daemon.json
{
  "log-driver": "json-file",
  "log-opts": { "max-size": "50m", "max-file": "5" }
}
```

После изменения файла выполните перезапуск службы Docker и пересоздание контейнеров.

Листинг 12.7 — Применение ограничения журналов

```
sudo systemctl restart docker
sudo make deploy-up
```

Ожидаемый результат: для каждого контейнера хранится не более пяти файлов журнала по 50 МБ.

Листинг 12.8 — Проверка того, что ограничение действует

```
sudo docker info --format 'драйвер: {{.LoggingDriver}}'
sudo docker inspect -f '{{.HostConfig.LogConfig}}' systemburo-backend-1
```

Ожидаемый результат: первая команда называет драйвер `json-file`. Во второй проверьте параметры `max-size` и `max-file` у заново созданного контейнера и сопоставьте их с настройками службы Docker и Compose. Пустой перечень `map[]` сам по себе не подтверждает включение ротации. При расхождении проверьте файл `daemon.json`, перезапуск службы и пересоздание нужного контейнера.

ПРИМЕЧАНИЕ. Ограничение применяется к контейнерам, созданным после перезапуска службы. Пересоздание в листинге выше нужно именно для этого: перезапуск самой службы Docker уже созданным контейнерам ограничение не переставляет.

12.4. Журналы внутри системы

Часть журналов доступна прямо в интерфейсе и не требует доступа к серверу. Их могут просматривать как специалисты ИТ, так и операторы бюро с соответствующим правом.

Таблица 12.3 — Журналы, доступные в интерфейсе

Журнал	Содержание	Где находится
Журнал запросов	Все обращения к системе: работник, действие, код ответа, длительность, время	Раздел администрирования, группа «Аудит и связь», открывается по отдельному праву
Журнал обращений к персональным данным	Обращения к сведениям о людях: работник, действие, раздел, адрес, время	Раздел администрирования, открывается по отдельному праву
Журнал изменений	Кто, когда и что изменил в сущностях системы	Раздел администрирования
Журнал отказов в доступе	Попытки выполнить действие без права	Раздел администрирования
Журнал входов	Входы, выходы, неудачные попытки, блокировки и их снятие администратором	Карточка работника

Журнал запросов и журнал обращений к персональным данным поддерживают фильтрацию и выгрузку в электронную таблицу (`.xlsx`).

Выгрузка журнала запросов повторяет отбор и порядок, показанные на экране, и отдаёт не более 10 000 строк за один раз. Если под отбор попало больше, файл говорит об этом подписью под заголовком, а экран предлагает сузить период или отбор: молча обрезанный файл читался бы как весь период.

Значения параметров адреса, кроме служебных, в журнале затёрты и в выгрузку не попадают - подробнее в техническом описании системы, подраздел 13.9. Сама выгрузка записывается в журнал изменений: кто, когда, каким отбором и сколько строк унёс.

Подробные записи журнала запросов хранятся 30 суток, затем сворачиваются в суточные итоги. Записи журнала обращений к персональным данным хранятся 36 месяцев без свёртки и вместе с журналом запросов не удаляются.

12.5. Что контролировать

В списке уведомлений и ответе на отметку прочтения выдаётся актуальное представление для читателя; новые записи и внешний push не хранят копию персонального текста. При отсутствии доверенной связи с событием, в том числе у старых и импортированных записей, выдаётся «Есть обновление» без исторических подробностей. Проверка доступа, согласия и масок выполняется при чтении; сохранённый старый текст не используется как запасной вариант. Это не массовая очистка старых строк базы. Состояние уже доставленного OS-push находится вне контроля сервера.

Внутренние уведомления о подаче и дополнении формируются из сохранённых событий, независимо от внешней push-доставки. Неподтверждённые события повторяются с паузой до пяти минут и не удаляются только из-за возраста. Исполнитель работает раз в две секунды и по сигналу, до 25 событий за проход; состояние сохраняется в notification_intents. Потеря внешнего push не означает потерю внутреннего уведомления. Очередь push хранится в памяти, ограничена 1024 заданиями;

задания старше 10 минут отбрасываются, а переполнение и остановка учитываются диагностическими счётчиками. Перезапуск не восстанавливает эту очередь. Проверять её следует отдельно от устойчивых очередей почты и событий уведомлений.

Таблица 12.4 — Показатели для наблюдения

Показатель	Норма	Признак неблагополучия
Состояние служб	Все `Up`, база `healthy`	Перезапуски, состояние `Restarting`
Свободное место на диске	Более 20 процентов	Менее 10 процентов требует немедленных действий
Сообщения уровня `error` в журнале	Единичные	Повторяющиеся однотипные ошибки
Отказы в доступе	Единичные	Массовые отказы указывают на неверно настроенные права
Срок действия сертификата	Более 30 суток	Менее 30 суток требует продления
Число подключений к базе	Значительно меньше 150	Приближение к пределу
Размер базы и крупнейшие таблицы	Измеренный рост при обычной нагрузке и действующих сроках хранения	Необъяснимое ускорение роста, отдельная разрастающаяся таблица или риск исчерпания места
Состояние файлового архива	Ошибок записи нет, очередь разбирается	Растущее число ошибок либо остановка очереди по нехватке места
Возраст последней резервной копии	Не старше суток	Двое суток и более означают, что копирование сломано
Проверка живости	Состояние `ok`, письма не приходят	Письмо об отказе; состояние `broken` означает, что сломалась сама проверка, подраздел 12.2

Размер базы в разрезе таблиц показывает команда `make deploy-storage`. Она только читает и ничего не изменяет, поэтому её можно выполнять в любое время. Кроме размеров она отвечает, сколько из занятого объёма подлежит очистке по срокам хранения, то есть сразу подсказывает, есть ли смысл браться за подраздел 9.9 «Очистка накопленных данных».

Листинг 12.9 — Обзор занятого места

```
sudo make deploy-storage
```

Ожидаемый результат: размер базы, перечень крупнейших таблиц с числом записей и размером, затем сводка по группам очистки. Число таблиц в перечне задаётся ключом `-top`, например `make deploy-storage ARGS="-top=30"`.

ПРИМЕЧАНИЕ. Разделы журнальных таблиц в перечне не показываются отдельными строками, их объём учтён в родительской таблице. Число записей берётся из внутренней статистики базы и может незначительно расходиться с точным, пока не отработала автоматическая очистка.

13. Диагностика и типовые неисправности

13.1. Порядок диагностики

Таблица 13.1 — Команды диагностики

Что выяснить	Команда
Состояние служб	`sudo docker compose -f docker-compose.base.yml -f docker-compose.prod.yml ps`
Причина незапуска службы	`sudo docker compose -f docker-compose.base.yml -f docker-compose.prod.yml logs <служба>`
Потребление ресурсов	`sudo docker stats`
Свободное место	`df -h` и `sudo docker system df`
Доступность базы	`sudo docker compose -f docker-compose.base.yml -f docker-compose.prod.yml exec db pg_isready -U postgres`
Действующие параметры прикладного сервера	`sudo docker compose -f docker-compose.base.yml -f docker-compose.prod.yml exec backend env`
Проверка сертификата	`openssl x509 -in nginx/certs/fullchain.pem -noout -dates`
Что видит проверка живости	`sudo make deploy-health-check`

Если разбор начался с письма об отказе, порядок проб уже проделан за вас: письмо называет, какая из трёх ответила отказом, и с чего начинать, подраздел 12.2.

13.2. Система недоступна, страница не открывается

Таблица 13.2 — Диагностика недоступности

Шаг	Действие	Вывод
1	Проверить состояние служб	Службы не запущены, перейти к шагу 2, иначе к шагу 3
2	`sudo make deploy-up`, затем `sudo make deploy-logs`	Причина незапуска будет в журнале

Продолжение таблицы 13.2

Шаг	Действие	Вывод
3	`curl -s http://localhost/health`	Ответ есть, значит неисправность в сети или правилах межсетевого экрана. Ответа нет, значит в прикладном сервере
4	`df -h`	Заполнение диска приводит к отказу базы данных

13.3. Обратный прокси не запускается

Наиболее вероятная причина: отсутствует файл сертификата. Прокси с настроенным защищённым соединением не стартует, если сертификата нет.

Листинг 13.1 — Проверка наличия сертификата

```
ls -l nginx/certs/fullchain.pem nginx/certs/privkey.pem
```

Ожидаемый результат: оба файла существуют. При их отсутствии выпустите сертификат по подразделу 7.3.

13.4. Предупреждение о недоверенном сертификате

Признак: браузер сообщает о недоверенном сертификате и не предлагает продолжить.

Отсутствие предложения продолжить нормально и вызвано требованием обязательного защищённого соединения. Причина в том, что корневой сертификат не установлен на рабочем месте, устранение по подразделу 7.4.

Листинг 13.2 — Проверка, какой сертификат отдаёт сервер

```
openssl s_client -connect <адрес>:443 -servername <адрес> \
  </dev/null 2>/dev/null \
  | openssl x509 -noout -subject -ext subjectAltName
```

Ожидаемый результат: в поле альтернативных имён присутствует адрес, по которому обращаются работники. Если обращение идёт по сетевому адресу, а сертификат выпущен только на имя, требуется перевыпуск с указанием адреса вторым аргументом.

13.5. Прикладной сервер не запускается

Таблица 13.3 — Сообщения об ошибках при запуске

Сообщение в журнале	Причина	Устранение
Ошибка обязательного параметра	Не задан или недопустим один из трёх обязательных параметров	Проверить файл параметров, подраздел 6.2
Ошибка подключения к базе	База не готова либо неверны учётные данные	Проверить доступность базы, подраздел 13.1
Ошибка приведения схемы	Несовместимое изменение схемы	Восстановить базу из копии, сделанной до обновления
Требуется ключ шифрования	Включено требование шифрования, ключ отсутствует	Восстановить или задать корректный ключ. Не отключать требование шифрования ради запуска
Ключ шифрования не подходит к данным этой базы	Ключ не тот, которым зашифрованы сведения документов: подставлен чужой, создан заново вместо утраченного либо убран с зашифрованной базы	Вернуть прежний ключ из хранилища либо перевести данные на новый, подраздел 9.12. Данные при отказе не изменены

13.6. Изменение параметра не подействовало

Наиболее частая причина: параметр не входит в набор, передаваемый через файл параметров, см. подраздел 6.1.

Листинг 13.3 — Проверка действующих параметров

```
sudo docker compose -f docker-compose.base.yml -f docker-compose.prod.yml \
  exec backend env | grep RESET_TIMEZONE
```

Ожидаемый результат: выведено заданное значение. Если параметр отсутствует в выводе, добавьте его в раздел `environment` файла описания рабочего сервера.

13.7. Заканчивается место на диске

Таблица 13.4 — Освобождение дискового пространства

Потребитель	Как проверить	Как освободить
Журналы контейнеров	`sudo docker system df`	Настроить ограничение, подраздел 12.3

Продолжение таблицы 13.4

Потребитель	Как проверить	Как освободить
Неиспользуемые образы	`sudo docker system df`	`sudo docker system prune -a --filter "until=168h"` (при выпуске новой версии убираются сами, см. ниже)
Кеш сборки образов	`sudo docker system df`, строка `Build Cache`	`sudo docker builder prune -a`
Журнал запросов в базе	Размер таблиц журнала	Уменьшить `REQUEST_LOG_DETAIL_DAYS`
История, слепки и суточные итоги	`sudo make deploy-storage`	Команда очистки, подраздел 9.9
Резервные копии	`sudo make deploy-backup-status`	Уменьшить сроки хранения, подраздел 11.3
Загруженные файлы	`sudo docker system df -v`	Перенести том на отдельный раздел
Файловый архив бланков	Раздел администрирования «Файловый архив»	Перенести каталог на отдельный раздел либо задать архиву предельный объём, подраздел 9.6

ВАЖНО. Не уменьшайте срок хранения журнала обращений к персональным данным без правовых оснований. Он определяется требованиями к обработке персональных данных.

ВАЖНО. На диске небольшого объёма первым по величине оказывается не журнал и не образы, а кеш сборки. Уборка образов его почти не задевает: она отбирает по возрасту, а кеш свежий - он остался от последней сборки. На сервере с диском 10 ГБ команда уборки образов освободила 0.4 ГБ, тогда как в строке `Build Cache` лежало 3.5 ГБ, и забрала их отдельная команда `docker builder prune -a`. Смотреть нужно строку `Build Cache` в `docker system df`, а не только `Images`. Кеш нужен лишь для ускорения следующей сборки, поэтому его удаление ничего не ломает: следующая сборка просто пройдёт дольше.

ПРИМЕЧАНИЕ. После успешного выпуска новой версии сервер убирает за собой сам - удаляются висячие слои, кеш сборки старше недели и образы, на которые не ссылается ни один контейнер, старше суток. Тома не затрагиваются ни одной из этих команд: база и загруженные файлы остаются на месте. Ручная уборка из таблицы нужна, если выпуски давно не выполнялись или место кончилось между ними.

13.8. Система работает медленно

Проверьте, не достигнут ли общий бюджет операций или отдельная очередь вычисления пароля. HTTP 503 с Retry-After: 2 означает временную

занятость; число отказов сопоставляйте с процессором, памятью, пулом БД и длительностью операций. В nginx начальный общий профиль API составляет 256 одновременных запросов и 200 запросов в секунду с burst 400. Это общие пределы установки, а не гарантированная производительность и не лимит одного сотрудника. До чтения тела в Go действует общий предел восьми API-запросов с телом без очереди, независимо от метода, Content-Length, chunked или HTTP/2. Запросы без тела не занимают эти слоты; их параллельность, SSE и скачивание не сводятся к восьми соединениям. Общий предел тела — 50 МиБ, меньшие пределы маршрутов сохраняются. Занятость возвращает 503 с Retry-After: 2, превышение размера — 413. Освобождение слота включает завершение чтения и уборку multipart-файлов. В API-location nginx установлены proxy_request_buffering off и proxy_http_version 1.1: тело передаётся приложению, не накапливаясь целиком в прокси. Этот допуск отделён от бюджета выполнения тяжёлых операций. Правила входа по сетевому адресу учитывайте отдельно: за одним внешним адресом может находиться весь офис. Изменение профиля требует измерений на целевой виртуальной машине и оценки её канала; эти пределы не доказывают устойчивость к любой DDoS-нагрузке.

Таблица 13.5 — Диагностика замедления

Проверка	**Толкование**
`sudo docker stats`	Постоянная загрузка процессора базой близко к пределу означает нехватку ресурсов либо тяжёлые отчёты
Журнал запросов, сортировка по длительности	Выявляет конкретные медленные операции
Число подключений к базе	Приближение к 150 указывает на исчерпание предела
Свободная память	Нехватка приводит к вытеснению кэша базы и резкому замедлению

13.9. Работник не видит раздел или получает отказ

1. Открыть журнал отказов в доступе: там зафиксировано, какое именно право не было предоставлено.
2. Проверить назначенные работнику роль и группы прав.

3. Учесть задержку до 30 секунд после изменения прав.
4. Убедиться, что учётная запись не заблокирована.

Отдельный случай - работник не может войти вовсе и видит обратный отсчёт. Это временная блокировка входа после пяти неудачных попыток пароля; в карточке работника она помечена, и там же снимается кнопкой. Журнал входов показывает, сколько попыток и с каких адресов было сделано.

13.10. Не приходят обновления без перезагрузки страницы

Убедитесь, что прокси пропускает SSE, клиент получает новый одноразовый билет, а лимиты владельца сеанса и процесса не исчерпаны. Допускается до 8 открытых потоков и 16 билетов на владельца, до 200 потоков и 400 билетов на процесс; работа от чужого имени не создаёт владельцу новый независимый бюджет. Билет живёт 60 секунд, служебный сигнал приходит раз в 25 секунд, соединение обновляется примерно через 10 минут. Зависшая запись закрывается по пятисекундному пределу. После разрыва браузер повторяет подключение с паузой и разбросом, обновляет данные обычными запросами и в предусмотренных случаях переходит к опросу. Отказ авторизации не устраняется опросом или увеличением лимита.

Признак: данные на открытой странице обновляются только после ручного обновления.

Проверить, что обратный прокси не буферизует поток событий. В поставляемой конфигурации буферизация отключена и увеличен предельный срок ожидания для трёх адресов: потока событий и двух адресов выгрузки из файлового архива. Если конфигурация изменялась, восстановите эти параметры.

Принудительное переподключение каждые 10 минут является штатным поведением.

13.11. Выгрузка из файлового архива не начинается или обрывается

Признак: после нажатия кнопки скачивания загрузка долго не начинается, обрывается на середине либо сервер отвечает ошибкой.

Таблица 13.6 — Диагностика выгрузки из архива

Проверка	Толкование
Сообщение о превышении предела	Объём выгрузки больше заданного предела, по умолчанию два гигабайта. Сузить период либо поднять предел на сервере: `sudo make deploy-archive ARGS="set -zip-max 4G"`
Отказ в доступе	Скачивание исторических оригиналов разрешено только действующему незаблокированному суперадминистратору и при выполнении проверок согласия и выдачи из подраздела 9.6
Настройки прокси для адресов выгрузки	При включённой буферизации прокси накапливает весь архив, прежде чем отдать первый байт: выгрузка за месяц упирается в память сервера. Буферизация должна быть отключена, предельный срок ожидания увеличен
Свободная память на сервере	`sudo docker stats` во время выгрузки: рост потребления памяти прокси означает, что буферизация всё же включена

ПРИМЕЧАНИЕ. Ссылка на скачивание действует минуту и гасится при первом использовании. Повторное открытие сохранённой ссылки заканчивается отказом - это штатное поведение, выгрузку запускают заново из интерфейса.

13.12. Бланки не появляются в каталоге архива

Признак: заявки подаются, а в каталоге пусто или новых файлов нет. Проверки идут сверху вниз, от общего выключателя к отдельному вложению.

Таблица 13.7 — Диагностика записи в архив

Проверка	Толкование
`sudo make deploy-archive ARGS="show"`, строка «Выгрузка бланков»	Значение «выключена» - записи не будет ни по одной заявке. Включается `sudo make deploy-archive ARGS="on"`
Тумблер автосохранения в карточке типа вложения	Выгружаются только те типы, у которых он включён. Тумблер недоступен, пока у типа нет действующего Excel-бланка
Состояние вложения в карточке заявки	«Нет шаблона» - приложить бланк типу вложения; «Нет места» - освободить место на разделе; «Ошибка» - причина видна в разделе администрирования, вкладка «Лента» с отбором по состоянию
Владелец каталога `ARCHIVE_HOST_PATH`	Процесс не меняет владельца и на чужом каталоге останавливается. Владелец должен совпадать с пользователем прикладного сервера, см. подраздел 9.6

Продолжение таблицы 13.7

Проверка	Толкование
Причина у строки в ленте раздела	Сообщение о запрете доступа при создании каталога (`permission denied`) означает именно чужого владельца каталога архива, а не нехватку места или отсутствие бланка
Дата заявки	Записываются заявки, изменённые после включения архива. Ранее поданные выкладывает только выгрузка задним числом за нужный период
Журнал прикладного сервера	`sudo make deploy-logs`: сообщения об остановке записи по месту и об ошибках файловой системы пишутся туда

ПРИМЕЧАНИЕ. Файл появляется не в момент нажатия кнопки, а через несколько секунд: запись идёт фоновой очередью. Если файл не появился и через минуту, проверки из таблицы уместны.

ВАЖНО. После устранения причины файлы появляются не мгновенно. Сорвавшуюся строку система забирает на повтор в ближайший обход очереди, а он идёт раз в пять минут (`ARCHIVE_SWEEP_INTERVAL`); строки, остановленные нехваткой места, повторяются раз в пятнадцать минут. Ждать нужно этот срок, а не нажимать повтор в разделе: он делает ту же работу, только раньше. Срок ближайшей попытки подписан у строки в ленте раздела администрирования.

13.13. Письма не приходят

Признак: работники сообщают, что не получают писем от системы. Проверки идут сверху вниз: сначала выясняется, отправляет ли система вообще, затем что отвечает почтовый сервер, и только потом что происходит с письмом на стороне получателя.

Шаг 1. Отправить проверочное письмо. Порядок описан в подразделе 6.7. Ответ на него разделяет две разные неисправности. Ответ с отказом означает, что система не дошла до почтового сервера или он её не принял: причина в таблице ниже. Ответ об успешной отправке при неполученном письме означает, что письмо ушло, а потерялось уже на стороне получателя: проверьте папку со спамом и записи SPF и DKIM домена отправителя, см. подраздел 6.7.

Шаг 2. Прочитать очередь. Проверочное письмо отправляется мимо очереди, поэтому его успешная отправка ещё не означает, что очередь разбирается. Состояние писем видно в таблице `email_messages`.

Листинг 13.4 — Состояние очереди писем

```
docker compose -f docker-compose.base.yml -f docker-compose.prod.yml \
  exec -T db psql -U postgres -d "$DB_NAME" \
  -c "SELECT id, to_address, template_code, status, attempts,
            next_attempt_at, last_error
       FROM email_messages ORDER BY created_at DESC LIMIT 20;"
```

Ожидаемый результат: перечень последних писем. Значение `sent` в столбце состояния означает переданное почтовому серверу письмо, `pending` - ожидающее отправки, `failed` - письмо, по которому исчерпаны попытки. Столбец `attempts` показывает число сделанных попыток, `next_attempt_at` - время ближайшей следующей, `last_error` - дословный ответ почтового сервера на последнюю неудачную.

ВАЖНО. Столбец `body` в запрос включать не следует. В тексте письма могут быть сведения, предназначенные только получателю, и в выводе на экран сервера им не место. По этой же причине текст письма не отдаётся и через программный интерфейс.

Таблица 13.8 — Толкование состояния очереди

Что видно	Толкование
Таблица пуста, писем нет вовсе	Ни одно событие не поставило письмо в очередь. При выключенной почте это штатно: постановка отклоняется сразу
Письма в состоянии `pending`, число попыток не растёт	Фоновая задача разбора не работает. Проверить журнал прикладного сервера: при пустом `SMTP_HOST` при запуске записано, что почта не настроена и задача не запущена
Число попыток растёт, состояние `pending`	Почтовый сервер отказывает, отправка повторяется. Причина в столбце последней ошибки, разбор в следующей таблице
Письма в состоянии `failed`	Попытки исчерпаны, система больше не пытается, причина записана в журнал прикладного сервера. Устранение причины прежние письма не возобновляет: заново отправляются только новые

Шаг 3. Разобрать ответ почтового сервера. Причину даёт ответ сервера в столбце последней ошибки или в ответе на проверочное письмо. Отказов немного, и у каждого типовая причина.

Таблица 13.9 — Типовые отказы почтового сервера

Ответ	Причина	Что делать
535, отказ в проверке подлинности	Логин или пароль не приняты	Проверить `SMTP_USERNAME` и `SMTP_PASSWORD`. Почтовая служба может требовать отдельный пароль приложения вместо пароля веб-почты
550, отправитель не разрешён	Адрес в `SMTP_FROM` не совпадает с ящиком, под которым система входит на сервер	Привести `SMTP_FROM` к адресу ящика из `SMTP_USERNAME`
Истёк срок ожидания ответа	Соединение до почтового сервера не проходит	Проверить исходящее соединение с самого сервера, см. подраздел 6.7. Обычная причина - закрытый исходящий порт
Соединение отклонено, имя не разрешается	Неверный адрес или порт почтового сервера	Проверить `SMTP_HOST` и `SMTP_PORT`
Ошибка защищённого соединения, отказ по сертификату	Режим защиты не соответствует порту	Привести `SMTP_TLS_MODE` в соответствие: порту 587 обычно отвечает `starttls`, порту 465 - `tls`
Почта не настроена	`SMTP_HOST` пуст либо не дошёл до прикладного сервера	Проверить, что параметры переданы в контейнер, см. подраздел 6.7

ПРИМЕЧАНИЕ. Система переводит ответ почтового сервера на понятный язык сама, поэтому в ответе на проверочное письмо видно и объяснение причины, и дословный ответ сервера. В очереди хранится только дословный ответ, обрезанный до 500 символов: код ответа стоит в его начале и при обрезке не теряется.

13.14. Система не требует смены паролей по истечении срока

Признак: срок действия паролей включён, а работников с давними паролями система по-прежнему пускает без смены. Проверки идут сверху вниз, от общего выключателя к отдельной учётной записи. Порядок настройки описан в подразделе 9.3.

Таблица 13.10 — Диагностика срока действия паролей

Проверка	Толкование
Тумблер «Требовать смену пароля по истечении срока», раздел «Настройки», вкладка «Безопасность»	Выключен - проверки сроков не будет вовсе. В поставке он выключен, и включать его требуется отдельно

Продолжение таблицы 13.10

Проверка	Толкование
Значение «Срок действия пароля вышел у работников»	Ноль означает, что срок не вышел ни у кого: либо срок задан большим, либо пароли недавно менялись. Это штатное состояние, а не неисправность
Значение «Ближайшая проверка»	Проверка идёт ежесуточно в 04:00 по часовому поясу `RESET_TIMEZONE`. Если систему запустили после 04:00, первая проверка будет только на следующие сутки
Журнал прикладного сервера	`make deploy-logs`: при запуске записывается строка о запуске планировщика со временем ближайшей проверки, по завершении - сколько паролей помечено истёкшими и сколько пометить не удалось
Открытая вкладка работника	Требование вступает в силу не позже чем через 30 секунд и на уже открытой вкладке: она упирается в форму смены на первом же обращении к серверу. Ждать, пока истечёт выданный ранее маркер доступа, не нужно

ПРИМЕЧАНИЕ. Почта этой возможности не нужна, и её отсутствие проверку сроков не останавливает. Без настроенной почты не уходят только предупреждения о скором истечении, и работник узнаёт о нём уже на входе.

Отдельный случай - жалоба вида «система требует сменить пароль и никуда не пускает». Это не неисправность, а работа самого механизма: либо у работника вышел срок действия пароля, либо ему выслали придуманный системой пароль и включена настройка «Требовать сменить пароль при первом входе». Пока он не задаст свой пароль в открывшейся форме, система в остальном ему отказывает. Выключать настройку ради удобства не следует - именно она ограничивает срок жизни пароля, лежащего в почтовом ящике открытым текстом, а на смену по истечении срока она всё равно не влияет.

Признак другого рода: работникам разослали новые пароли кнопкой «Обновить пароль всем работникам» или из карточки, а писем никто не получил.

Таблица 13.11 — Диагностика рассылки придуманных паролей

Проверка	Толкование
Строка о ненастроенной почте в блоке «Срок действия паролей»	Почта не настроена: кнопка «Обновить пароль всем работникам» не показывается, смена из карточки отвечает отказом. Настройка описана в подразделе 6.7

Продолжение таблицы 13.11

Проверка	Толкование
Значение «Без почты»	Учётные записи без указанного адреса рассылка пропускает: выслать пароль некуда. Полный список открывается в разделе «Пользователи» переключателем режима в положение «Без почты», адреса проставляет бюро
Очередь писем	Пароли сменились, а писем нет - неисправность не в смене, а в отправке, см. подраздел 13.13. Письма ищутся в очереди по виду `password_rotated`

ВАЖНО. Пароль работника, у которого письмо не дошло, уже сменён - прежний не действует. Такому работнику пароль восстанавливают отдельно: в разделе «Пользователи», в карточке работника, кнопкой «Сменить пароль и отправить письмом» после того, как устранена причина недоставки, либо заданием пароля вручную там же.

13.15. Правка файла настроек не подействовала

Признак: файл настроек обратного прокси или другой файл, отданный контейнеру по отдельности, исправлен, прокси перезагружен командой из подраздела 7.5, а поведение прежнее. Проверка внутри контейнера показывает старое содержимое, тогда как на сервере лежит новое.

Листинг 13.5 — Сверка содержимого на сервере и в контейнере

```
grep -n "<искомая строка>" nginx/nginx.conf
sudo docker exec systemburo-nginx-1 \
  grep -n "<искомая строка>" /etc/nginx/conf.d/default.conf
```

Ожидаемый результат при неисправности: строка есть в первом выводе и отсутствует во втором.

Причина не в перезагрузке. Отдельный файл отдаётся контейнеру по своему месту на диске, а не по имени в каталоге, и связь устанавливается один раз при создании контейнера. Редакторы и потоковые обработчики - `sed -i`, `vim` с настройками по умолчанию, `mv` поверх - обычно не пишут в тот же файл, а создают новый и заменяют им прежний. Прежний файл при этом остаётся существовать - его продолжает держать контейнер, - и все дальнейшие правки уходят в новый файл, которого контейнер не видит.

Перезагрузка прокси перечитывает старое содержимое, поэтому и выглядит бесполезной.

Листинг 13.6 — Применение правки

```
sudo docker compose -f docker-compose.base.yml -f docker-compose.prod.yml \
  up -d --force-recreate nginx
```

Ожидаемый результат: контейнер пересоздан, сверка содержимого показывает новую строку в обоих выводах.

ВАЖНО. Следствие для всей эксплуатации - файлы, отданные контейнеру по отдельности (настройки обратного прокси, сертификат и ключ), после правки требуют пересоздания контейнера. Перезагрузки достаточно только там, где содержимое файла изменено записью в него же: так делает сценарий подготовки сертификата, поэтому в подразделах 7.3 и 7.5 перезагрузка указана верно. Если сомневаетесь, что именно сделал редактор, - пересоздайте контейнер: это безопасно и занимает те же секунды.

13.16. Ошибка получения исходников после команд root

Признак: `git fetch` в каталоге системы отвечает `Host key verification failed`, а после исправления доступа - `error: cannot open '.git/FETCH_HEAD': Permission denied`. Обновление по подразделу 10.5 на этом и останавливается.

Причина в смешивании двух учётных записей в одном каталоге. Доступ к репозиторию выдан учётной записи сопровождения (подраздел 4.1), и ключ вместе с записью о хосте лежит в её домашнем каталоге - у суперпользователя их нет, отсюда первая ошибка. Вторая появляется после того, как в каталоге хоть раз выполнили `git` от суперпользователя: служебные файлы `.git/HEAD`, `.git/index` и `.git/FETCH_HEAD` пересоздаются с его владельцем, и учётная запись сопровождения больше не может в них писать.

Листинг 13.7 — Возврат владельца служебных файлов

```
sudo find /opt/systemburo/.git -user root
sudo chown buro:buro $(sudo find /opt/systemburo/.git -user root -printf '%p ')
```

Ожидаемый результат: первая команда перечисляет файлы, доставшиеся суперпользователю, после второй тот же поиск ничего не находит, а `git fetch` от учётной записи сопровождения выполняется.

ВАЖНО. Разделение простое - `git` в каталоге системы выполняется только от учётной записи сопровождения, а команды управления системой требуют прав на Docker и выполняются через `sudo` (подраздел 9.1). Смешивать нельзя: сборка от суперпользователя каталогу `.git` не вредит, а вот `git checkout`, выполненный им же, ломает следующее обновление. Проверять владельца стоит и после любого разбора неисправностей, в котором приходилось работать от суперпользователя.

14. Перенос системы на другой сервер

Раздел описывает перенос работающей системы вместе со всеми данными: например, с сервера во внутренней сети организации на арендованный сервер. Порядок обеспечивает сохранность данных и возможность вернуться к прежнему серверу, если что-то пойдёт не так.

14.1. Что переносится

Таблица 14.1 — Состав переносимых данных

Элемент	Где находится	Последствия, если не перенести
База данных	Том `pgdata`	Утрата всех заявок, реестров, учётных записей и журналов
Загруженные файлы	Том `uploads`	Фотографии, шаблоны бланков и приложения к заявкам пропадут, записи в базе будут ссылаться в пустоту
Файл параметров	`.env` в каталоге системы	См. предупреждение ниже
Сертификат	`nginx/certs/`	Потребуется выпустить заново, что при смене адреса и так необходимо

ОПАСНО. Файл параметров переносится со старого сервера, а не создаётся заново. В нём хранится ключ шифрования персональных данных. Новый или чужой ключ не расшифрует перенесённые данные: приложение откажет при проверке ключа. Сохраните прежний ключ в защищённом хранилище; отказ запуска не изменяет данные базы.

14.2. Что меняется при переносе

Следующие параметры сверяют при смене адреса системы. Дополнительно проверьте `EDGE_TRUSTED_PROXY_CIDRS`: на новой площадке адрес непосредственного внешнего прокси может измениться, а при прямом подключении к ВМ значение оставляют пустым. Для нескольких установок на одном Docker-сервере проверьте отсутствие пересечений `PROXY_NETWORK_PREFIX`, см. подраздел 7.6.

Таблица 14.2 — Параметры, зависящие от адреса

Параметр	Что задаёт
`CORS_ALLOWED_ORIGINS`	Адреса, с которых интерфейсу разрешено обращаться к серверу
`VITE_API_BASE_URL`	Адрес, по которому интерфейс обращается к серверу
`COMPOSE_PROJECT_NAME`	Имя установки, от которого зависят имена томов с данными

ВАЖНО. Адрес сервера встраивается в интерфейс на этапе сборки, а не читается при запуске. После изменения адреса образ интерфейса необходимо собрать заново, иначе интерфейс на новом сервере продолжит обращаться к старому.

14.3. Подготовка нового сервера

Выполните на новом сервере подразделы 5.1, 5.2 и 5.3: установите служебные пакеты и Docker, разместите исходный код в рабочем каталоге. Создание файла параметров и первый запуск пока не выполняйте.

ВАЖНО. Новому серверу нужен собственный доступ к репозиторию. Ключ развёртывания старого сервера не переносится вместе с данными: пара ключей создаётся на новом сервере, её открытая часть добавляется в репозиторий, а после вывода прежнего сервера из работы его ключ отзывается, см. подраздел 4.1.

14.4. Выгрузка данных со старого сервера

Работы выполняются при остановленном прикладном сервере, чтобы данные не менялись во время выгрузки. База данных при этом остаётся запущенной.

Шаг 1. Загрузите параметры установки в оболочку. Без этого имя базы и имя тома окажутся пустыми, и команды выгрузки отработают вхолостую.

Листинг 14.1 — Загрузка параметров в оболочку

```
cd /opt/systemburo
set -a; . ./.env; set +a
BACKEND_ID=$(sudo docker compose -f docker-compose.base.yml \
  -f docker-compose.prod.yml ps -a -q backend)
[ -n "$BACKEND_ID" ] || { echo 'Контейнер backend не найден'; exit 1; }
UPLOADS_VOL=$(sudo docker inspect --format \
  '{{range .Mounts}}{{if eq .Destination "/app/uploads"}}{{.Name}}{{end}}{{end}}' \
  "$BACKEND_ID")
[ -n "$UPLOADS_VOL" ] || { echo 'Именованный том загрузок не найден'; exit 1; }
echo "база: $DB_NAME, том файлов: $UPLOADS_VOL"
```

Ожидаемый результат: выведены непустые имя базы и имя тома, например `база: auto_registry, том файлов: systemburo_uploads`. Если что-то пусто, дальше идти нельзя.

Шаг 2. Остановите приём заявок.

Листинг 14.2 — Остановка приёма заявок

```
sudo docker compose -f docker-compose.base.yml -f docker-compose.prod.yml \
  stop backend frontend nginx
```

Ожидаемый результат: три службы остановлены, служба базы данных продолжает работать.

Шаг 3. Выгрузите базу данных.

Листинг 14.3 — Выгрузка базы данных

```
sudo docker compose -f docker-compose.base.yml -f docker-compose.prod.yml \
  exec -T db pg_dump -U postgres -Fc "$DB_NAME" > /tmp/buro-db.dump
ls -lh /tmp/buro-db.dump
```

Ожидаемый результат: файл создан, размер отличен от нуля и составляет обычно от десятков мегабайт.

Шаг 4. Убедитесь, что выгрузка читается. Повреждённый файл лучше обнаружить сейчас, а не на новом сервере.

Листинг 14.4 — Проверка выгрузки

```
sudo docker compose -f docker-compose.base.yml -f docker-compose.prod.yml \
  exec -T db pg_restore --list < /tmp/buro-db.dump | head -20
```

Ожидаемый результат: выводится перечень объектов базы. Сообщение об ошибке формата означает, что выгрузка повреждена и её нужно повторить.

Шаг 5. Выгрузите загруженные файлы.

Листинг 14.5 — Выгрузка загруженных файлов

```
sudo docker run --rm \
  -v "$UPLOADS_VOL":/data:ro \
  -v /tmp:/backup alpine \
  tar czf /backup/buro-uploads.tar.gz -C /data .
ls -lh /tmp/buro-uploads.tar.gz
```

Ожидаемый результат: архив создан, размер отличен от нуля.

Шаг 6. Передайте на новый сервер три файла: выгрузку базы, архив файлов и файл параметров.

Листинг 14.6 — Передача файлов на новый сервер

```
scp /tmp/buro-db.dump /tmp/buro-uploads.tar.gz .env \
  buro@<адрес нового сервера>:/tmp/
```

ВАЖНО. Файл параметров содержит пароли и ключ шифрования персональных данных. Передавайте его защищённым каналом и удалите промежуточные копии из каталога `/tmp` на обоих серверах после завершения переноса.

14.5. Восстановление на новом сервере

Шаг 1. Разместите файл параметров в каталоге системы и внесите изменения по подразделу 14.2: новый адрес в `CORS_ALLOWED_ORIGINS` и `VITE_API_BASE_URL`. Строку подключения к базе менять не требуется, она указывает на службу внутри контура.

Листинг 14.7 — Размещение файла параметров

```
cd /opt/systemburo
cp /tmp/.env ./.env
chmod 600 ./.env
set -a; . ./.env; set +a
echo "база: $DB_NAME"
```

Ожидаемый результат: выведено непустое имя базы.

Шаг 2. Запустите базу данных и восстановите выгрузку.

Листинг 14.8 — Запуск базы данных и восстановление

```
sudo docker compose -f docker-compose.base.yml -f docker-compose.prod.yml \
  up -d db
sleep 10
sudo docker compose -f docker-compose.base.yml -f docker-compose.prod.yml \
  exec -T db pg_restore -U postgres -d "$DB_NAME" --clean --if-exists \
  < /tmp/buro-db.dump
```

Ожидаемый результат: восстановление завершается без сообщений об ошибках. Отдельные предупреждения о несуществующих объектах при первом восстановлении нормальны.

Шаг 3. Проверьте, что данные на месте, ещё до запуска приложения.

Листинг 14.9 — Проверка восстановленной базы

```
sudo docker compose -f docker-compose.base.yml -f docker-compose.prod.yml \
  exec -T db psql -U postgres -d "$DB_NAME" \
  -c "SELECT count(*) AS applications FROM applications;" \
  -c "SELECT count(*) AS users FROM users;"
```

Ожидаемый результат: числа совпадают с прежним сервером. Нули означают, что восстановление не выполнено.

Шаг 4. Восстановите загруженные файлы. Имя тома должно совпадать с тем, которое создаст описание контура.

Листинг 14.10 — Восстановление загруженных файлов

```
UPLOADS_VOL="$(basename $PWD)_uploads"
[ -n "$COMPOSE_PROJECT_NAME" ] \
  && UPLOADS_VOL="${COMPOSE_PROJECT_NAME}_uploads"
sudo docker volume create "$UPLOADS_VOL"
sudo docker run --rm \
  -v "$UPLOADS_VOL":/data \
  -v /tmp:/backup alpine \
  tar xzf /backup/buro-uploads.tar.gz -C /data
sudo docker run --rm -v "$UPLOADS_VOL":/data alpine \
  sh -c "ls /data | head; ls /data | wc -l"
```

Ожидаемый результат: выводится перечень каталогов и их число, совпадающее с прежним сервером.

14.6. Сертификат и запуск

Подготовьте сертификат для нового адреса по разделу 7. При переносе в сеть общего пользования это возможность перейти на сертификат

общедоступного удостоверяющего центра и отказаться от установки корневого сертификата на рабочие места.

Листинг 14.11 — Сборка и запуск на новом сервере

```
sudo make deploy-build
sudo make deploy-up
sudo make deploy-logs
```

Ожидаемый результат: службы запускаются, в журнале нет сообщений уровня `error`. Приведение схемы базы данных выполняется быстро: схема уже соответствует восстановленным данным.

14.7. Проверка после переноса

Проверки выполняются до переключения пользователей на новый сервер.

Таблица 14.3 — Проверки после переноса

Проверка	Ожидаемый результат
Вход администратора	Выполняется с прежним паролем
Число заявок в центре заявок	Совпадает с прежним сервером
Открытие заявки со вложениями	Открывается, файлы и фотографии отображаются
Открытие карточки человека	Сведения документа, удостоверяющего личность, читаются
Открытие таблицы поста	Перечень допущенных объектов на месте
Формирование печатной формы	Формируется, шаблон найден
Журналы	Записи за прежний период доступны

ВАЖНО. Проверка чтения сведений документа обязательна. Запуск уже проверяет соответствие ключа базе, а открытие карточки дополнительно подтверждает чтение нужных зашифрованных полей и доступ к ним в рабочем сценарии. Не подменяйте его одной проверкой доступности сервера.

14.8. Переключение и вывод прежнего сервера из работы

После успешной проверки измените запись в системе доменных имён либо сообщите работникам новый адрес.

Прежний сервер не выключайте сразу. Держите его остановленным, но сохранным не менее двух недель: этого срока достаточно, чтобы проявились

расхождения, незаметные при первичной проверке. Возврат в этом случае сводится к обратному переключению адреса.

Данные, внесённые на новом сервере после переключения, при возврате будут потеряны, поэтому решение о возврате принимается в первые сутки.

По истечении срока хранения удалите с прежнего сервера тома с данными и файл параметров: они содержат персональные данные и ключ шифрования. Тогда же отзовите его доступ к репозиторию исходного кода - удалите ключ развёртывания из настроек репозитория, порядок в подразделе 4.1. Пока ключ действует, выведенный из работы сервер сохраняет право получать код системы.

Приложение А. Порты и сетевые доступы

Таблица А.1 — Сетевые доступы

Порт	Протокол	Откуда доступен	Назначение
443	TCP	Сеть организации	Работа с системой
80	TCP	Сеть организации	Перенаправление, проверка при выпуске сертификата, проверка состояния
22	TCP	Адреса подразделения ИТ	Управление сервером
8080	TCP	Только внутренняя сеть контейнеров	Прикладной сервер
80	TCP	Только внутренняя сеть контейнеров	Веб-приложение
5432	TCP	Только внутренняя сеть контейнеров	База данных

Снаружи остаются доступны ровно три порта из этого перечня: 80 и 443 открывает сама система, 22 нужен для управления сервером и к системе отношения не имеет. Остальное закрывает межсетевой экран политикой по умолчанию, порядок его настройки - в подразделе 5.10. Порты внутренней сети контейнеров экраном не управляются: они не публикуются на внешние адреса вовсе.

ВАЖНО. Экран не закрывает порты, опубликованные контейнерами. На рабочем сервере их публикует только обратный прокси, 80 и 443, поэтому перечень выше верен; но всякая новая публикация порта в описании контейнера открывает его наружу независимо от правил экрана, см. подраздел 5.10.

Во время работы внешние соединения нужны для настроенных возможностей из таблицы. Для установки, сборки и обновления дополнительно требуется доступ к используемым реестрам образов, пакетам и репозиторию либо заранее подготовленная автономная поставка.

Таблица А.2 — Исходящие соединения

Порт	Протокол	Куда	Назначение	Когда требуется
587 либо 465	TCP	Почтовый сервер из `SMTP_HOST`	Отправка писем системы	Настроена почтовая рассылка, подраздел 6.7
443	TCP	Удостоверяющий центр	Выпуск и продление сертификата	Сертификат выпускается общедоступным центром, подраздел 7.3
443	TCP	Службы Web Push браузеров	Доставка системных уведомлений на устройства	Настроены VAPID и подписки; разрешён маршрут согласно `PUSH_ALLOWED_HOSTS`

ВАЖНО. Почтовый порт и режим защиты согласуйте с почтовой службой. В таблице приведены обычные порты защищённой отправки; если организация использует другой порт, доступ настраивается для него. Не предполагайте, что исходящий SMTP разрешён сетью по умолчанию.

Проверка живости пользуется теми же направлениями: она обращается к системе по её внешнему адресу и отправляет письмо почтовым сервером из файла параметров, подраздел 12.2.

Нестандартные почтовые службы и исходящий прокси требуют собственных подтверждённых адресов и портов. Если каталог файлового архива отдаётся работникам сетевой папкой, порт файлового доступа поднимается средствами операционной системы, наружу не публикуется и ограничивается сетью организации, см. подраздел 9.7.

Приложение Б. Перечень параметров конфигурации

Таблица Б.1 — Подключение и запуск

Параметр	По умолчанию	Назначение
DATABASE_URL	нет, обязателен	Строка подключения к базе данных
DB_NAME	auto_registry	Имя базы данных
DB_USER	postgres	Пользователь базы данных
DB_PASSWORD	нет	Пароль пользователя базы данных
BIND_HOST	0.0.0.0	Адрес приёма соединений
BIND_PORT	8090	Порт прикладного сервера, в поставке задан 8080
LOG_LEVEL	info	Подробность журнала
SWAGGER_ENABLED	false	Публикация описания программного интерфейса

Таблица Б.2 — Выбор готовых образов PostgreSQL и pgAdmin

Параметр	По умолчанию	Назначение
POSTGRES_IMAGE	systemburo-postgres:production	Необязательное имя образа PostgreSQL для Compose; автоматическая поставка задаёт образ точного SHA
PGADMIN_IMAGE	systemburo-pgadmin:production	Необязательное имя образа инструмента pgAdmin в локальном Compose и на стенде; в рабочем Compose сервиса нет

Таблица Б.3 — Пул соединений, таймауты и проверка пароля

Параметр	По умолчанию	Назначение
DB_MAX_OPEN_CONNS	50	Наибольшее число одновременных соединений с базой; считается заодно с max_connections базы, см. подраздел 6.5
DB_MAX_IDLE_CONNS	25	Сколько соединений держится наготове между запросами; больше числа открытых задать нельзя, сервер откажется запуститься
DB_CONN_MAX_LIFETIME	1h	Наибольший срок жизни соединения
DB_CONN_MAX_IDLE_TIME	10m	Через сколько закрывается простаивающее соединение
HTTP_READ_HEADER_TIMEOUT	10s	На приём заголовков запроса
HTTP_READ_TIMEOUT	120s	На приём запроса целиком, вместе с телом

Продолжение таблицы Б.3

Параметр	По умолчанию	Назначение
HTTP_WRITE_TIMEOUT	120s	На работу обработчика и отправку ответа
HTTP_IDLE_TIMEOUT	120s	Сколько открытое соединение ждёт следующего запроса
ARGON2_HASH_CONCURRENCY	0	Сколько проверок пароля выполняется одновременно; 0 - по числу ядер процессора

Ноль у параметров пула означает, что соответствующая настройка к пулу не применяется и сохраняется исходное значение библиотеки. Нулевые HTTP-таймауты отключают соответствующий предел ожидания. Отрицательные значения отклоняются. У ARGON2_HASH_CONCURRENCY=0 отдельный смысл — число ядер процессора. Параметры передаются через environment Compose и задаются в .env, см. подраздел 6.5.

Таблица Б.4 — Маркеры доступа и вход

Параметр	По умолчанию	Назначение
JWT_SECRET	нет, обязателен	Ключ подписи маркера доступа
JWT_REFRESH_SECRET	нет, обязателен	Ключ подписи маркера продления
JWT_ACCESS_TTL	15m	Срок жизни маркера доступа
JWT_REFRESH_TTL	168h	Срок жизни маркера продления
COOKIE_SECURE	true	Передача маркера продления только по защищённому соединению
LOGIN_RATE_LIMIT_MAX	10	Попыток входа с одного адреса за интервал
LOGIN_RATE_LIMIT_WINDOW_SEC	60	Интервал подсчёта попыток, секунды

Таблица Б.5 — Защита персональных данных

Параметр	По умолчанию	Назначение
DATA_ENCRYPTION_KEY	пусто	Ключ шифрования, 64 шестнадцатеричных символа

Продолжение таблицы Б.5

Параметр	По умолчанию	Назначение
REQUIRE_ENCRYPTION	true	Запрет запуска без ключей шифрования персональных данных и файлового архива
PD_AUDIT_RETENTION_MONTHS	36	Срок хранения журнала обращений к персональным данным
APPLICATION_RETENTION_MONTHS	0	Месяцы до необратимого обезличивания закончившихся заявок; 0 выключает автоматическую обработку
AUDIT_RETENTION_MONTHS	0	Месяцы хранения истории сущностей при автоматической уборке; 0 выключает её

Таблица Б.6 — Ограничения обращений и загрузка файлов

Параметр	По умолчанию	Назначение
RATE_LIMIT_PER_MINUTE	200	Запросов в минуту на работника
RATE_LIMIT_WINDOW_SEC	60	Интервал подсчёта
RATE_LIMIT_IP_PER_MINUTE	2000	Общий предел обращений с одного IP-адреса за интервал RATE_LIMIT_IP_WINDOW_SEC; учитывается также до входа
RATE_LIMIT_IP_WINDOW_SEC	60	Интервал общего ограничения по IP-адресу в секундах
TRUSTED_PROXY_CIDRS	пусто	Адреса прокси, от которых прикладной сервер принимает адрес посетителя; при прямом запуске пусто. В Compose автоматически задаётся адрес nginx из сети backend_proxy

Продолжение таблицы Б.6

Параметр	По умолчанию	Назначение
EDGE_TRUSTED_PROXY_CIDRS	пусто	Подтверждённые CIDR внешнего входящего прокси перед nginx, через запятую. При прямом подключении к ВМ оставляется пустым; параметр читает сценарий запуска nginx
PROXY_NETWORK_PREFIX	172.30.250 для стенда; 172.30.251 для рабочего сервера	Первые три октета отдельной сети backend_proxy с маской /29. Меняется только при пересечении с занятыми сетями или нескольких установках на одном Docker-сервере
PAGINATION_MAX_LIMIT	100	Наибольший размер страницы выдачи
UPLOAD_PATH	./uploads	Каталог хранения файлов, в поставке /app/uploads
UPLOAD_MAX_FILE_SIZE	10485760	Наибольший размер файла в байтах
UPLOAD_ALLOWED_IMAGE_TYPES	image/jpeg,image/png,image/webp	Разрешённые типы изображений
UPLOAD_ALLOWED_DOC_TYPES	application/pdf и формат документов	Разрешённые типы документов

Таблица Б.7 — Файлы, приложенные к заявке

Параметр	По умолчанию	Назначение
APPLICATION_FILE_MAX_COUNT	30	Сколько файлов разрешено приложить к одной заявке
APPLICATION_FILE_MAX_TOTAL_SIZE	104857600	Наибольший суммарный размер файлов одной заявки в байтах, 100 МБ
APPLICATION_FILE_DRAFT_MAX_COUNT	50	Предел количества файлов-черновиков одной учётной записи; учитывает параллельные загрузки этой записи
APPLICATION_FILE_DRAFT_MAX_TOTAL_SIZE	209715200	Предел суммарного размера файлов-черновиков одной учётной записи в байтах, 200 МиБ

Продолжение таблицы Б.7

Параметр	По умолчанию	Назначение
APPLICATION_FILE_DRAFT_TTL	24h	Срок хранения файла-черновика от момента загрузки; после этого файл может удалить фоновая уборка. К файлам поданных заявок этот срок не применяется
APPLICATION_FILE_IMAGE_MAX_SIDE	2000	Предел длинной стороны изображения в точках; снимки крупнее уменьшаются
APPLICATION_FILE_JPEG_QUALITY	82	Качество перекодирования снимков, от 1 до 100

Таблица Б.8 — Файловый архив бланков

Параметр	По умолчанию	Назначение
ARCHIVE_PATH	./archive	Каталог файлового архива бланков внутри контейнера, в поставке /app/archive; на сервере ему соответствует ARCHIVE_HOST_PATH
ENTITY_EXPORT_PATH	пусто	Каталог, куда консольная выгрузка складывает пакет с данными организации. Пока значение не задано, команда выгрузки отказывает с подсказкой и ничего не пишет: место хранения выбирает тот, кто разворачивает систему, а в пакете лежат персональные данные целиком. Каталог обязан лежать вне каталога загрузок - по той же причине, что и каталог файлового архива
ARCHIVE_HOST_PATH	./archive	Каталог файлового архива на диске сервера, подключаемый к контейнеру. На рабочем сервере сценарий подготовки параметров задаёт /srv/systemburo/archive; каталог создают до первого запуска и отдают пользователю прикладного сервера, см. подраздел 9.6

Продолжение таблицы Б.8

Параметр	По умолчанию	Назначение
ARCHIVE_AGE_RECIPIENT	пусто	Открытый ключ принимающей стороны для шифрования файлов архива. При REQUIRE_ENCRYPTION=true отсутствие необходимых ключей блокирует запуск; незашифрованная запись возможна только при явно отключённом требовании
ARCHIVE_AGE_IDENTITY	пусто	Закрытый ключ системы, вторым получателем. Без него служба не прочитает собственный архив и не соберёт выгрузку по кнопке; задаётся только вместе с предыдущим
ARCHIVE_WORKER_TICK	15s	Как часто разбирается очередь выгрузки бланков
ARCHIVE_SWEEP_INTERVAL	5m	Как часто повторяются выгрузки, сорвавшиеся по временной причине

Таблица Б.9 — Почтовая рассылка

Параметр	По умолчанию	Назначение
SMTP_HOST	пусто	Адрес почтового сервера. Пустое значение выключает отправку писем целиком
SMTP_PORT	587	Порт почтового сервера: 587 при starttls, 465 при tls
SMTP_USERNAME	пусто	Учётная запись для входа на почтовый сервер, обычно полный адрес ящика. Задаётся вместе с паролем; оба пустых означают сервер без проверки подлинности
SMTP_PASSWORD	пусто	Пароль к ней. Почтовая служба может требовать отдельный пароль приложения вместо пароля веб-почты
SMTP_FROM	пусто	Адрес отправителя. Обязателен при заданном адресе сервера и должен совпадать с ящиком входа, иначе письма отвергаются с ошибкой 550
SMTP_FROM_NAME	Бюро пропусков	Имя отправителя, показываемое получателю рядом с адресом
SMTP_TLS_MODE	starttls	Режим защиты соединения: starttls, tls или none. Значение none допустимо только для сервера в закрытом контуре
SMTP_TIMEOUT_SEC	15	Предельный срок ожидания ответа почтового сервера, секунды
SMTP_RATE_PER_HOUR	400	Собственный предел отправки, писем в час. Задаётся заведомо ниже предела поставщика услуг

Продолжение таблицы Б.9

Параметр	По умолчанию	Назначение
MAIL_RETRY_ATTEMPTS	5	Число попыток доставки, после которого письмо признаётся недоставленным
MAIL_WORKER_TICK	15s	Как часто разбирается очередь писем

Таблица Б.10 — Журналы и обслуживание

Параметр	По умолчанию	Назначение
LOG_FILE_PATH	пусто	Файл журнала, пустое значение означает вывод только в поток
LOG_MAX_SIZE_MB	100	Размер файла до смены
LOG_MAX_AGE_DAYS	30	Срок хранения файлов журнала
LOG_MAX_BACKUPS	14	Число хранимых файлов
LOG_COMPRESS	true	Сжатие при смене
REQUEST_LOG_DETAIL_DAYS	30	Глубина подробных записей журнала запросов
REQUEST_LOG_PARTITION_PRECREATE_DAYS	7	На сколько суток вперёд создаются разделы
REFRESH_TOKEN_RETENTION_DAYS	30	Через сколько суток удаляются недействительные маркеры продления
READ_NOTIFICATION_RETENTION_DAYS	30	Через сколько суток удаляются прочитанные уведомления
NOTIFICATION_RETENTION_DAYS	90	Через сколько суток удаляются непрочитанные уведомления
MAIL_MESSAGE_RETENTION_DAYS	30	Срок хранения отработанных записей очереди почты; не срок хранения писем в ящике получателя

Продолжение таблицы Б.10

Параметр	По умолчанию	Назначение
VAPID_PUBLIC_KEY	пусто	Открытый ключ подписи уведомлений в браузер. Пустой вместе с закрытым выключает доставку вне системы
VAPID_PRIVATE_KEY	пусто	Закрытый ключ подписи уведомлений в браузер. Хранится как секрет
PUSH_ALLOWED_HOSTS	пусто	Разрешённые имена служб Web Push, через запятую, без схемы, порта и пути. При пустом списке за исходящим прокси используются известные службы; при прямом соединении проверяется сетевой адрес. Непустой список ограничивает оба маршрута; слово known включает известные службы вместе с указанными вручную
VAPID_SUBJECT	пусто	Контакт бюро для служб доставки уведомлений, вида mailto: или адрес сайта
PUSH_SUBSCRIPTION_RETENTION_DAYS	180	Через сколько суток удаляется подписка устройства, которому ни разу не удалось доставить
ANALYTICS_CACHE_REFRESH_SEC	60	Периодичность пересчёта показателей

Таблица Б.11 — Резервное копирование

Параметр	По умолчанию	Назначение
BACKUP_DIR	/var/backups/systemburo	Каталог копий на сервере

Продолжение таблицы Б.11

Параметр	По умолчанию	Назначение
BACKUP_OPERATOR_GROUP	buro	Группа учётной записи оператора, которой открывается состояние копирования: отчёт о последнем запуске и журнал. Сами копии и их перечень остаются закрытыми. Пустое значение отключает доступ; отсутствие строки в файле означает значение по умолчанию. См. подраздел 11.4
BACKUP_AGE_RECIPIENT	пусто	Открытый ключ шифрования копий. Пока не задан, копирование останавливается с ошибкой и копию не создаёт
BACKUP_ALLOW_UNENCRYPTED	пусто	Осознанное разрешение снимать копии без шифрования; принимается единственное значение yes. Копия создаётся открытой, о чём сообщают вывод, журнал и команда состояния
BACKUP_KEEP_DAILY	7	Сколько суточных копий хранить
BACKUP_KEEP_WEEKLY	4	Сколько недельных копий хранить
BACKUP_KEEP_MONTHLY	6	Сколько месячных копий хранить
BACKUP_UPLOADS_MODE	weekly	Как часто архивировать файлы: weekly или daily

Продолжение таблицы Б.11

Параметр	По умолчанию	Назначение
BACKUP_S3_REMOTE	пусто	Имя настройки rclone для выгрузки вне сервера
BACKUP_S3_BUCKET	пусто	Бакет и префикс во внешнем хранилище

Таблица Б.12 — Прочие параметры

Параметр	По умолчанию	Назначение
CORS_ALLOWED_ORIGINS	http://localhost:8081	Адреса, которым разрешены обращения к интерфейсу
RESET_TIMEZONE	Europe/Moscow	Часовой пояс суточного сброса статусов
COMPOSE_PROJECT_NAME	имя каталога; сценарий подготовки параметров задаёт systemburo	Имя установки, определяет имена томов с данными
VITE_API_BASE_URL	нет	Адрес, по которому интерфейс обращается к серверу. Значение встраивается в образ интерфейса при сборке, поэтому после его изменения образ пересобирают

Параметры следующей таблицы прикладной сервер тоже не читает: их читает сценарий проверки живости, работающий на самом сервере, рядом с системой. Сценарий подготовки параметров их не создаёт - строки берут из образца `.env.example` и заполняют руками, см. подраздел 12.2.

Таблица Б.13 — Проверка живости системы

Параметр	По умолчанию	Назначение
HEALTH_CHECK_URL	пусто	Внешний адрес системы, тот же, по которому её открывает человек. Пусто - проверка останавливается с ошибкой

Продолжение таблицы Б.13

Параметр	По умолчанию	Назначение
HEALTH_CHECK_USERNAME	пусто	Имя учётной записи, которой проверка входит в систему. Пусто - вход не проверяется
HEALTH_CHECK_PASSWORD	пусто	Пароль этой учётной записи. Хранится в открытом виде, как и пароль почты
HEALTH_CHECK_BASIC_AUTH	пусто	Логин и пароль базовой авторизации прокси, если адрес закрыт ею; вида `логин:пароль`
HEALTH_ALERT_TO	пусто	Кому писать об отказе, несколько адресов через запятую. Пусто - письма не отправляются
HEALTH_ALERT_REPEAT_MIN	60	Через сколько минут повторять напоминание, пока отказ длится; `0` отключает напоминания
HEALTH_STATE_DIR	/var/lib/systemburo/health	Каталог журнала и файла состояния проверки, закрыт правами 700
HEALTH_TIMEOUT_SEC	20	Сколько ждать ответа от системы и от почтового сервера
HEALTH_CHECK_DB	auto	Проверять ли базу напрямую: `auto` - только если система стоит на этой же машине, `on` - всегда, `off` - никогда

Письма проверка отправляет напрямую по протоколу почты, беря адрес сервера и учётные данные из параметров `SMTP_*` того же файла.

Параметры следующей таблицы прикладной сервер не читает: их берут обратный прокси проверочного стенда и панель управления базой данных. На рабочем сервере они не действуют, и сценарий подготовки параметров их туда не записывает.

Таблица Б.14 — Доступ к проверочному стенду

Параметр	**По умолчанию**	**Назначение**
`BASIC_AUTH_USER`	`admin`	Имя, которое прокси стенда спрашивает при открытии адреса
`BASIC_AUTH_PASS`	`changeme`, значение из состава поставки	Пароль к нему; сценарий подготовки параметров задаёт случайный. См. подраздел 8.5
`PGADMIN_EMAIL`	нет	Учётная запись входа в панель управления базой данных стенда
`PGADMIN_PASSWORD`	нет	Пароль к ней; сценарий подготовки параметров задаёт случайный

Параметры выше описывают чтение конфигурации прикладным сервером. Чтобы значение из .env подействовало в контейнере, оно должно передаваться через environment Compose: наличие строки в .env само по себе передачи не создаёт. Настройки хранения не включают молча при обновлении.

Приложение В. Файлы, необходимые для запуска

Перечень содержимого поставки, без которого система не соберётся и не запустится.

Таблица В.1 — Контейнеры и конфигурация

Путь	Назначение
`docker-compose.base.yml`	Описание состава системы, общее для стенда и рабочего сервера
`docker-compose.prod.yml`	Дополнение для рабочего сервера: параметры базы, ресурсы, обратный прокси
`docker-compose.staging.yml`	Дополнение для проверочного стенда: запуск четырёх готовых образов одного SHA, отдельный профиль tools для pgAdmin и сохранённая возможность ручной сборки
`Dockerfile`	Сборка образа прикладного сервера
`frontend/Dockerfile`	Сборка образа веб-приложения
`postgres/Dockerfile`	Контекст сборки образа PostgreSQL с закреплённым базовым образом и сборкой gosu
`pgadmin/Dockerfile`	Контекст сборки образа инструмента pgAdmin для локального запуска и профиля tools стенда
`pgadmin/security-requirements.txt`	Закреплённая версия и SHA-256 дополнительного пакета, используемого при сборке pgAdmin
`frontend/nginx.conf`	Конфигурация отдачи статики внутри образа веб-приложения
`nginx/nginx.conf`	Конфигурация обратного прокси рабочего сервера
`nginx/staging.conf`	Конфигурация обратного прокси стенда
`nginx/Dockerfile.staging`	Сборка образа прокси для стенда: базовый образ дополняется средством создания файла паролей; используется CI при автоматической поставке и при ручной сборке
`nginx/staging-entrypoint.sh`	Создание файла паролей и пропуска стенда при каждом запуске прокси
`nginx/proxy-trust-entrypoint.sh`	Проверка и формирование настройки доверенного внешнего прокси для nginx

Таблица В.2 — Эксплуатационные сценарии

Путь	Назначение
`Makefile`	Команды управления
`scripts/init-env.sh`	Создание файла параметров
`scripts/init-tls.sh`	Выпуск и продление сертификата
`scripts/maintenance-off.sh`	Снятие режима технических работ мимо интерфейса
`scripts/backup.sh`	Снятие резервной копии: выгрузка базы, архивы файлов, раскладка по срокам хранения, выгрузка вне сервера
`scripts/backup-install.sh`	Постановка копирования и ежемесячной проверки восстановимости на расписание
`scripts/backup-status.sh`	Состояние копий: свежесть, состав, режим шифрования, итог последнего запуска
`scripts/backup-verify.sh`	Проверка того, что копия разворачивается, во временной базе рядом с рабочей
`scripts/backup-snapshot.sh`	Общие функции копирования и проверки. Получают выгрузку и счётчики из одного снимка PostgreSQL, формируют и читают контрольные данные.
`scripts/restore.sh`	Восстановление базы и загруженных файлов из копии
`scripts/health-check.sh`	Проверка сайта, входа и базы с оповещением на почту при отказе
`scripts/health-install.sh`	Постановка проверки живости на расписание
`scripts/package.sh`	Сборка архива поставки для площадки без доступа к репозиторию

Таблица В.3 — Исходники и зависимости

Путь	Назначение
`go.mod`, `go.sum`	Перечень библиотек серверной части
`cmd/`, `internal/`, `docs/`	Исходный код серверной части и описание программного интерфейса
`frontend/package.json`, `frontend/package-lock.json`	Перечень библиотек интерфейса
`frontend/src/`, `frontend/index.html`, `frontend/vite.config.js`	Исходный код интерфейса

Создаются на сервере при установке и в поставку не входят:

Таблица В.4 — Создаваемое при установке

Путь	Чем создаётся
`.env`	`make init-production`
`nginx/certs/`	`scripts/init-tls.sh`
`nginx/acme-webroot/`	`scripts/init-tls.sh` при выпуске через общедоступный центр

Вне каталога системы создаются ещё два: каталог резервных копий (по умолчанию /var/backups/systemburo, подраздел 11.4) и каталог состояния проверки живости (по умолчанию /var/lib/systemburo/health, подраздел 12.2). Оба задаются параметрами и в поставку не входят.

Приложение Г. Проверочный лист ввода в эксплуатацию

Шаги идут в том порядке, в котором их выполняют: лист читается сверху вниз и служит одновременно порядком работ и приёмкой. Столбец «Где описано» указывает подраздел с командами и ожидаемым результатом - лист не заменяет их, а связывает в последовательность. Отметка ставится, когда шаг выполнен и проверен, а не когда команда отработала без ошибки: разница видна в подразделе 5.9.

Порядок не произволен. Учётную запись сопровождения и доступ к репозиторию заводят прежде установки: система ставится уже под ту запись, которая будет ею управлять. Ключи шифрования задают до первого запуска - иначе часть данных ляжет на диск открытой и потребует пересборки. Резервное копирование настраивают до того, как в системе появятся рабочие данные, а не после.

Этап 1. Подготовка сервера

Таблица Г.1 — Проверочный лист, этап 1

№	Шаг	Где описано	Отметка
1	Операционная система обновлена, часовой пояс задан	5.1	
2	Дисковый массив собран в зеркало	3.2	
3	Заведена отдельная учётная запись сопровождения: в группу `docker` она не входит, команды выполняются через `sudo` с вводом пароля	5.1, 5.10	
4	Вход на сервер выполняется по ключу, парольный вход отключён и проверен	5.10	
5	Межсетевой экран включён, снаружи открыты только порты 22, 80 и 443	5.10, приложение А	
6	Защита от подбора пароля к серверу установлена, состояние правила проверено	5.10	
7	Docker и Docker Compose установлены, версии соответствуют требованиям	3.3, 5.2	
8	Сборщик BuildKit установлен: `docker buildx version` отвечает версией	3.3, 5.2	
9	Ограничение размера журналов контейнеров настроено	12.3	

Этап 2. Установка и первый запуск

Таблица Г.2 — Проверочный лист, этап 2

№	Шаг	Где описано	Отметка
10	Доступ сервера к репозиторию исходного кода выдан и проверен, закрытая часть ключа с сервера не покидала	4.1, 5.3	
11	Установлено то, что передал разработчик: выпущенная версия по метке, а при отсутствии меток - записанный определитель изменения из `git log -1`	5.3	
12	Файл параметров создан, секреты переданы в хранилище организации	5.4	
13	Ключ шифрования персональных данных сохранён отдельно от сервера и от резервных копий	5.4	
14	Требование шифрования персональных данных включено	6.2, 6.3	
15	Имя базы данных задано до первого запуска	5.5	
16	Публикация описания программного интерфейса отключена	6.3	
17	Сертификат выпущен, срок действия проверен	5.6, 7.3	
18	Корневой сертификат установлен на рабочие места, если применялся собственный центр	7.4	
19	Система запущена, все службы работают	5.7	
20	Учётная запись администратора создана, пароль отличается от исходного	5.8	
21	Демонстрационные данные на рабочем сервере отсутствуют	5.8, 8.4	

Продолжение таблицы Г.2

№	Шаг	Где описано	Отметка
22	Проверка состояния отвечает	5.9	
23	Обращение по незащищённому протоколу перенаправляется	5.9	
24	Система открывается с рабочего места по тому адресу, по которому ею будут пользоваться, и перенаправление не зацикливается	5.9, 7.6	
25	Вход выполняется, разделы открываются	5.9	

Этап 3. Обвязка и наблюдение

Таблица Г.3 — Проверочный лист, этап 3

№	Шаг	Где описано	Отметка
26	Почта настроена, ящику разрешена отправка, проверочное письмо получено; записи SPF и DKIM внесены	6.7	
27	Каталог файлового архива создан, владелец - пользователь прикладного сервера	9.6	
28	Выгрузка бланков включена: `deploy-archive ARGS="show"` это подтверждает	9.6	
29	Ключ шифрования копий задан, закрытая часть унесена в хранилище секретов	11.4	
30	Копирование настроено, копия снимается по расписанию	11.4	
31	Восстановление из копии выполнено и проверено	11.5, 11.7	
32	Мониторинг подключён	12.1, 12.5	
33	Проверка живости на расписании, письмо об отказе получено	12.2	

ЛИСТ РЕГИСТРАЦИИ ИЗМЕНЕНИЙ

В таблице фиксируются изменения документа. Версия повышается при выпуске новой редакции: вторая цифра при уточнениях и дополнениях, первая при переработке структуры или существенном изменении требований.

Таблица И.1 — Регистрация изменений

Версия	Дата	Содержание изменения	Внёс
1.0	28.07.2026	Первая редакция	
1.1	28.07.2026	Сверка с реализацией: уточнены требования к версии Docker Compose и порядок развёртывания стенда	
1.2	28.07.2026	Дополнен порядок работы с режимом технических работ: вход супер-администратора и аварийное снятие командой	
1.3	31.07.2026	Описана команда очистки накопленных данных и сроки хранения по группам	
1.4	31.07.2026	Уточнено, что сроки автоматической уборки задаются в описании службы, а не в файле параметров	
1.5	31.07.2026	Описаны обзор занятого места по таблицам, исключение групп из очистки и оговорка о том, что удаление строк не возвращает место операционной системе	
1.6	31.07.2026	Описано сужение очистки внутри группы: отрезок времени, тип сущности в журнале действий и отдельный пост для суточных слепков	
1.7	31.07.2026	Описано резервное копирование: состав копии, сроки хранения, настройка, проверка восстановимости и порядок восстановления	
1.8	31.07.2026	Добавлен способ создать ключи шифрования копий без установки средства на сервер	
1.9	31.07.2026	В разделе о копировании указано, где именно выполняются команды, и добавлен разбор причин сбоя по сообщениям журнала	
1.10	31.07.2026	Дополнены таблицы сообщений копирования: шаги файлов и архива бланков, сообщения проверки восстановимости	
1.11	31.07.2026	Описана подготовка каталога файлового архива бланков на сервере	
1.12	31.07.2026	Исправлены ссылки на подраздел об очистке накопленных данных	
1.13	01.08.2026	Уточнён состав пары ключей шифрования копий и порядок переноса закрытого ключа; в примерах восстановления указаны контуры и способ выбрать копию	

Продолжение таблицы И.1

Версия	Дата	Содержание изменения	Внёс
1.14	01.08.2026	Разобраны случаи прерванного восстановления и порядок ручного запуска после него; описана метка в имени копии	
1.15	01.08.2026	Описан ход восстановления: нумерация шагов, время выполнения и полоса объёма	
1.16	01.08.2026	Единое правило хранения секретов и порядок действий при их утрате	
1.17	01.08.2026	Файловый архив: фоновое наполнение и ночная сверка, раздел администрирования и выгрузка за период; параметры архива внесены в приложение Б	
1.18	01.08.2026	Лист регистрации изменений разделён по документам: у каждого своя редакция и свой перечень изменений	
1.19	02.08.2026	Описано восстановление давней копии на текущую версию системы и порядок действий, когда проверка восстановимости не прошла	
1.20	02.08.2026	Перечень копий для восстановления смотрят по состоянию копирования: дата словами, размер и метка вместо машинного имени файла	
1.21	03.08.2026	Добавлен подраздел о выдаче каталога архива по сети; уточнены пороги места и остановка очереди выгрузки, пределы восстановления удалённых файлов и настоящая причина запрета вкладывать архив в каталог загруженных файлов; добавлен разбор сбоев выгрузки из архива	
1.22	04.08.2026	Настройка файлового архива выполняется целью make наравне с прочими командами обслуживания; описан возврат каталога архива из резервной копии; уточнены сводка раздела администрирования и порядок изменения предела одной выгрузки	
1.23	04.08.2026	Добавлен разбор состояний выгрузки бланка: что означает каждое и что по нему делать дежурному	
1.24	04.08.2026	Добавлен разбор случая, когда бланки не появляются в каталоге архива	
1.25	04.08.2026	Указано, чем автоматическое обновление стенда отличается от ручного развёртывания со сборкой образов на сервере	
1.26	05.08.2026	Приведено в соответствие описание раздела файлового архива: вкладки обзора и ленты записей, состав показателей, отбор строк по состоянию	
1.27	05.08.2026	Уточнено, что панель управления базой данных на стенде поднимается по требованию отдельной командой	
1.28	05.08.2026	По итогам первого включения архива на стенде: владелец каталога, созданного без указания пути, сроки повтора после устранения причины, признак запрета доступа в ленте, предел длины пути при выборе шаблона	

Продолжение таблицы И.1

Версия	Дата	Содержание изменения	Внёс
1.29	05.08.2026	Состояния выгрузки названы так же, как в ленте архива; отмечено, что образы убираются при выпуске новой версии	
1.30	06.08.2026	Описано наполнение проверочного стенда вымышленными данными: отметка экземпляра, заготовки объёма, партии и их удаление	
1.31	06.08.2026	Описано ограничение доступа к проверочному стенду: устройство запроса имени и пароля, адреса, отвечающие без него, срок действия однажды введённого пароля, смена пары, отключение и обратное включение; параметры доступа к стенду добавлены в приложение Б, сценарий создания файла паролей - в приложение В	
1.32	06.08.2026	Приложение параметров дополнено настройками файлов, прикладываемых к заявке	
1.33	06.08.2026	Описан срок хранения непрочитанных уведомлений и добавлен задающий его параметр	
1.34	07.08.2026	Приложение параметров дополнено ключами доставки уведомлений вне системы и сроком хранения подписок устройств; уточнены пределы файлов заявки	
1.35	07.08.2026	Описано включение шифрования файлового архива: создание ключей, порядок хранения, чтение файлов и шифрование раздела	
1.36	07.08.2026	Описан состав папки заявки в архиве: бланки, машиночитаемое описание и приложенные файлы	
1.37	08.08.2026	Описан перевод ранее записанных файлов архива в шифрованный вид и распространение запрета запуска без ключей на файловый архив	
1.38	08.08.2026	Уточнено, что через систему файлы архива скачиваются расшифрованными: ключ нужен только при чтении каталога напрямую	
1.39	08.08.2026	Приложение параметров: перечень ограничений и файлов разделён на обращения, файлы заявки и файловый архив	
1.40	09.08.2026	Перечень копий дополнен составом копии и её архивами; описан просмотр состояния стороннего каталога копий	
1.41	10.08.2026	Добавлены гриф конфиденциальности и лист ограничений на использование: порядок развёртывания и параметры установки не подлежат передаче третьим лицам	
1.42	11.08.2026	Наполнение стенда: отметки прохода описаны отдельно от создаваемых записей, добавлено согласие на обработку данных у заводимых работников	
1.43	11.08.2026	Копирование без ключа шифрования описано как отказ, а не предупреждение; добавлен параметр явного разрешения незашифрованных копий	

Продолжение таблицы И.1

Версия	Дата	Содержание изменения	Внёс
1.44	11.08.2026	Описано получение исходного кода из репозитория: адрес, выдача и отзыв доступа, порядок для площадки без доступа в сеть общего пользования; перечни поставки дополнены сценариями резервного копирования и восстановления; описаны набор параметров рабочего сервера и права на файл параметров	
1.45	11.08.2026	Уточнено, что число отметок прохода в предварительном показе наполнения стенда приблизительное и может оказаться больше	
1.46	11.08.2026	Описана настройка почтовой рассылки: подключение к почтовому серверу поставщика услуг или организации, передача параметров прикладному серверу, проверочное письмо, записи SPF и DKIM у домена отправителя, исходящий порт и пределы отправки; добавлен разбор случая, когда письма не приходят; параметры почты внесены в приложение Б, исходящие соединения - в приложение А	
1.47	11.08.2026	Описана плановая смена паролей в разделе эксплуатации: включение и настройки, показания блока состояния, ручной прогон с обрывом сеансов работников, учётные записи без адреса почты, смена пароля отдельному работнику; добавлен разбор случая, когда пароли не меняются	
1.48	11.08.2026	В управление учётными записями добавлен вход в систему от имени работника: как войти и вернуться, срок сеанса, что в режиме закрыто и почему он не заменяет смену пароля	
1.49	11.08.2026	Переписана эксплуатация срока действия паролей: по истечении срока система требует смены при входе и ничего не рассылает, почта для этого не нужна; обновление паролей всем работникам и смена из карточки описаны как исключительные действия; разделена диагностика этих двух случаев	
1.50	12.08.2026	Описано заведение учётной записи: пароль придумывает система и высылает письмом, свой пароль работник задаёт при первом входе; описан запрет повторного использования паролей	
1.51	11.08.2026	Описано ограничение доступа к серверу при установке: межсетевой экран и особенность портов, опубликованных контейнерами, защита от подбора пароля и переход на вход по ключу; проверочный лист дополнен соответствующими пунктами	

Продолжение таблицы И.1

Версия	Дата	Содержание изменения	Внёс
1.52	11.08.2026	Модель доступа на сервере приведена к фактической: учётная запись оператора без членства в группе docker, разрешение на две команды через sudo, исключение для каталога архива и для системы контроля версий; команды документа приведены к этой модели, названы границы даваемой ею защиты	
1.53	11.08.2026	Описан запрос подтверждения перед командами рабочего сервера, ломающими данные, и границы даваемой им защиты; описан доступ оператора к состоянию копирования без полных прав, включая случаи отсутствующей группы и отключённого доступа; в перечень параметров добавлена группа оператора копирования	
1.54	12.08.2026	В приложение Б внесены параметры пула соединений с базой данных и сроков ожидания прикладного сервера	
1.55	12.08.2026	Описаны параметры работы под нагрузкой: пул соединений с базой и его соотношение с пределом базы, таймауты соединения, ограничение одновременных проверок пароля; описана проверка живости системы с оповещением на почту, её постановка на расписание и границы того, что она видит; описана сборка архива поставки	
1.56	12.08.2026	Описаны работа с данными одной организации командой entity и создание ключей уведомлений в браузер	
1.57	12.08.2026	Параметры файла настроек действуют все, кроме служебных: перечень набора дополнен пулом соединений, сроками ожидания, проверкой пароля, почтой и сроками хранения; снята оговорка о недействующих строках. Добавлено предупреждение о правах на копии, снятые до включения доступа оператора	
1.58	12.08.2026	Описание работы с данными одной организации доведено до реализации: перечень подкоманд и их ключей, обязательность проверенного и зашифрованного пакета для сноса, порядок гашение-снятие-проверка-снос, отвязка общих шаблонов отчётов, данные, до которых обезличивание не дотягивается, подготовка каталога пакетов и его параметры, порядок ухода с системы целиком	
1.59	12.08.2026	Перечень групп очистки накопленных данных приведён к действующему: добавлены непрочитанные уведомления и подписки устройств на доставку вне системы, уточнён параметр автоматической уборки	
1.60	12.08.2026	Описано, что запрет повторного использования паролей действует сразу после обновления; уточнено, что при запуске выполняются разовые правки данных	

Продолжение таблицы И.1

Версия	Дата	Содержание изменения	Внёс
1.61	13.08.2026	Описано выравнивание прав каталога резервных копий при включении доступа оператора на уже работающей установке: подкаталоги сроков хранения и файлы копий, оставшиеся более открытыми, чем предусмотрено защитой, установка закрывает сама и показывает в выводе, сколько записей выровняла; повторный запуск ничего не переставляет	
1.62	13.08.2026	В состав поставки добавлен сценарий самопроверки выравнивания прав каталога копий: описано, что он проверяет во временной песочнице, не трогает рабочий сервер и не входит в механизм копирования	
1.63	13.08.2026	По итогам установки на чистый сервер: в требования и в состав устанавливаемых пакетов добавлены git, age, средство межсетевого экрана и сборщик BuildKit, без которого поставка не собирается; описаны выбор выпущенной версии при получении кода, случай уже установленного Docker, права на создание каталога установки, запуск подготовки параметров без age; дополнены правила пересчёта параметров базы; описаны работа за прокси поставщика услуг, продление готового сертификата, разрешение ящику на отправку и применение правки файла, отданного контейнеру	
1.64	14.08.2026	По итогам повторной установки на голый сервер: описано, что делать, когда меток версий в репозитории нет, и уточнён соответствующий пункт проверочного листа	
1.65	14.08.2026	По итогам прогона операций сопровождения на рабочем сервере: предупреждение, что необратимый снос данных организации не спрашивает подтверждения; выключенная после установки выгрузка бланков и новый пункт проверочного листа; кеш сборки образов как крупнейший потребитель места; поведение местных правок файлов описания при смене версии; уточнён вывод команды снятия режима технических работ и порядок допуска изменения к выкладыванию	
1.66	14.08.2026	Необратимый снос данных организации закрыт подтверждением: команда спрашивает слово СНЕСТИ, перечень подтверждаемых команд дополнен	
1.67	14.08.2026	Добавлен разбор случая, когда получение кода перестаёт работать после команд git от суперпользователя: причина в смене владельца служебных файлов и возврат владельца	

Продолжение таблицы И.1

Версия	Дата	Содержание изменения	Внёс
1.68	14.08.2026	Лист условий использования сокращён и смягчён; проверочный лист ввода в эксплуатацию выстроен по порядку работ в три этапа, у каждого шага указан подраздел с командами	
1.69	14.08.2026	Свойства файла заполнены выходными данными документа: прежде в них стояли служебные значения средства сборки	
1.70	17.08.2026	Описаны сторонние компоненты и условия их использования: перечень в составе поставки, обязанность передавать его дальше вместе с системой, лицензия шрифта рядом с файлом шрифта	
1.71	19.08.2026	Описание методов на стенде закрыто общим паролем; уточнено, что признак доступа выводится из имени и пароля и обесценивается их сменой; поправлено обоснование запрета вложенных каталогов загрузки и архива	
1.72	20.08.2026	Журнал запросов указан там, где он на самом деле находится, - в группе «Аудит и связь» раздела администрирования; описаны формат его выгрузки, потолок в 10 000 строк с оговоркой об обрезке и запись выгрузки в журнал изменений	
1.73	29.08.2026	Описан порядок хранения ключа шифрования персональных данных, сверка ключа при запуске и смена ключа командой перевода данных	
1.74	14.10.2026	Уточнены обезличивание и хранение, параметры конфигурации, доставка JS/CSS, проверка стенда и предупреждения резервных копий.	
1.75	21.10.2026	Актуализированы параметры и доверие прокси, ограничения файлов и хранение, доступ к архиву, сессии и проверки поставки; удалены настройки отключённой интеграции.	
1.76	30.10.2026	Описана временная автоматическая доставка трёх образов стенда через GHCR по одной ревизии, проверка фактической поставки и возврат готового комплекта; ручная сборка рабочего сервера сохранена. Исправлено распознавание разрывов таблиц при сборке документа.	
1.77	05.11.2026	Описаны готовые образы точной ревизии, проверка ключа SSH-узла, ограниченный допуск операций, диагностика, устойчивые задания и сопровождение pgAdmin; уточнены нейтральные уведомления, допуск тел запросов, проверка deploy-ключа и совместимость восстановления.	
1.78	08.11.2026	Уточнён порядок восстановления с сохранением журнала удалений и повтором после сбоя.	
1.79	10.11.2026	Описаны проверка комплекта для независимого восстановления и смена писателя журнала после восстановления.	

Продолжение таблицы И.1

Версия	Дата	Содержание изменения	Внёс
1.80	10.11.2026	Уточнены сообщения проверки восстановимости и назначение общего сценария контрольных данных копии.	

Окончание таблицы И.1

Версия	**Дата**	**Содержание изменения**	**Внёс**